

На правах рукописи

Артур Игоревич ЕФИМОВ

**ПОСТРОЕНИЕ КОМПИЛЯТОРА
С ЯЗЫКА ПРОГРАММИРОВАНИЯ ОБЕРОН
ДЛЯ ПРОЦЕССОРА ARM64**

22.04.00 — программное обеспечение вычислительной
техники и автоматизированных систем

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

СОДЕРЖАНИЕ

Аннотация	3
Автореферат	6
Введение	8
Глава 1. Анализ на предварительном этапе	10
1.1. Текущие тенденции в компиляторостроении .	10
1.2. Актуальное состояние компиляторов языка Оберон и их архитектурная поддержка	13
1.3. Подготовительный этап: раскрутка компиля- тора и исправление в Ofront+	15
1.4. ARM64 как целевая архитектура	16
Глава 2. Исходный язык	18
2.1. Диалекты языка Оберон	19
2.1.1. Оберон (1990)	19
2.1.2. Оберон-2	19
2.1.3. Компонентный Паскаль	20
2.1.4. Пересмотренный Оберон (Оберон-07) . . .	21
2.1.5. Пересмотренный Оберон-2	22
2.2. Сравнение синтаксиса Оберона с другими по- пулярными языками	23
Глава 3. Особенности программирования на Обероне	25
3.1. Влияние языка на стиль программирования .	25
3.2. Отсутствие префиксов в именах объектов . .	27
3.3. Регистр букв как способ маркировки иденти- фикаторов	31
3.4. Передача управления из процедуры	33
3.5. Отсутствие процедур с переменным количе- ством параметров	35
3.6. Расширение записных типов	39
3.7. Цикл Дейкстры: WHILE с ветвями	42
3.8. Отсутствие перечисляемых типов	44
3.9. Обязательный END	48
Глава 4. Общая модульная структура компилятора	52
Глава 5. Загрузка текста	55
5.1. Программный модуль как единица компиляции	56
5.2. Модуль Machine	56
Глава 6. Лексический анализатор	57
Глава 7. Нисходящий рекурсивный синтаксический анализатор	61
7.1. Модуль Builder	70

Глава 8. Проверка типов	71
Глава 9. Промежуточное представление программы	73
9.1. Таблица символов	73
9.2. Абстрактное синтаксическое дерево	76
Глава 10. Генератор кода	80
Глава 11. Компоновщик	81
11.1. Использование типа SET	87
Глава 12. Особенности архитектуры Apple Silicon .	88
12.1. Инструкции, регистры и соглашения	89
12.2. Особенности системных вызовов на macOS .	90
Глава 13. Исследование формата файла Mach-O .	91
Заключение	95
Список использованной литературы	96
Приложение А. Синтаксис Оберона	104
Приложение Б. Исходный код компилятора	106
Модуль Moc (главный модуль)	106
Модуль Machine (загрузка текста)	108
Модуль Scanner (лексический анализатор)	112
Модуль Parser (синтаксический анализатор)	120
Модуль Table (таблица символов)	132
Модуль Tree (абстрактное синтаксическое дерево)	140
Модуль Builder (построитель абстрактного синтак-	
сического дерева)	142
Модуль Traverser (обходчик абстрактного синтак-	
сического дерева)	146
Модуль Emitter (испускатель машинного кода) . .	148
Модуль Generator (низкоуровневый генератор) . .	150
Модуль Linker (компоновщик)	154
Модуль MocErrors (коды и текст ошибок)	167

АННОТАЦИЯ

Данная работа посвящена разработке компилятора языка программирования Оберон (последователя языка Паскаль), предназначенного для преобразования программ в машинный код современных процессоров. В качестве целевой платформы выбран процессор Apple Silicon (ARM64) под управлением операционной системы macOS.

Основной целью исследования является создание инструмента, позволяющего компилировать исходный код, написанный на языке Оберон, в объектный код, совместимый с современными процессорами архитектуры ARM64. В работе предусматривается рассмотрение и реализация всех основных этапов трансляции программного кода: от лексического и синтаксического анализа и проверки совместимости типов до генерации промежуточного представления, машинного кода и компоновки исполняемого файла. Особое внимание уделяется специфике архитектуры современных ARM64-процессоров и проблемам, связанным с адаптацией особенностей языка Оберон к низкоуровневой платформе. Также подчёркивается важность стиля и ясности исходного кода, поскольку сам компилятор разрабатывается на языке Оберон.

Итогом работы является функционирующий прототип компилятора, который может быть использован для дальнейшего развития и исследования потенциала языка Оберон в современных вычислительных системах.

Ключевые слова: компилятор, Оберон, Apple, ARM64, машинный код, Компонентный Паскаль.

ABSTRACT

Compiler Construction for the Language Oberon Targeting ARM64 Machine Code

This thesis presents the design and implementation of a compiler for the Oberon programming language, a successor of Pascal, targeting modern processor architectures. The chosen platform is Apple Silicon (ARM64) running the macOS operating system.

The primary goal of this research is to develop a toolchain capable of translating Oberon source code into object code compatible with contemporary ARM64 processors. The work encompasses all major stages of compilation, including lexical and syntactic analysis, type checking, intermediate code generation, machine code emission, and executable linking. Particular emphasis is placed on addressing the architectural specifics of ARM64 and the challenges associated with adapting the high-level semantics of Oberon to a low-level execution environment. The clarity and structure of the source code are also highlighted, as the compiler itself is implemented in Oberon.

The outcome of this thesis is a functioning compiler prototype, providing a foundation for future development and further investigation into the applicability of the Oberon language in modern computing environments.

Keywords: compiler, Oberon, Apple, ARM64, machine code, Component Pascal.

ANOTĀCIJA

Kompilatora izveide Oberona valodai ARM64 procesora mašīnкодā

Darbs ir veltīts programmēšanas valodas Oberons (Paskāļa sekotāja) kompilatora izstrādei, lai pārveidotu to par mūsdienu procesoru mašīnкодu. Izvēlētais procesors ir *Apple Silicon* (ARM64) ar operētājsistēmu *macOS*.

Pētījuma galvenais mērķis ir izveidot instrumentu, kas ļauj kompilēt Oberonā rakstīto pirmkodu objektкодā, kas ir saderīgs ar mūsdienu ARM64 arhitektūras procesoriem. Darbā paredzēts apskatīt un realizēt visus galvenos programmēšanas valodas tulkošanas posmus: sākot no leksiskās un sintaktiskās analīzes un tipu saderības pārbaudes līdz starpreprezentācijas koda un mašīnкодā ģenerēšanai un izpildāmā faila sastatīšanai. Īpaša uzmanība tiks pievērsta mūsdienu ARM64 arhitektūras procesora specifikai un problēmām, kas saistītas ar Oberona valodas īpašību pielāgošanu zema līmeņa platformai. Tāpat tiks uzsvērtā pirmkoda stila un skaidrības nozīme, jo pats kompilators tiks rakstīts Oberona valodā.

Darba galarezultāts ir funkcionējošs kompilatora prototips, kuru varēs izmantot turpmākai attīstībai un Oberona valodas potenciāla izpētei mūsdienu skaitļošanas sistēmās.

Atslēgas vārdi: kompilators, Oberons, Apple, ARM64, mašīnкодs, Komponentais Paskālis.

АВТОРЕФЕРАТ

Актуальность исследования. Язык программирования Оберон сочетает компактность, строгую типизацию и высокую систематичность, что обеспечивает надёжность и предсказуемость программ. Однако на платформе Apple Silicon (ARM64) нативный компилятор Оберона отсутствует, а существующие решения (трансляторы в Си или LLVM) не обеспечивают желаемой скорости компиляции и прозрачности преобразования программ. В связи с этим разработка нативного компилятора Оберона для архитектуры ARM64 является актуальной задачей.

Цель работы. Разработать полноценный нативный компилятор Оберона, способный преобразовывать исходный код напрямую в машинный код ARM64 в формате Mach-O под операционную систему macOS.

Научная новизна

1. Создан прототип нативного компилятора с языка Оберон, непосредственно генерирующий машинный код ARM64 в формате Mach-O без использования сторонних IR-фреймворков (LLVM) или генерации кода на Си.
2. Предложены методы адаптации высокоуровневых конструкций Оберона к особенностям Apple Silicon, включая правильную передачу вещественных параметров через VFP-регистры.
3. Реализована полная прозрачность процесса компиляции на современной вычислительной платформе: все этапы трансляции (лексика, синтаксис, семантика, промежуточное представление, кодогенерация, компоновка) доступны для изучения и модификации на том же языке программирования, который одновременно является и предметом компиляции.
4. Улучшена модульная архитектура компилятора по сравнению с существующими версиями.
5. Разработан компоновщик исполнимых файлов в формате Mach-O.

Практическая значимость

1. Разработанный инструмент может служить наглядным учебным материалом при освоении технологий построения компиляторов, а также машинного кода ARM64.

2. Компилятор готов к дальнейшему развитию и перенацеливанию на другие вычислительные системы, а исходные тексты — к включению в открытые образовательные и исследовательские проекты.

Структура работы. Работа состоит из введения, 13 глав, заключения, списка литературы и приложений. Основные главы посвящены предварительному анализу, описанию исходного языка, особенностям программирования на Обероне, проектированию и реализации всех этапов компиляции, адаптации к архитектуре ARM64 и формату Mach-O.

ВВЕДЕНИЕ

Язык программирования Оберон, разработанный Никлаусом Виртом [1], — это компактный строго типизированный язык, предназначенный для создания как прикладных, так и системных программ. Он применяется при разработке операционных и встроженных систем, драйверов и компиляторов, а также поддерживает построение расширяемых объектно-ориентированных программных систем без нарушения строгой типизации [2]. Оберон зарекомендовал себя при разработке программно-аппаратных комплексов с повышенными требованиями к надёжности. Он нашёл применение в цифровых системах атомных электростанций [3] и электrorаспределительных подстанций [4], агропромышленном оборудовании [5], беспилотных летательных аппаратах [6, 7], железнодорожных и авиационных системах [8] и даже в имплантируемых медицинских устройствах [9, 10, 11, 12].

Благодаря лаконичному синтаксису и высокой степени систематичности¹ язык применяется в образовательных проектах [13, 14, 15]. Строгая типизация, включая межмодульный контроль, способствует формированию у студентов профессионального стиля программирования. Дополнительным преимуществом является высокая скорость компиляции: для таких систем, как *BlackBox Component Builder* [16, 17], время компиляции проектов из десятков тысяч строк кода измеряется миллисекундами.

Тем не менее, Оберон не получил столь же широкого распространения, как его предшественник Паскаль, что, по-видимому, связано не с техническими особенностями, а с коммерческими и маркетинговыми факторами. Вследствие этого количество полноценных компиляторов Оберона ограничено, а поддержка новых аппаратных платформ осуществляется эпизодически. Кроме того, существует проблема наличия нескольких не вполне совместимых диалектов языка, затрудняющих стандартизацию и развитие единой экосистемы.

Одна из таких новых аппаратных платформ — процессоры Apple Silicon, основанные на архитектуре ARM64. Для этой

¹ Систематичность языка Оберон обеспечивается, в частности, высокой степенью ортогональности его конструкций, т. е. как правило, для каждой программной идеи существует один оптимальный способ выразить её. Из языка исключены избыточные и дублирующие элементы, оставлен только минимально необходимый набор.

платформы нет нативного компилятора¹ языка Оберон. Трансляторы Оберона в Си и другие промежуточные языки обладают тем существенным недостатком, что компиляция происходит медленно — то есть теряется одно из существенных преимуществ данного языка.

Таким образом, создание нативного компилятора языка Оберон для платформы Apple Silicon не только актуально, но и имеет стратегическое значение. Разработка такого компилятора позволит существенно повысить скорость компиляции программ на Обероне на устройствах с архитектурой ARM64, расширить применение языка в разработке современного программного обеспечения и предоставить научному и образовательному сообществу надёжный и перспективный инструмент. Кроме того, исходный код компилятора может быть использован в качестве наглядного приложения к учебным материалам по построению компиляторов.

Первый компилятор Оберона, совместно с одноимённой операционной системой и ЭВМ «Ceres» на базе процессора NS32000, был разработан Н. Виртом в 1988—1990 гг. Последняя реализация в рамках «Проекта Оберон» [18, 19] также принадлежит Вирту и создана в 2013—2016 гг. для специально разработанного RISC-процессора [20], функционирующего на ПЛИС (FPGA).

Целью данной работы является применение теории построения компиляторов, развитой в проекте Оберон (1988—2016) [1], к современной аппаратной платформе.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести исследование существующих подходов и методов создания компиляторов языка Оберон, а также выявить их преимущества и недостатки с точки зрения использования на платформе Apple Silicon.

2. Проанализировать архитектурные особенности процессоров Apple Silicon с точки зрения генерации машинного кода, в том числе исследовать формат исполнимых файлов, использующийся на платформе macOS.

3. Спроектировать архитектуру нового компилятора: очертить круг его функционала и разбить его на программные модули, включая лексический и синтаксический анализаторы, систему учёта контекста, внутреннее представление кода, генерацию кода и компоновку (linking).

¹ Здесь и далее под нативным компилятором понимается компилятор, преобразующий исходный код на языке высокого уровня непосредственно в машинный код аппаратной платформы.

4. Реализовать модули лексического и синтаксического анализа языка Оберон, а также механизмы семантического анализа и проверки типов.

5. Разработать эффективные методы генерации нативного машинного кода для архитектуры ARM64.

6. Реализовать компоновщик (linker) или механизмы интеграции порождаемых компилятором объектных файлов в существующую систему сборки приложений macOS.

7. Провести испытания компилятора на примерах типовых задач, продемонстрировав таким образом корректность его работы.

8. Подготовить набор учебных примеров и рекомендаций по использованию компилятора.

Реализация поставленных задач позволит продемонстрировать потенциал языка Оберон на платформе macOS и станет значимым шагом на пути его интеграции в образовательную, научную и инженерную практику.

Глава 1

АНАЛИЗ НА ПРЕДВАРИТЕЛЬНОМ ЭТАПЕ

1.1. Текущие тенденции в компиляторостроении

В современной практике построения компиляторов наблюдается всё более широкое использование готовых инструментов. Целью компилятора в сущности является перевод программы с одного языка на другой — с языка программирования высокого уровня в низкоуровневый машинный код. Эта цель определяет и два противоположных момента компилятора: *препроцессор* (frontend), связанный с исходным кодом, и *постпроцессор* (backend), связанный с целевой платформой. Даже если создатель компилятора не подразумевает явного разделения его на две указанные составные части, в нём всё равно с лёгкостью можно выделить эти противоположности: в виде разных наборов программных модулей, процедур или строк кода. Оба этих момента находят отражение в многообразии инструментов для решения задач компиляторостроения: с одной стороны, со стороны препроцессора, существуют генераторы лексических

и синтаксических анализаторов. С другой стороны, со стороны постпроцессора, существуют фреймворки, обеспечивающие генерацию машинного кода, такие как LLVM.

Утилита `ge2c` — пример генератора *лексического* анализатора [21]. Она генерирует код на языке Си. Распространённым средством, позволяющим генерировать исходный код *синтаксического* анализатора, является утилита Yacc. Более новый её аналог — GNU Bison — использован для создания синтаксического анализатора языка PHP [22]. В качестве входных данных такие генераторы получают формальное описание синтаксиса, а на выходе мы имеем абстрактное синтаксическое дерево, готовое для обработки постпроцессором компилятора.

Готовые форматы промежуточных представлений и связанные с ними генераторы машинного кода, в том числе включающие в себя этап оптимизации, с одной стороны, позволяют сократить сроки разработки компилятора, с другой — порождают сложности при адаптации к специфическим целям. В частности, для образовательных и исследовательских целей такой компилятор вряд ли подходит, ведь львиная доля его функционала скрыта в утилите-генераторе.

Одним из самых известных фреймворков компиляции является LLVM [23]. Он предоставляет как промежуточное представление, так и набор инструментов для его оптимизации и генерации машинного кода. LLVM определяет собственную виртуальную архитектуру и язык ассемблера. Компиляторы, использующие LLVM, преобразуют исходный код в промежуточное представление LLVM, а затем используют инструменты LLVM для генерации кода под целевую платформу.

Это обеспечивает высокий уровень переиспользуемости и доступ к сложным оптимизациям. Однако, как показал опыт ряда проектов [24], интеграция с LLVM может оказаться обременительной из-за:

- 1) высокой сложности API и документации;
- 2) большого объёма зависимостей и сборочной системы;
- 3) трудностей с отладкой IR и промежуточных фаз;
- 4) сравнительно низкой скорости на этапе генерации кода.

Для таких языков, как Оберон, где критически важна прозрачность всех этапов трансляции¹, интеграция с LLVM представляется избыточной и неоправданной. В данной работе был

¹ Прозрачность компилятора приобретает особое значение в случае языков с минималистичным синтаксисом и строго определённой семантикой. Простота языка представляет собой самостоятельную ценность, выражающуюся в надёжности и предсказуемости вычислительного процесса: программист способен объять умом весь процесс компиляции и исполнения программы. Эта ценность легко нивелируется при использовании громоздких промежу-

сознательно выбран путь прямой генерации машинного кода без обращения к сторонним промежуточным представлениям. Такой подход позволил значительно упростить архитектуру компилятора, обеспечить полный контроль над структурой выходного файла и сделать все этапы трансляции доступными для понимания без наличия «чёрных ящиков».

Альтернативный путь — генерация кода на языке Си. Такой подход использовался, например, в компиляторах языков Эйфель, Модула-3 и некоторых реализациях Оберона. Язык Си в качестве промежуточного представления обеспечивает переносимость (хоть и с некоторыми оговорками), но оставляет разработчику лишь ограниченный контроль над машинными особенностями, затрудняет оптимизацию под конкретные архитектуры и замедляет процесс компиляции, ведь теперь в качестве завершающего этапа компиляции необходимо запускать компилятор Си.

При всей своей автоматизации генераторы вроде Yacc и Bison обладают рядом ограничений, делающих их непривлекательными для реализации компиляторов языка Оберон:

1. Язык изначально проектировался с учётом лёгкости разбора. Его грамматика идеально подходит для синтаксического анализа методом рекурсивного спуска — подхода, в котором каждый нетерминал грамматики реализуется как отдельная процедура, тело которой по определённым правилам выводится из описания этого нетерминала в РБНФ¹.

2. Сделанный вручную синтаксический анализатор позволяет точно контролировать структуру и логику разбора, что особенно важно в образовательном или исследовательском контексте. Автоматически сгенерированный код Bison, напротив, зачастую содержит плохо читаемую структуру с множеством `goto`, переходов по состояниям и управляющих конструкций низкого уровня.

3. Как подробно показано в «Дисциплине программирования» Э. Дейкстры [25], обширное использование `goto` приводит к усложнению логики программы с точки зрения её структурности и к невероятному усложнению анализа корректности и верифицируемости такой программы. В проектировании современных программ, и компиляторов в том числе, стремятся к чистому стилю, и поэтому не используют переходов `goto`, чтобы не нарушать предсказуемости потока управления программы.

точных этапов, таких как LLVM, скрывающих логику преобразования за слоями абстракции.

¹ РБНФ — расширенная Бэкус — Наурова форма (Extended Backus—Naur Form, EBNF).

Кроме того, если анализаторы и генераторы будут написаны самостоятельно, мы вольны выбирать язык программирования, на котором мы будем это делать. Логичным шагом представляется писать компилятор Оберона на самом Обероне. В этом случае, изучая данную работу, будущие исследователи смогут изучить и «прочувствовать» язык Оберон в процессе чтения исходного кода компилятора, что устранил любое недопонимание характера и особенностей компилируемого языка.

1.2. Актуальное состояние компиляторов языка Оберон и их архитектурная поддержка

Поскольку целью настоящей работы является реализация компилятора языка Оберон с поддержкой платформы Apple Silicon, необходимо провести обзор существующих компиляторов Оберона, оценить их применимость, поддерживаемые архитектуры и модели исполнения. Особый интерес представляет наличие или отсутствие нативной поддержки архитектуры ARM64, а также уровень зрелости и юридическая доступность (лицензии) существующих решений.

Среди существующих компиляторов Оберона можно выделить следующие категории по способу генерации исполнимого кода:

1. **Нативные компиляторы** — генерируют машинный код напрямую, без промежуточных представлений (например, компилятор Project Oberon от Никлауса Вирта, предназначенный для архитектуры RISC5 [20]). Ни один компилятор Оберона нативно не поддерживает платформу ARM64. Поддержка других современных платформ также крайне ограничена.

2. **Компиляторы с генерацией кода на Си** — используют язык Си в качестве промежуточного представления. Получающийся код на Си затем компилируется стандартными средствами (GCC, Clang). Примерами могут служить системы Ofront [26, 27], Ofront+ [28, 29], CPFront [30], Vishap Oberon Compiler (VOC) [31, 32]. Данный подход обеспечивает переносимость, но не даёт прямого контроля над особенностями целевой архитектуры, что критично для эффективной поддержки Apple Silicon. Кроме того, переносимость Си условна, так как для работы как самого компилятора, так и скомпилированных им программ, необходима библиотека времени исполнения, которую, как правило, приходится адаптировать к каждой новой платформе (Windows, Linux, BSD, macOS и др.).

3. **Компиляторы, использующие LLVM** — транслируют Оберон в промежуточное представление LLVM. Это позволяет

получить поддержку многих платформ «из коробки», но требует интеграции с экосистемой LLVM. Пример — OBNC-LLVM (экспериментальный форк), либо исследовательские компиляторы.

4. **Интерпретаторы и виртуальные машины** — такие реализации, как OberonJS [33, 34] и Online Oberon [35], интерпретируют код или транслируют его в WebAssembly (WASM), что позволяет запускать приложения в браузере [36]. Хотя это интересно с точки зрения переносимости, такие решения не подходят для задач, требующих низкоуровневого контроля над платформой.

5. **Компиляция в байткод других платформ (JVM, «.NET»)** — имеются попытки реализации компиляторов, генерирующих байткод Java [37] или «.NET» [38]. Платформозависимые аспекты языка, в частности работа с памятью и модулями, затрудняют перенос Оберона в эти среды.

Особняком стоит компилятор МультиОберон [39, 40], который имеет сразу три постпроцессора и способен транслировать исходную программу как в Си, так и в код LLVM, так и нативно в машинный код для некоторых платформ. Такое разнообразие объясняется нормативными требованиями к программному обеспечению для атомных электростанций. Приёмной комиссии должно быть продемонстрировано, что программа успешно компилируется различными компиляторами на различных вычислительных системах и, кроме того, работает одинаково. МультиОберон поддерживает следующие вычислительные системы:

- 1) Windows (x86);
- 2) Windows (x86_64);
- 3) Linux (x86);
- 4) Linux (x86_64);
- 5) Raspberry Pi (ARMv7l);
- 6) Ubuntu Linux (AArch64, 64-bit Raspberry Pi).

Кроме того, МультиОберон способен гибко изменять синтаксис входного языка. Для этого в нём были предусмотрены специальные синтаксические конструкции USE и RESTRICT [41].

Независимо от архитектуры или модели генерации кода, любая реализация компилятора требует наличия:

- 1) **библиотеки времени исполнения** — минимального набора функций, реализующих базовые возможности языка (работа со строками, ввод-вывод, сборка мусора при наличии и т. д.);
- 2) **системной библиотеки модулей** — реализация базовых модулей стандарта языка (например, In, Out, Files, Strings).

Таким образом, хотя на сегодняшний день существует множество реализаций компиляторов Оберона, ни одна из них не предоставляет полноценной поддержки платформы Apple

Silicon с нативной генерацией кода в формате Mach-O. Это обосновывает актуальность настоящей работы, направленной на разработку такого компилятора.

Одним из доступных инструментов для компиляции программ на Обероне является интегрированная среда разработки Free Oberon [42], автором которой является ваш покорный слуга. Free Oberon работает с помощью транслятора в Си Ofront+, содержит кроссплатформенный набор модулей, интегрированную среду разработки и удобный консольный компилятор с функцией автосборки проекта. К моменту начала работы над данной работой Free Oberon поддерживал системы Windows и Linux. В ходе работы автор портировал его также и на macOS.

1.3. Подготовительный этап: раскрутка компилятора и исправление в Ofront+

Поскольку разрабатываемый компилятор будет написан на том же языке, который он компилирует, возникает классическая задача раскрутки (bootstrapping): для начальной компиляции компилятора требуется уже существующий компилятор, способный транслировать код на Обероне. Пока создаваемый компилятор не станет достаточно зрелым, чтобы компилировать сам себя, необходимо использовать сторонний инструмент.

В качестве такого базового компилятора была выбрана система Free Oberon — кроссплатформенная среда разработки на языке Оберон, использующая в качестве промежуточного представления язык Си. Исходный код на Обероне преобразуется в эквивалентный код на языке Си с помощью транслятора Ofront+, после чего компилируется в машинный код стандартными средствами, такими как GCC или Clang. Это позволяет получать исполнимые файлы для широкого спектра архитектур, включая x86_64 и ARM64.

На момент начала работы Free Oberon не имел поддержки macOS. Автор выполнил перенос системы на платформу macOS, используя версию для GNU/Linux в качестве основы. Основными задачами при портировании стали: адаптация системы сборки, в том числе самосборки системы Free Oberon, настройка компилятора Clang, а также обеспечение совместимости используемых библиотек и модулей языка с особенностями платформы Darwin.

В процессе портирования Free Oberon под macOS была выявлена ошибка в трансляторе Ofront+ [43], унаследованная от его предшественника Ofront. Ошибка проявлялась исключительно на архитектуре ARM64 (Apple Silicon) и заключалась в

некорректной трансляции передачи параметров типа REAL (вещественных чисел) по значению при вызове процедур. При передаче значений других типов, а также при передаче параметров по ссылке (как параметры-переменные), ошибки не возникало. Также необходимым условием возникновения ошибки было то, что значение типа REAL должно быть полем записного типа, причём непременно неэкспортированным (приватным).

В ходе отладки оказалось, что правила ABI (Application Binary Interface) архитектуры ARM64 требуют передачи вещественных аргументов через регистры VFP (Vector Floating Point), тогда как целочисленные значения передаются через общие регистры. Транслятор Ofront+ не учитывал эту особенность, а именно: неэкспортированные поля записей вне зависимости от типа этих полей переводились на язык Си как массив типа `char`, который по размеру совпадал со спрятанными полями. Это работало для всех типов, кроме вещественных чисел, потому что массив типа `char` в действительности передавался процедуре точно так же, как, например, значение типа `int`. В результате, сгенерированный Си-код нарушал соглашения о вызовах, приводя к ошибкам времени выполнения и неверным результатам вычислений.

Временное и довольно простое решение состояло в том, чтобы пометить поля типа REAL как экспортированные. Проведённый анализ и отладка позволили локализовать и устранить ошибку в Ofront+, после чего транслятор стал способен формировать корректно исполнимый код для платформы ARM64. Таким образом, и Free Oberon стал пригоден для использования в качестве исходного компилятора на платформе macOS с процессорами Apple Silicon.

1.4. ARM64 как целевая архитектура

Разрабатываемый компилятор нацелен на прямую генерацию машинного кода для архитектуры ARM64. Эта архитектура, официально обозначаемая как AArch64 (в соответствии со спецификацией ARMv8-A [44]), представляет собой 64-битное расширение архитектуры ARM.

На момент написания работы архитектура ARM64 используется во всей современной линейке устройств Apple: от портативных и настольных ЭВМ (MacBook, Mac Mini, Mac Studio, iMac) до мобильных устройств (iPhone, iPad), а также встроенных платформ — Apple Watch (watchOS) и Apple TV (tvOS). Центральными процессорами этих устройств служат интегральные микросхемы Apple Silicon серий M и A. Разработка,

испытание и отладка разрабатываемого компилятора выполняются на ЭВМ «MacBook Pro» с процессором M3 Pro 2023 г. выпуска, а также на ЭВМ «MacBook Air» с процессором M1 2020 г. выпуска.

В операционной системе macOS основным форматом исполнимых файлов является Mach-O (Mach Object) [45]. Этот формат был разработан как часть микроядерной архитектуры Mach и используется в системах на базе ядра XNU, включая macOS, iOS, iPadOS и другие.

Mach-O представляет собой модульный контейнерный формат, предназначенный для хранения машинного кода, данных, таблиц символов, информации для отладки и других метаданных, необходимых для корректного выполнения и сопровождения программ. Он поддерживает как статическую, так и динамическую компоновку, а также имеет встроенные механизмы для реализации безопасной загрузки, цифровой подписи и разметки сегментов памяти (таких как «__TEXT» и «__DATA»).

Аналогами Mach-O в других операционных системах являются:

1. ELF (Executable and Linkable Format)—используется в Unix-подобных системах, включая Linux и BSD. Это наиболее распространённый формат исполнимых файлов в мире открытого программного обеспечения.

2. PE (Portable Executable)—применяется в операционных системах семейства Windows. Основан на формате COFF, но дополнен структурами, специфичными для Windows. Исполнимые файлы имеют расширение EXE и начинаются со специальной заглушки (stub)—заголовка MS-DOS в формате MZ [46, с. 8].

3. COFF (Common Object File Format)—формат использовался в операционной системе Unix System V.

4. A.out — старейший формат исполнимых файлов в Unix-системах, ныне практически не используется.

5. NE (New Executable) и LE (Linear Executable)—устаревшие форматы, использовавшиеся в ранних версиях Windows и OS/2.

6. MZ (DOS Executable)—используется в MS-DOS и Free-DOS. Один из первых широко распространённых форматов, известный по сигнатуре MZ в заголовке. Послужил основой для NE и PE.

Бинарный формат Mach-O используется не только для исполнимых файлов, но и для:

- а) динамически подключаемых библиотек (dylib);
- б) объектных файлов (object files, «.o»);
- в) статических библиотек (archives, «.a»);
- г) плагинов приложений;

- д) модулей ядра и расширений системы;
- е) тестовых и отладочных сборок.

Mach-O — универсальный формат представления машинного кода в среде macOS. Его поддержка в компиляторе обязательна для интеграции с экосистемой Apple, в частности с системным загрузчиком.

В рамках данной работы компилятор генерирует исполнимые файлы напрямую в формате Mach-O, минуя промежуточные слои вроде LLVM или GCC. Это требует как детального понимания внутренней структуры формата, так и правил вызова (ABI) и соглашений о размещении кода и данных в памяти [47]. Поэтому помимо обращения к официальной документации, выполнено и экспериментальное исследование файлов-образцов формата Mach-O: как с применением специальных утилит, так и на уровне шестнадцатеричного кода ¹.

Глава 2

ИСХОДНЫЙ ЯЗЫК

В качестве исходного языка для разрабатываемого компилятора выбран Оберон [48]. Данный язык представляет собой результат последовательной эволюции языков структурного программирования, начатой с языка Алгол (1960), продолженной в Паскале (1970) и Модуле-2 (1980) и достигшей логического завершения в Обероне (1990).

Язык Оберон разработан профессором Никлаусом Виртом в Швейцарской высшей технической школе (ETH Zürich) как упрощённый и концептуально более целостный преемник языка Модуля-2. Основной целью его создания являлось достижение максимальной выразительности при минимальной сложности синтаксиса и семантики. Важнейшим принципом при проектировании языка стала ориентация на простоту, ортогональность конструкций языка и минимализм, обеспечивающие как удобство практического программирования, так и простоту построения компиляторов.

Эволюционная цепочка «Алгол → Паскаль → Модуль-2 → Оберон» демонстрирует движение в сторону упрощения язы-

¹ См. гл. 13 на с. 91 настоящей работы.

ковых конструкций при одновременном сохранении (а в ряде случаев и расширении) выразительных возможностей. Оберон отличается строгой модульной структурой, статической типизацией и расширяемостью записных типов.

2.1. Диалекты языка Оберон

2.1.1. Оберон (1990)

Язык Оберон существует в нескольких диалектах. Оригинальная версия, разработанная в конце 1980-х годов, отличается от последующих, в частности, отсутствием цикла FOR. Эта конструкция была сознательно исключена: предполагалось, что она избыточна, ведь её функциональность может быть реализована с помощью цикла WHILE. Во всех других диалектах Оберона цикл FOR присутствует.

2.1.2. Оберон-2

Оберон-2, появившийся в 1991, возвращает в язык цикл FOR, а также делает существенное дополнение — возможность объявления процедур, связанных с записным типом (type-bound procedures). При объявлении процедуры перед её названием в скобках указывается *приёмник* — переменная записного типа (см. код 1), которая является функциональным аналогом ключевого слова `this` или `self` в других языках программирования. Примечательно, что в языке Го (2009) используется в точности тот же синтаксис.

Такие процедуры могут перегружаться в типах-расширениях, что напоминает виртуальные методы языка Си++. Также в языке Оберон-2 добавлена возможность помечать переменные и поля записей знаком «-», что обеспечивает их экспорт только для чтения.

Листинг 1. Связанные с типом процедуры в Обероне-2

```
1 MODULE TypeBoundExample;  
2  
3 TYPE  
4   Rectangle = RECORD  
5     a-, b-: REAL  
6   END;
```

```

7
8  PROCEDURE InitRectangle(VAR r: Rectangle; a, b: REAL);
9  BEGIN
10     r.a := a;
11     r.b := b
12 END InitRectangle;
13
14 PROCEDURE (r: Rectangle) Square(): REAL;      (* Здесь r — приёмник *)
15 BEGIN
16     RETURN r.a * r.b
17 END Square;
18
19 PROCEDURE (VAR r: Rectangle) SetSize(a, b: REAL);      (* и здесь *)
20 BEGIN
21     r.a := a;
22     r.b := b
23 END SetSize;
24
25 PROCEDURE Do*;
26 VAR r: Rectangle;
27     x: REAL;
28 BEGIN
29     InitRectangle(r, 4, 7);
30     x := r.Square();
31     r.SetSize(5, 6)
32 END Do;
33
34 END TypeBoundExample.

```

2.1.3. Компонентный Паскаль

Компонентный Паскаль представляет собой ещё один диалект языка Оберон. Изменения по сравнению с Обероном-2 минимальны и касаются в основном введения новых возможностей для более строгой типизации и описания интерфейсов. В частности, добавлены специальные атрибуты для типов и методов: ABSTRACT, EXTENSIBLE, LIMITED и EMPTY [49].

Также была расширена работа со строками: появилась возможность сцепления строк с использованием оператора «+», а также поддержка операций копирования строк до нуль-терминатора ¹.

¹ Под нуль-терминатором понимается литера с кодом 0, обозначающий конец строки.

Компонентный Паскаль также допускает определять в модуле секцию CLOSE. Последовательность операторов в этой секции будет выполнена при выгрузке модуля из ОЗУ.

Листинг 2. Секция CLOSE в Компонентном Паскале

```
1 MODULE ExamplesClose;
2 IMPORT Log;
3 BEGIN
4   Log.String('Hello'); Log.Ln
5 CLOSE
6   Log.String('Good bye'); Log.Ln
7 END ExamplesClose.
```

2.1.4. Пересмотренный Оберон (Оберон-07)

Несмотря на участие в создании языка Оберон-2, Никлаус Вирт в дальнейшем пришёл к выводу, что многие из нововведений оказались избыточными и усложняли исходную концепцию языка. В 2007 году он вернулся к первой версии языка Оберон и выпустил пересмотренную спецификацию, получившую неофициальное название Оберон-07. Эта редакция основана на оригинальном Обероне 1990 года, но содержит ряд уточнений и упрощений, направленных на повышение строгости и однозначности языка.

Ключевым отличием Оберона-07 от Оберона-2 является отказ от процедур, связанных с типами (type-bound procedures), а также упрощение и уточнение ряда синтаксических конструкций. Если в оригинальном Обероне и Обероне-2 тип POINTER мог указывать как на запись, так и на массив, то в Обероне-07 указатели разрешены исключительно на записи. Это убирает из языка поддержку динамических массивов.

С целью исключения использования типа CHAR не по назначению в низкоуровневых задачах в язык был добавлен совместимый с INTEGER базовый тип BYTE. Также устранено неявное преобразование из INTEGER в REAL: теперь приведение типов осуществляется явно с помощью встроенной функции FLT.

Синтаксическая конструкция WITH, использовавшаяся ранее для работы с переменными динамического типа, была исключена. Её функциональность теперь реализована конструкцией CASE, которая до этого использовалась только с числовыми переменными. Из оператора CASE была убрана ветвь ELSE — таким

образом, невозможность выбрать ветвь приводит к аварийному останову.

Оператор цикла WHILE теперь поддерживает ветви ELSIF ... DO, что превращает его в оператор цикла Дейкстры. Убран оператор «бесконечного» цикла (цикла с выходом из середины) LOOP, а вместе с ним убран и оператор EXIT.

Упрощена семантика цикла FOR: теперь он определён через цикл WHILE, что устраняет возможные неоднозначности, связанные с граничными значениями диапазона управляющей переменной цикла, т. е. при целочисленном переполнении.

Возврат из процедуры теперь разрешён только в её конце (см. подробнее на с. 33).

Передача параметров массивов и записей по значению осуществляется в режиме только для чтения, что позволяет избежать лишнего копирования данных и повышает безопасность.

Строковые литералы обозначаются только двойными кавычками «"». Обозначать их одинарными кавычками (апострофами, «'») в Пересмотренном Обероне нельзя.

Последняя версия спецификации языка датируется 2016 годом и опубликована под названием «The Programming Language Oberon» [48].

2.1.5. Пересмотренный Оберон-2

Интересной ветвью развития языка является Пересмотренный Оберон-2 (Revised Oberon-2), представленный в 2020 году Андреасом Пирклбауэром [50]. В отличие от оригинального Оберона-2¹, который был надмножеством Оберона 1990 года и добавлял, среди прочего, процедуры, связанные с типами (type-bound procedures), новая версия основывается на Пересмотренном Обероне (Обероне-07) и аналогичным образом расширяет его функциональность.

Кроме того, Пересмотренный Оберон-2 вводит в язык Оберон образца 2016 года (Оберон-07) следующие средства:

1. Процедура динамического выделения памяти в куче для массивов фиксированной длины и открытых массивов. Во всех диалектах Оберона предусмотрена стандартная процедура NEW, предназначенная для динамического распределения памяти, в частности для размещения в куче динамических записей. Однако в Обероне-07 была исключена возможность размещения в куче массивов, длина которых определяется во время выполнения программы. Пересмотренный Оберон-2 вос-

¹ См. с. 19 настоящей работы.

становливая эту возможность, тем самым обеспечивая большую гибкость в управлении памятью.

2. Ограничение доступа к промежуточным объектам из вложенных областей видимости. В языке Оберон-07 вложенные процедуры не имеют доступа к *переменным*, объявленным в окружающих процедурах. Пересмотренный Оберон-2 расширяет это правило, запрещая доступ также к определённым во внешних процедурах *константам и типам*.

3. Финализация модуля. В дополнение к секции BEGIN в модуле может быть определена секция FINAL, предназначенная для выполнения завершающих действий при выгрузке модуля из оперативной памяти. Аналогичная конструкция предусмотрена в языке Компонентный Паскаль¹, где для этих целей используется ключевое слово CLOSE.

2.2. Сравнение синтаксиса Оберона с другими популярными языками

Язык Оберон принадлежит к семейству языков с синтаксисом, восходящим к Алголу. Это отличает его от языков с Си-подобным синтаксисом. На первый взгляд различия кажутся несущественными. Например, в простейших объявлениях, таких как «x: INTEGER» в Обероне и «int x» в Си, различие заключается лишь в обратном порядке слов и наличии двоеточия. Однако при работе с более сложными типами данных эти различия становятся более заметными и влияют на способность человека быстро и безошибочно воспринимать программный код.

Си-подобный синтаксис подчиняется стековому принципу синтаксического анализа, в рамках которого указательность, массивность и возвращаемые типы «наращиваются» справа от имени объявляемого типа или переменной. Это приводит к сложным для восприятия конструкциям. Например, тип указателя на массив массивов указателей на функцию, принимающую целое число и возвращающую целое число, может быть записан на языке Си так:

```
int (*(x[]))()(int);
```

Для корректного прочтения подобных типов в языке Си существует специальный алгоритм, который, в частности, требует мысленно расставить скобки в выражении в соответствии с приоритетами операций (всего в языке Си 15 уровней приоритета операций). Алгоритм этот таков:

¹ См. с. 21 настоящей работы.

1. Расставьте скобки в объявлении, как если бы это было выражение.

2. Найдите самые внутренние скобки.

3. Произнесите: «имя переменной является...». Если встречается «[]», добавьте «массив из...». Если встречается «*», добавьте «указатель на...».

4. Перейдите к следующему уровню скобок.

5. Если есть ещё уровни, вернитесь к шагу 3.

6. В конце добавьте базовый тип (например, «int», «char» и т. д.).

В итоге чтение происходит с чередованием направлений слева направо и справа налево, что делает это крайне неудобным занятием.

В отличие от этого, в Обероне и других языках с синтаксисом, восходящим к Паскалю, имя объявляемого объекта всегда размещается в начале, что обеспечивает читаемость.

Например, переменная сложного типа, аналогичная приведённой выше, может быть объявлена в Обероне следующим образом:

```
VAR x: POINTER TO ARRAY OF PROCEDURE(x: INTEGER): INTEGER;
```

Более сложный пример объявления переменной на Си продемонстрирован в листинге 3. Следует отметить, что подобные объявления, несмотря на их запутанность, всё же нередко встречаются в практике программирования на Си — в [51, 52] указывается, что сложные для восприятия объявления в языке Си занимают около 15% от их общего числа.

Листинг 3. Сложное объявление переменной на языке Си

```
1 char* (*(*abc[5])(char*))[];  
2 // abc is an array of 5 pointers to a function that  
3 // accepts a pointer to a char and returns a pointer  
4 // to an array of pointers to a char.
```

Примечательно, что при попытке интерпретации сложных объявлений языка Си программисты стихийно воспроизводят порядок лексем, характерный для Оберона [53]:

- Are you sure that `int (*x)[13]` declares x array [13] of pointer to int?
- It doesn't, it's a pointer to an array of 13 ints.

Таким образом, синтаксические решения, реализованные в Обероне, позволяют упростить восприятие сложных типов, не снижая при этом прозрачности и лаконичности описания простых.

Глава 3

ОСОБЕННОСТИ ПРОГРАММИРОВАНИЯ НА ОБЕРОНЕ

Так как нам предстоит написать не самую простую программу на языке Оберон — компилятор, — необходимо отдельно остановиться на нюансах этого языка, чтобы ознакомиться с ним поближе и учесть все особенности в практической части.

3.1. Влияние языка на стиль программирования

Каждый язык программирования, помимо специфического набора средств и особенностей синтаксиса, несёт и свой уникальный стиль программирования — некоторый набор часто используемых приёмов написания программного кода. Яркий пример представляет такое простое действие, как перебор элементов массива на языках Паскаль и Си (см. листинги 4 и 5).

Листинг 4. Пример реализации алгоритма поиска на языке Паскаль

```
1  VAR
2    m: ARRAY [1..N] OF INTEGER;
3    i, x, indexFound: INTEGER;
4  BEGIN
5    { ... }
6    FOR i := 1 TO LENGTH(m) DO BEGIN
7      IF m[i] = x THEN BEGIN
8        indexFound := i;
9        BREAK
10     END
11  END
```

В листинге 5 показан специфический для языка Си вариант реализации этой же задачи с помощью арифметики указателей. Для простоты в обоих примерах предполагается, что в m всегда есть хотя бы один элемент, равный x .

Листинг 5. Пример реализации алгоритма поиска на языке Си

```
1  int m[N];
2  int *a = m;
3  while (*a != x) a++;
4  int indexFound = a - m;
```

В варианте на Паскале управляющая переменная цикла i имеет целочисленный тип и является индексом в массиве m . Адрес же элемента массива $m[i]$ вычисляется заново при каждой итерации¹. Для вычисления адреса $m[i]$ к адресу массива m должно быть прибавлено значение i , умноженное на длину элемента массива в байтах. Часто это может быть реализовано с помощью побитового сдвига, так как из-за необходимости

¹ Следует отметить, что оптимизирующий компилятор может для указанной программы на Паскале выдать машинный код, близкий тому, который получается в варианте на Си. Тогда вместо повторного вычисления адреса для каждого i адрес будет обновляться инкрементально — путём приращения предыдущего значения адреса на размер элемента.

выравнивания данных в ОЗУ [54, с. 43] характерными размерами элементов массива являются степени двойки¹. Инструкция LEA (Load Effective Address) в одной из наиболее популярных процессорных архитектур x86 (и x86_64) используется для ускоренного вычисления адреса элемента в массиве — она одновременно умножает значение регистра на указанную степень двойки и складывает результат с базовым адресом, находящимся в указанном регистре [55].

```
mov eax, [edi + esi*4] ; загружает значение array[esi]
lea  eax, [edi + esi*4] ; вычисляем адрес array[esi]
```

В варианте Си *a* — непосредственный указатель на некоторый элемент массива. При каждой итерации он сдвигается вправо на длину одного элемента. Такой вариант перебора элементов массива наиболее близок к тому, как это обычно делают на языке ассемблера. Во времена создания языка Си (1969—1973 гг.) множество программ писалось на языке ассемблера, и особое внимание уделялось тому, чтобы такие программы можно было безболезненно переписать на новый язык, в том числе не сильно меняя стиль программирования. Через синтаксис языка это неизбежно накладывает отпечаток на стиль программирования на Си.

Итак, рассмотрим некоторые подходы, свойственные программам на Обероне.

3.2. Отсутствие префиксов в именах объектов

Программы на Обероне состоят из именованных модулей, которые, импортируя друг друга, образуют ациклический граф. Каждый модуль может содержать константы, типы, переменные и процедуры. С точки зрения компилятора все они являются *объектами*.

Когда в исходном коде одного модуля программист обращается к объекту, находящемуся в другом модуле (например,

¹ Другими словами, если размер переменной равен, например, 3 байтам, то в качестве элемента массива та же переменная будет занимать 4 байта — так как это ближайшая степень двойки, вмещающая в себя указанную переменную. Такое выравнивание значительно увеличивает скорость доступа к данным как при чтении, так и при записи, поскольку вычислительные системы, как правило, имеют отдельные инструкции для работы с машинными словами — доступ к части такого слова оказывается замедленным.

при вызове процедуры из другого модуля), программист обязан всякий раз указывать имя модуля, к объекту которого он обращается (см. `Greetings.Hello` на строке 17 листинга 6).

Листинг 6. Обращение к объектам импортированного модуля

```
1  MODULE Greetings;
2    IMPORT Out;
3
4    PROCEDURE Hello*;
5    BEGIN
6      Out.String("Hello!");
7      Out.Ln
8    END Hello;
9
10 END Greetings.
11
12 MODULE MyProgram;
13   IMPORT Greetings;
14
15   PROCEDURE Do*;
16   BEGIN
17     Greetings.Hello
18   END Do;
19
20 END MyProgram.
```

Таким образом, вместо `Hello` мы видим `Greetings.Hello`. Синтаксически, `Greetings` здесь может означать либо переменную записного типа, либо имя импортированного модуля. Программист, читающий программу на Обероне, из контекста легко поймёт, что это именно имя модуля, тем более, что имена модулей принято писать с заглавной буквы, а переменные — со строчной¹.

При обращении к таким импортированным объектам вместо истинного имени модуля можно указывать псевдоним (*alias*), который может быть указан при импорте модуля. В качестве

¹ См. таблицу 2 на с. 33 настоящей работы.

псевдонима может быть произвольно выбран любой незанятый идентификатор:

Листинг 7. Указание псевдонима при импорте модуля

```
1 MODULE MyProgram;  
2   IMPORT G := Greetings;  
3  
4   PROCEDURE Do;  
5     BEGIN  
6       G.Hello  
7     END Do;  
8  
9 END MyProgram.
```

Обязательные имена модулей при ссылках на импортированные объекты значительно сокращают время, необходимое для ориентирования в незнакомом коде. Но обязательность имён модулей в свою очередь влияет и на имена объектов (констант, типов, переменных и процедур). В таких языках, как Си и Паскаль, для того чтобы можно было избежать коллизий при одновременном использовании нескольких третьесторонних библиотек, создателям таких библиотек часто рекомендуется снабжать префиксами все имена объектов [56] (см., например, `GtkWidget` вместо `Widget` и `GtkTextBuffer` вместо `TextBuffer` в библиотеке GTK+ для языка Си [57]). В Обероне же имя модуля само играет роль префикса: `Gtk.Widget`, `Gtk.TextBuffer` — более того, префикс оказывается отделённым от собственно имени объекта точкой, что помогает мысленно отбрасывать его при необходимости. А значит, имена самих объектов в префиксах не нужны: `Widget`, `TextBuffer`. Как следствие, когда обращение к этим объектам осуществляется внутри самого модуля, запись остаётся максимально короткой. То же самое имеет место и в языке Го (Go): несмотря на то, что язык Го позволяет импортировать объекты модуля непосредственно в пространство имён импортирующего модуля¹ [58] (см. листинг 8), программисты на Го к этому практически никогда не прибегают (исключая редкие случаи, такие как написание модульных тестов).

¹ В языке Го программные модули называются пакетами (package). Слово же «модуль» в языке Го используется в значении программного компонента.

Листинг 8. Импорт всех объектов модуля в текущее пространство имён в языке Го

```
1 package main
2 import (
3     . "fmt" // с помощью точки все объекты пакета
4             // пакета fmt импортируются напрямую
5             // в текущее пространство имён
6 )
7
8 func main() {
9     Println("Широка страна моя родная")
10    // Нормальная форма: fmt.Println(...)
11 }
```

При именовании констант в программах на Обероне также часто не используются префиксы. Так, например, в Проекте Оберон 2013 модуль `Texts` содержит экспортированные константы для работы с объектом `Texts.Scanner`. Эти константы называются `Inval`, `Name`, `String`, `Int`, `Real` и `Char` и не имеют префиксов в своём названии. Однако при их использовании они фактически будут иметь настраиваемый программистом префикс (см. листинг 9).

Листинг 9. Константы модуля `Texts` Проекта Оберон 2013

```
1 MODULE Texts;
2
3 (* ... *)
4
5 CONST (*scanner symbol classes*)
6     Inval* = 0;    (*invalid symbol*)
7     Name* = 1;    (*name s (length len)*)
8     String* = 2;  (*literal string s (length len)*)
9     Int* = 3;     (*integer i (decimal or hexadecimal)*)
10    Real* = 4;     (*real number x*)
11    Char* = 6;     (*special character c*)
12
13 (* ... *)
14
15 END Texts.
16
```

```

17 MODULE Client;
18
19   IMPORT Texts;
20
21   PROCEDURE Do*;
22   VAR
23     T: Texts.Text;
24     S: Texts.Scanner;
25   BEGIN
26     Texts.Open(T, "numbers.txt");
27     Texts.OpenScanner(S, T, 0);
28     Texts.Scan(S);
29     IF S.class = Texts.Int THEN
30       (* An integer has been scanned *)
31     END
32   END Do;
33
34 END Client.

```

Следует отметить, что константы в модуле `Texts` начинаются с буквы в верхнем регистре, что вообще для программ на Обероне нетипично, что можно видеть в определении рекомендованных стандартных модулей в «Дубовых требованиях» («Oakwood Guidelines») [59].

3.3. Регистр букв как способ маркировки идентификаторов

В разных языках программирования приняты разные стили составления идентификаторов для именования объектов. В некоторых языках параллельно существует несколько стандартов или рекомендаций [60], и программисты обычно должны согласовывать единый стиль при работе над одним проектом в команде.

Известно несколько распространённых вариантов написания идентификаторов [61]:

Таблица 1.

Примеры различных стилей идентификаторов

Наименование стиля	Пример идентификатора
Pascal case	MapRider
Camel case	mapRider
Snake case	map_rider
Kebab case	map-rider

В Обероне принято использовать `PascalCase` и `camelCase`. Отличаются они только регистром первой буквы. Верхний регистр указывает на то, что идентификатор является модулем, процедурой или типом. Если же имя объекта начинается с буквы в нижнем регистре, перед нами переменная или константа. Последнее обстоятельство помогает в случае необходимости легко заменить константу на переменную (если она, конечно, не используется в качестве длины массива или, в целом, в составе некоторого константного выражения).

Часто в одной и той же области видимости (`scope`) можно встретить идентификаторы, различающиеся только регистром первой буквы. Например, `Node` будет именем типа данных, а `node` — именем переменной типа `Node`. Примеры идентификаторов можно видеть в таблице 2.

Таблица 2.

**Примеры типичных имён программных объектов
в Обероне**

Стиль	Идентификатор	Класс объекта
Pascal case	Files	модуль
Pascal case	Traverser	модуль
Camel case	x	переменная
Camel case	topScope	переменная
Camel case	codeLen	переменная
Camel case	maxCodeLen	константа
Camel case	pointerSize	константа
Pascal case	Idenf	тип
Camel case	ident	переменная
Pascal case	Rider	тип
Pascal case	Node	тип — указатель на запись
Pascal case	NodeDesc	записной тип, на который указывает Node
Pascal case	PrintObject	процедура
Pascal case	ChmodAndSign	процедура
Pascal case	HasIntersection	функциональная процедура
Pascal case	NewLeafNode	функциональная процедура
Camel case	eof	поле записи
Camel case	offset	поле записи
Camel case	intVal	поле записи
Camel case	base	поле записи

Стиль `snake_case` в Обероне практически не используется, поскольку сообщение о языке запрещает знак подчёркивания в идентификаторах. В диалекте Оберона Компонентном Паскале знаки подчёркивания разрешены, но использовать их рекомендуется только при написании внешних модулей (*foreign modules*), т. е. привязок (*language bindings*) к коду на других языках программирования.

3.4. Передача управления из процедуры

В новейшем варианте языка Оберон — Оберон-07 — ключевое слово `RETURN` перестало обозначать отдельный оператор и стало синтаксической частью конструкции `PROCEDURE`. Таким образом, возврат значения из функциональной процеду-

ры может и должен осуществляться только в одном месте — в самом конце процедуры. В некоторых случаях это значительно влияет на стиль программирования (см. листинг 10), ведь теперь невозможно выйти из процедуры досрочно без аварийного останова. Если учесть отсутствие операторов `BREAK`, `EXIT` и `CONTINUE`, можно сделать вывод, что программам на Обероне-07 гарантирована полная структурность в смысле Дейкстры [25] и отсутствие завуалированных аналогов оператора `GOTO` [62].

Листинг 10. Запрет на RETURN в произвольном месте внутри процедуры

```
1 (* Стиль Оберона-1990, Оберона-2
2   и Компонентного Паскаля *)
3 PROCEDURE Max(a, b: INTEGER): INTEGER;
4 BEGIN
5   IF a > b THEN RETURN a END;
6   RETURN b
7 END Max;
8
9 (* Стиль Оберона-07 *)
10 PROCEDURE Max(a, b: INTEGER): INTEGER;
11 BEGIN
12   IF a < b THEN a := b END
13 RETURN a END Max;
```

Существенное влияние указанного синтаксического изменения проявляется при разработке протяжённых и ветвящихся процедур. В частности, цикл `FOR` оказывается неприменим в случаях, когда количество итераций не может быть определено ещё перед его началом. Рассмотрим код в листинге 11, в котором решается задача поиска элемента с заданным значением в массиве. Мы ищем элемент с наибольшим индексом, а если их нет, возвращается `-1`. Начинаящим программистам первый вариант кажется более естественным. Однако он не вполне соответствует принципу структурности, что затрудняет верификацию кода. При этом возможность верификации важна сама по себе — даже если программист не проводит её явно, а лишь мысленно отслеживает корректность программы [63, с. 33]. Если программы пишутся без учёта требований структурности, в более сложных случаях в них могут возникать

избыточные конструкции из вложенных циклов, как показано в работе [64, разд. 2.5].

Листинг 11. Лине́йный поиск в массиве: неструктурный и структурный варианты

```
1 (** Возвращает индекс последнего вхождения x в m
2   или -1. Вариант с множественными RETURN. *)
3 PROCEDURE Find1(m: ARRAY OF INTEGER;
4                 x: INTEGER): INTEGER;
5 VAR i: INTEGER;
6 BEGIN
7   FOR i := LEN(m) - 1 TO 0 BY -1 DO
8     IF m[i] = x THEN
9       RETURN i
10    END
11  END;
12  RETURN -1
13 END Find1;
14
15 (** Возвращает индекс последнего вхождения x в m
16   или -1. Вариант с единственным RETURN. *)
17 PROCEDURE Find2(m: ARRAY OF INTEGER;
18                 x: INTEGER): INTEGER;
19 VAR i: INTEGER;
20 BEGIN
21   i := LEN(m) - 1;
22   WHILE (i # -1) & (m[i] # x) DO DEC(i) END
23 RETURN i END Find2;
```

Более подробный обзор диалекта Оберон-07 см. на с. 21.

3.5. Отсутствие процедур с переменным количеством параметров

В языке Паскаль есть операторы `Read` и `Write` (а также `ReadLn` и `WriteLn`) для работы с консолью и для файлового ввода-вывода (см. листинг 12). Это единственные процедуры

в языке Паскаль, которые принимают произвольное количество фактических параметров.

Листинг 12. Произвольное количество параметров операторов ввода-вывода языка Паскаль

```
1 Program IOTest;  
2 Var name: String;  
3 Begin  
4   Write('Введите имя: ');  
5   ReadLn(name);  
6   WriteLn('Привет, ', name, '! 2 + 2 = ', 2 + 2);  
7   ReadLn;  
8 End.
```

При выполнении простых задач это представляет некоторое удобство, но проблемой данного подхода является то, что операторы ввода-вывода вшиты в язык, а не находятся в библиотеке, которую можно было бы произвольно заменить на другую.

В языке Си данная проблема решена иначе. Он допускает произвольное количество фактических параметров у любой функции (см. листинг 13). Это позволяет вынести функции ввода-вывода в библиотеку и исключить их из определения языка. Кроме того, разработчики необычных вычислительных систем могут писать свои специфические функции ввода-вывода, внешний интерфейс которых будет полностью совпадать со стандартными `printf`-функциями. Менее очевидным следствием такого решения является ограничение на порядок передачи параметров: вызываемая функция должна обнаружить первый аргумент на вершине стека, чтобы на его основе определить количество и типы последующих.

Листинг 13. Произвольное количество параметров функций в языке Си

```
1 int printf(const char *format, ...) {  
2     va_list args;  
3     int result;  
4     va_start(args, format);  
5     // ...  
6     // Получаем следующий аргумент  
7     int i = va_arg(args, int);  
8     // ...  
9     va_end(args);  
10    return result;  
11 }
```

В языке Си++ аналогичный результат достигается за счёт перегрузки операторов (см. листинг 14), что ещё более усложняет возможность глубокого понимания логики системы ввода-вывода со стороны программиста. Перегрузка операторов в целом может серьёзно осложнять понимание любой программы, поскольку по одной лишь символу, например, «*», зачастую невозможно определить, идёт ли речь об умножении, разыменовании, определении указательного типа или вызове произвольной функции, определённой в одном из многочисленных файлов проекта. Применение перегрузки операторов при написании программ вряд ли когда-либо является строго необходимым [65], а значит, наличие такого механизма в языке нельзя оправдать с точки зрения стремления к простоте и прозрачности его конструкции.

```
1 string name;  
2 std::cout << "Введите имя: ";  
3 std::cin >> name;  
4 std::cout << "Привет, " << name  
5      << "! 2 + 2 = " << 2 + 2 << std::endl;
```

Начиная с языка Модуля-2 и далее в Обероне, операторы ввода-вывода были удалены из определения самого языка и помещены в модули. Но стремление к простоте языка — принцип «просто, насколько возможно, но не проще этого» — диктует отказ от процедур произвольного количества фактических параметров. Во-первых, они используются довольно редко — какое-то существенное удобство они дают главным образом в операциях ввода-вывода. Во-вторых, они усложняют язык, а следовательно, и несколько затрудняют понимание языка программистом. В-третьих, они усложняют построение компилятора.

В Обероне вывод осуществляется поэлементно, т. е. каждое значение выводится посредством отдельного процедурного вызова (см. листинг 15). Это несколько громоздко, но зато такой подход максимально прост и избавляет от необходимости языковой поддержки процедур с переменным числом параметров. Таким образом, система ввода-вывода в Обероне принципиально не отличается от любой другой подсистемы, представляющей собой набор модулей с содержащимися в них процедурами, типами, константами и переменными.

```
1 MODULE IOTest;
2 IMPORT In, Out;
3 VAR name: ARRAY 30 OF CHAR;
4 BEGIN
5   In.Open;
6   Out.String("Введите имя: ");
7   In.Line(name);
8   Out.String("Привет, ");
9   Out.String(name);
10  Out.String("! 2 + 2 = ");
11  Out.Int(2 + 2, 0);
12  Out.Ln
13 END IOTest.
```

3.6. Расширение записных типов

Расширение записных типов (record type extensions)—единственное добавление в язык по сравнению с Модуль-2. В языке Оберон несколько своеобразно понимается ООП. Вместо более распространённых конструкций, позволяющих писать в объектно-ориентированном стиле, как в языках Си++ и Ява, в Обероне существует расширение записных типов. Расширение заменяет собой наследование. Если в языке Си++ есть конструкции `struct` и `class`, то в Обероне в обоих случаях используется `RECORD`. Инкапсуляция осуществляется на уровне модулей в форме *сокрытия* (information hiding). Полиморфизм достигается по-разному, в зависимости от конкретного диалекта Оберона: процедурными переменными (указателями на процедуры) или процедурами, связанными с типом (type-bound procedures), — так в литературе, посвящённой языку Оберон, часто называются *методы*.

Модульные программы — программы, состоящие из модулей, где реализация каждого модуля может изменяться независимо от другого, причём неизменными сохраняются лишь интерфейсы между модулями.

Под расширяемыми программами я понимаю такие модульные программы, которые написаны таким образом, что для того, чтобы расширить их функционал, необходимо лишь написать новые модули, без необходимости переписывать старые.

Оберон возник как развитие (и одновременно упрощение) языка Модуля-2, из которого были убраны избыточные конструкции, такие как типы-перечисления (enumeration types). Практически, в языке остались только совершенно необходимые средства. Так в первой версии языка Оберон отсутствовал даже цикл FOR, поскольку все его возможности полностью покрываются циклом WHILE. Во всех последующих версиях Оберона цикл FOR присутствует, так как его использование значительно упрощает восприятие некоторых участков программного кода.

Одновременно с этим практика программирования на Модуле-2 показала, что на этом языке невозможно, не теряя строгой типизации, создавать сложные расширяемые программы. Язык Оберон имеет лишь одну новую конструкцию, которая отсутствовала в Модуле-2 — расширение записных типов. Эта конструкция является последним недостающим звеном, обеспечивающим возможность создания расширяемых программ на Обероне [2]. При объявлении записного типа (типа RECORD, аналога struct в таких языках, как Си или Го) в скобках разрешается указать название другого записного типа. Новый тип называется *расширением* указанного (базового) типа, что позволяет подставлять его на место формального параметра базового типа (см. листинг 16).

Листинг 16. Расширение записных типов в Обероне

```
1  MODULE Figures;
2  TYPE
3    Figure = RECORD
4      x, y: INTEGER
5    END;
6
7    Circle = RECORD(Figure)
8      radius: INTEGER
9    END;
10
11   Rectangle = RECORD(Figure)
12     width, height: INTEGER
13   END;
14
15   ColoredCircle = RECORD(Circle)
16     color: INTEGER
17   END;
18
19  PROCEDURE P(VAR f: Figure);
20  BEGIN
21    (* ... *)
22  END P;
23
24  PROCEDURE Do*;
25  VAR
26    c: Circle;
27    r: Rectangle;
28    cc: ColoredCircle;
29  BEGIN
30    P(c);
31    P(r);
32    P(cc)
33  END Do;
34
35  END Figures.
```

При передаче параметра-переменной типа `Figure` в процедуру `P` фактический параметр может быть прямым или опосредствованным расширением этого типа, то есть как `Circle`, `Rectangle`, так и `ColoredCircle`. В оригинальном Обероне и Обероне-07 методы класса организуются как указатели на процедуры в каждом объекте; существуют и другие способы осуществления ООП, такие как отдельная запись с указателями на процедуры и шина сообщений [66, с. 59].

Операция `IS` позволяет осуществить проверку типа.

```
IF f IS Circle THEN (* ... *) END
```

Операция охраны типа (`type guard`) позволяет получить доступ к специфическим полям записи, относящимся к расширению.

```
f(Circle).radius
```

Таким образом, программа, написанная на Обероне, может работать со значениями записного типа в режиме полиморфизма. Новые типы могут быть добавлены в новых модулях, без переписывания старых модулей и даже без их перекомпиляции [14, с. 167]. В диссертации [2] описана реализация целой расширяемой операционной системы на Обероне-2, где новую файловую систему или даже систему распределения памяти в ОЗУ можно включить в уже работающую систему не только без её перекомпиляции, но даже и без перезапуска.

Чтобы реализовать подобный расширяемый функционал в Модуле-2, необходимо было ослаблять проверку типов между модулями — передавать значения типа запись как нетипизированный указатель. Это нарушает строгую типизацию и в целом ухудшает безопасность программ.

3.7. Цикл Дейкстры: `WHILE` с ветвями

Разрабатывая математический метод исчисления преобразователей предикатов, позволяющий доказывать корректность программ, Эдсгер Дейкстра в своей книге «Дисциплина программирования» [25, с. 32] определил оператор цикла «`do od`» подобно оператору выбора «`if fi`»:

$$\begin{array}{l}
\mathbf{if} \ x = y \rightarrow L_1 \\
\quad \square \ x < y \rightarrow L_2 \\
\quad \square \ x > y \rightarrow L_3 \\
\mathbf{fi}
\end{array}
\tag{3.1}$$

$$\begin{array}{l}
\mathbf{do} \ x = y \rightarrow L_1 \\
\quad \square \ x < y \rightarrow L_2 \\
\quad \square \ x > y + 6 \rightarrow L_3 \\
\mathbf{od}
\end{array}
\tag{3.2}$$

$$\text{Определение:}
\tag{3.3}$$

$$\begin{array}{l}
\langle \text{оператор} \rangle ::= \mathbf{if} \\
\quad \langle \text{набор охраняемых команд} \rangle \\
\quad \mathbf{fi} \mid \\
\quad \mathbf{do} \\
\quad \quad \langle \text{набор охраняемых команд} \rangle \\
\quad \mathbf{od}
\end{array}$$

$$\begin{array}{l}
\langle \text{набор охраняемых команд} \rangle ::= \langle \text{охраняемая команда} \rangle \\
\quad \{ \square \langle \text{охраняемая команда} \rangle \}
\end{array}$$

$$\begin{array}{l}
\langle \text{охраняемый заголовок} \rangle ::= \langle \text{логическое выражение} \rangle \rightarrow \\
\quad \langle \text{оператор} \rangle
\end{array}$$

$$\begin{array}{l}
\langle \text{охраняемая команда} \rangle ::= \langle \text{охраняемый заголовок} \rangle \\
\quad \{ ; \langle \text{оператор} \rangle \}
\end{array}$$

Ближайшим аналогом оператора «**do od**» в современных языках программирования является оператор цикла с условием продолжения работы **WHILE**. Однако, как правило, оператор **WHILE** не имеет ветвей. Оказалось, что цикл с ветвями позволяет записывать алгоритмы с вложенной структурой в более плоском, и поэтому простом виде. В Обероне-07 определение нетерминала **WHILE** расширено с тем, чтобы добавить в него ветви.

Отличие от цикла «**do od**», однако, заключается в том, что его ветви, охрана которых имеет значение «истина», исполняются в случайном порядке, тогда как в Обероне цикл **WHILE** проверяет условия ветвей по порядку, останавливаясь на первом истинном условии. Оба цикла завершаются, когда их условия всех ветвей оказываются ложными.

Пример цикла Дейкстры в языке Оберон-07 приведён в листинге 17.

Листинг 17. Пример цикла Дейкстры на Обероне

```
1 WHILE x > y DO
2   x := x - y
3 ELSIF x < y DO
4   y := y - x
5 END
6 (* x = y *)
```

У него может быть сколько угодно ветвей **ELSIF**, в том числе ноль, но, конечно, не может быть ветви **ELSE**. В других диалектах Оберона цикл Дейкстра эмулируется с помощью условно бесконечного цикла **LOOP** и оператора **IF** (см. листинг 18). Выход из цикла осуществляется при помощи оператора **EXIT**¹.

Листинг 18. Эмуляция цикла Дейкстры через цикл LOOP

```
1 LOOP IF x > y DO
2   x := x - y
3 ELSIF x < y DO
4   y := y - x
5 ELSE EXIT END END
```

3.8. Отсутствие перечисляемых типов

¹ Оператор **EXIT** работает только для цикла **LOOP**, с т. е. с его помощью нельзя выйти из цикла **WHILE**, **FOR** или **REPEAT**. В диалекте Оберон-07 цикл **LOOP** отсутствует.

Языки Паскаль и Модула-2 позволяли объявлять перечисляемые типы (см. листинг 19). Однако в процессе разработки сложных программных систем, особенно в условиях командной работы, выяснилось, что использование таких типов может создавать определённые трудности. В частности, при добавлении новых значений в перечисление все клиентские модули¹ должны быть перекомпилированы, даже если изменения, казалось бы, являются обратно совместимыми.

Листинг 19. Перечисляемый тип в языке Модула-2

```
1  DEFINITION MODULE Channel;
2
3  TYPE
4      ChannelErrorTYPE =
5      (NoError,
6       NotDone,
7       TooManyChannels,
8       FormatError,
9       ValueError,
10      UseError);
11
12  PROCEDURE Open(Name: ARRAY OF CHAR; VAR Error: ChannelErrorTYPE);
13
14  PROCEDURE ErrorString(Error: ChannelErrorTYPE;
15                      VAR String: ARRAY OF CHAR);
16
17  END Channel.
```

Ещё одной проблемой является невозможность частичного экспорта перечисления: нельзя сделать часть значений доступной извне, а другую — оставить для внутреннего использования. В результате во многих случаях программисты были вынуждены заменять перечисляемые типы простыми наборами именованных констант, что, в свою очередь, снижало ортогональность языка — одну и ту же идею в программе становилось возможным выразить несколькими различными способами [67].

В стремлении к упрощению синтаксиса и повышению его ортогональности в языке Оберон было принято решение отказаться от перечисляемых типов. Их роль выполняет набор

¹ Клиентами модуля *A* называются модули, импортирующие модуль *A*.

целочисленных констант (см. листинг 20). Такой подход имеет важное практическое преимущество: добавление новой константы не требует перекомпиляции клиентских модулей, поскольку изменение символического файла носит обратно совместимый характер и не влияет на уже скомпилированный код. Кроме того, есть возможность экспортировать только часть констант, оставляя остальные для внутреннего использования.

Листинг 20. Перечень констант в Обероне как замена перечисляемому типу

```
1  MODULE Channel;
2
3  CONST
4    (** Channel Errors **)
5    noError*      = 0;
6    notDone*      = 1;
7    tooManyChannels* = 2;
8    formatError*  = 3;
9    valueError*   = 4;
10   useError*     = 5;
11
12   PROCEDURE Open*(name: ARRAY OF CHAR; VAR error: INTEGER);
13   BEGIN
14     error := noError;
15     (* ... *)
16   END Open;
17
18   PROCEDURE ErrorString*(error: INTEGER; VAR string: ARRAY OF CHAR);
19   BEGIN
20     IF error = noError THEN
21       string := "No error"
22     ELSIF error = notDone THEN
23       string := "Not done"
24     ELSIF (* ... *)
25     ELSE
26       ASSERT(FALSE)
27     END
28   END ErrorString;
29
30 END Channel.
```

Такой подход имеет очевидный недостаток: ошибка при наборе кода может привести к случайному дублированию значений констант, что, в свою очередь, способно вызвать труднообнаружимую ошибку в поведении программы. К недостаткам можно

отнести также и то, что клиент модуля может по ошибке передать в процедуру совсем не те значения типа `INTEGER`, которые от него ожидают.

В тех случаях, когда подобные ошибки недопустимы, можно использовать более строгий приём: определить записной тип (и указатель на него), а затем передавать в качестве аргументов не числа, а указатели на экземпляры этого типа. Благодаря этому компиляция сопровождается строгой проверкой типов, что предотвращает возможность передачи произвольных значений и повышает надёжность интерфейса [68]. Стоит отметить, что передаются именно указатели, и таким образом накладные расходы оказываются сравнимыми с передачей значения типа `INTEGER`.

Рассмотрим листинг 21, иллюстрирующий данный подход. Как и ранее, для каждого типа ошибки объявлена константа, однако в данном случае эти константы не экспортируются. Далее определяется записной тип `ErrorDesc` и соответствующий указательный тип `Error`. Затем объявляются три экспортируемые переменные типа `Error`, которые впоследствии используются для передачи информации об ошибках в процедуры модуля. Эти переменные доступны только для чтения и инициализируются однократно при загрузке модуля — в секции `BEGIN`.

Листинг 21. Более надёжная альтернатива перечню констант

```
1 MODULE Channel;
2
3   CONST
4     noErrorNum          = 0;
5     notDoneNum          = 1;
6     tooManyChannelsNum = 2;
7
8   TYPE
9     Error* = POINTER TO ErrorDesc;
10    ErrorDesc = RECORD
11      num: INTEGER;
12      string: ARRAY 64 OF CHAR
13    END;
14
15  VAR
16    noError-,
17    notDone-,
18    tooManyChannels-: Error;
19
20  PROCEDURE Open*(name: ARRAY OF CHAR;
```



```

21             VAR error: Error);
22 BEGIN
23     error := noError;
24     (* ... *)
25 END Open;
26
27 PROCEDURE ErrorString*(error: Error;
28                        VAR string: ARRAY OF CHAR);
29 BEGIN
30     ASSERT(error # NIL);
31     COPY(error.string, string)
32 END ErrorString;
33
34 BEGIN
35     NEW(noError); noError.num := noErrorNum;
36     COPY("No error", noError.string);
37
38     NEW(notDone); notDone.num := notDoneNum;
39     COPY("Not done", noError.string);
40
41     NEW(tooManyChannels);
42     tooManyChannels.num := tooManyChannelsNum;
43     COPY("Too many channels", noError.string)
44 END Channel.

```

3.9. Обязательный END

Традиционно в языках программирования конструкции IF THEN ELSE, а также WHILE и FOR и другие, синтаксически устроены таким образом, что *непосредственно* допускают в качестве тела только один оператор. Чтобы внутри конструкции WHILE (или аналогичной) использовать *последовательность* операторов, её заключают в операторные скобки (например, BEGIN ... END или { ... }). В результате такая последовательность становится составным (блоковым) оператором, который синтаксически трактуется как единая конструкция — одна из возможных форм оператора вообще.

В качестве иллюстрации приведём определение конструкции IF THEN ELSE из языка Паскаль в нотации РБНФ:

```
if-statement  = if expression then statement  
               [ else statement ] .  
statement    = [ label ":" ]  
               (simple-statement | structured-statement) .  
structured-statement = compound-statement |  
                       conditional-statement | ... .  
conditional-statement = if-statement | case-statement .  
compound-statement  = begin statement-sequence end .  
statement-sequence  = statement { ";" statement } .
```

Реальная конструкция выглядит так:

Листинг 22. Пример конструкции IF THEN ELSE на языке Паскаль

```
1 IF (a < b) AND (b < c) THEN BEGIN  
2   WriteLn('a b c:');  
3   WriteLn(a, b, c)  
4 END ELSE IF (b < c) AND (c < a) THEN BEGIN  
5   WriteLn('b c a:');  
6   WriteLn(b, c, a)  
7 END ELSE BEGIN  
8   WriteLn('other')  
9 END
```

Такой синтаксис утяжелён многократным повторением слов BEGIN и END, а отказ от них несёт в себе некоторую синтаксическую неопределённость (см. листинг 23). Здесь совершенно не очевидно, относится ли ELSE к первому IF или ко второму.

Листинг 23. Проблема IF THEN ELSE в языке Паскаль

```
1 IF x = 5 THEN
2   IF y = 4 THEN
3     Write('A')
4 ELSE
5   Write('B')
```

```
1 IF x = 5 THEN
2   IF y = 4 THEN
3     Write('A')
4 ELSE
5   Write('B')
```

Та же самая синтаксическая неоднозначность касается и языков с Си-подобным синтаксисом (см. листинг 24).

Листинг 24. Проблема if then else в языке Си

```
1 if (5 == x)
2   if (4 == y)
3     printf("B\n");
4 else
5   printf("B\n");
```

```
1 if (5 == x)
2   if (4 == y)
3     printf("B\n");
4 else
5   printf("B\n");
```

В том числе по причине этой синтаксической неоднозначности в рекомендациях по стилю программирования на Си и других языках предлагают *всегда* использовать операторные скобки в ветвях оператора IF и в аналогичных конструкциях управления потоком выполнения.

Язык Оберон унаследовал решение данной проблемы от своего предшественника — языка Модула-2. Здесь синтаксис требует, чтобы все структурные операторы завершались ключевым словом END *всегда* (см. листинг 25). При этом необходимость в использовании ключевого слова BEGIN отпадает. Кроме того, ключевые слова ELSIF и ELSE в Обероне одновременно играют роль, ранее выполняемую END (или «}»), — они замыкают соответствующие последовательности операторов.

Листинг 25. Вложенный IF THEN ELSE в Обероне

<pre>1 IF x = 5 THEN 2 IF y = 4 THEN 3 Write('A') 4 END 5 ELSE 6 Write('B') 7 END</pre>	<pre>1 IF x = 5 THEN 2 IF y = 4 THEN 3 Write('A') 4 ELSE 5 Write('B') 6 END 7 END</pre>
---	---

Примечательно, что оператор REPEAT UNTIL в языках с паскалеподобным синтаксисом изначально был устроен таким образом, что сам по себе выполнял роль операторных скобок, ограничивая последовательность операторов без использования дополнительных ключевых слов.

Листинг 26. REPEAT UNTIL в Паскале, Модуле-2 и Обероне

```
1 i := 0;
2 REPEAT
3   INC(i)
4 UNTIL i = 5;
```

В этом контексте требование обязательного использования ключевого слова END в Обероне можно интерпретировать как стремление привести к единой форме синтаксиса всех операторов управления потоками выполнения.

ОБЩАЯ МОДУЛЬНАЯ СТРУКТУРА КОМПИЛЯТОРА

Процесс компиляции состоит из следующих этапов:

- 1) загрузка текста;
- 2) лексический анализ;
- 3) синтаксический анализ;
- 4) проверка типов;
- 5) генерация кода;
- 6) компоновка.

Компилируемая программа — в данной версии компилятора она представляет собой единственный модуль — последовательно проходит через все эти этапы. Результатом компиляции является исполнимый файл формата Mach-O, готовый к запуску на операционной системе macOS на ЭВМ с процессорами ARM64. Исполнение программы начинается с секции инициализации модуля.

В нашем случае компилятор состоит из следующих модулей: Builder, Generator, Machine, MocErrors, Scanner, Traverser, Emitter, Linker, Moc, Parser, Table и Tree (схему взаимодействия модулей см. на рис. 4-1).

Модуль Moc является первичным модулем программы. Он представляет собой консольный интерфейс компилятора, то есть выводит инструкции по использованию и разбирает аргументы командной строки, а также осуществляет общее руководство препроцессором и постпроцессором, т. е. организует весь процесс компиляции.

Через модуль Machine осуществляется связь остальных модулей компилятора с вычислительной системой — здесь открываются файлы и выводятся сообщения об ошибках. Модуль Machine отслеживает и позицию чтения в файле с исходным кодом. При выводе ошибки модулем выводится также и позиция в файле. К вспомогательному модулю, близкому к модулю Machine, можно отнести и модуль MocErrors. Он содержит коды ошибок и их текстовое выражение.

Модули Scanner, Parser и Builder представляют собой препроцессор (frontend), тогда как модули Traverser, Emitter, Generator и Linker образуют постпроцессор (backend) компилятора (см. рис. 4-2).

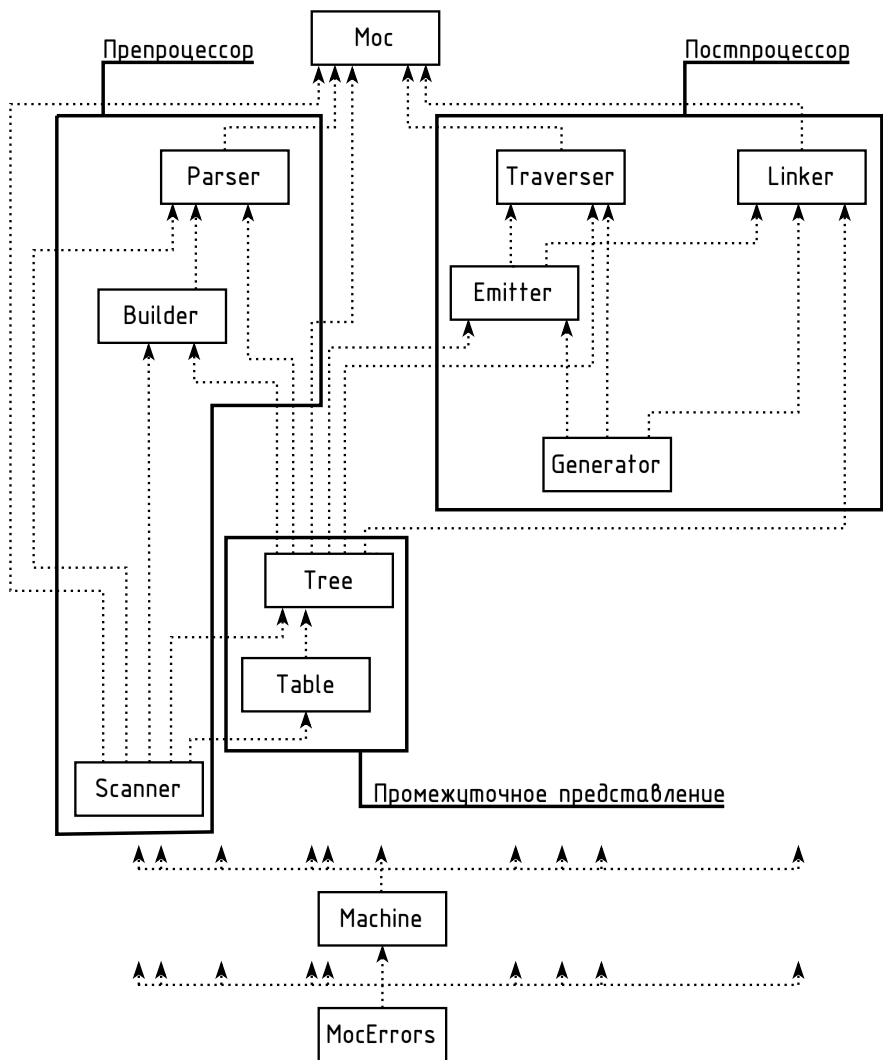


Рис. 4-2. Схема разделения модулей на логические группы: препроцессор, постпроцессор и промежуточное представление.

ЗАГРУЗКА ТЕКСТА

Для начала процесса компиляции необходим предмет компиляции, то есть исходный код на языке программирования высокого уровня. Как правило, он находится в одном или нескольких файлах.

Сначала текст программы необходимо считать из файла и загрузить в оперативную память. Это может быть выполнено как целиком, так и по частям — по мере анализа. Язык Оберон спроектирован таким образом, чтобы обеспечивать возможность однопроходной компиляции: в частности, каждый программный объект должен быть объявлен до его использования.

Это правило имеет одно важное исключение, унаследованное ещё от языка Паскаль: при объявлении *указательного* типа допустимо использовать в качестве базового такой тип, который на момент этого объявления ещё не определён. Такое исключение представляется технически возможным благодаря тому, что на уровне машинной реализации все указатели представляют собой целочисленные значения одной и той же длины, а информация о типе, на который они ссылаются, интерпретируется лишь средствами языка высокого уровня. Другими словами, размер указателя не зависит от базового типа, что позволяет заранее определить структуру данных, содержащую ссылки на ещё не объявленные типы. Яркий пример использования этого исключения — объявление рекурсивных типов (см. листинг 27).

Листинг 27. Рекурсивное объявление типов с использованием указателей

```
1 TYPE
2   Node = POINTER TO NodeDesc;    (* Здесь о типе NodeDesc ещё
3                                   ничего не известно *)
4   NodeDesc = RECORD
5     data: INTEGER;
6     next: Node    (* Типы Node и NodeDesc взаимно рекурсивны *)
7   END;
```

5.1. Программный модуль как единица компиляции

Теоретически важно определить *единицу компиляции*, то есть законченный блок, который достаточен для того, чтобы выполнить его перевод в машинный код. Например, произвольную процедуру нельзя скомпилировать в отрыве от её контекста, потому что в ней, например, может быть обращение к глобальным переменным. Для компиляции необходимо знать их тип. Единицей компиляции в языке Оберон является модуль. Модуль, а не программа, является и единицей загрузки в операционных системах, построенных на базе Оберона (то есть пользователь ОС может загружать и выгружать каждый модуль отдельно от остальных). Наконец, модуль является синтаксическим началом языка Оберон, то есть целевым (или начальным) символом его грамматики [69, с. 235][70, с. 15].

5.2. Модуль Machine

В данном компиляторе считывание текста из файла и отдача его в лексический анализатор осуществляется в модуле Machine — модуле, обеспечивающем связь компилятора с вычислительной платформой на которой он запущен. При переносе компилятора на другую платформу или при адаптации его к новому формату файла и т. д. достаточно будет подправить модуль Machine. Остальные модули компилятора могут переноситься на новые платформы совершенно без изменений.

Модуль Machine определяет записной тип Pos (позиция) с полями line и col, означающими номер строки и номер столбца в файле. Предполагается, что файл сохранён в кодировке ASCII или UTF-8. Значение поля col хранит не количество *байтов* от начала строки, а корректно подсчитывает литеры, что актуально в случае UTF-8.

Модуль экспортирует две переменные типа Pos: переменная pos — хранит текущую позицию в файле, а переменная errorPos — позицию последней возникшей ошибки.

В модуле Machine также определены константы, характеризующие вычислительную платформу, такие как максимальное значение целочисленного типа, ширина экспоненты вещественного числа с плавающей запятой и ширина указательного типа.

ЛЕКСИЧЕСКИЙ АНАЛИЗАТОР

В каждом языке есть алфавит — набор элементарных знаков, из которых формируются слова и предложения. Программы записываются, в общем-то, теми же самыми буквами, цифрами и другими литерами, которыми мы пользуемся на письме в естественных языках. Несмотря на это, в случае языков программирования алфавит состоит не из букв, а из *символов* (лексем).

Символы языка неделимы и также произвольно могут быть заменены на другие. Например, BEGIN и «+» — символы языка Оберон — могли бы быть заменены на НАЧАЛО и ПЛЮС, от этого в языке больше ничего бы не поменялось. Вообще набор символов языка программирования конечен, хотя некоторые из них, такие как числа и идентификаторы, представляют целые классы цепочек литер — теоретически бесконечные по вариантам своих значений. Например, оба символа `pageCount` и `x` являются идентификаторами, хотя и различными, а каждый из символов `4.6`, `0.0` и `276.08` представляет собой (с точностью до значения) не что иное как вещественное число. Следовательно, некоторые типы символов связаны с дополнительными данными. Таких «многоликих» символов в языке Оберон несколько, они перечислены в таблице 3.

Таблица 3.
Символы языка Оберон, с которыми связаны
дополнительные данные

Наименование символа	Пример	Дополнительные данные
<code>ident</code>	<code>Out</code>	строка (цепь литер)
<code>int</code>	<code>217</code>	число типа <code>INTEGER</code>
<code>real</code>	<code>15.53</code>	число типа <code>REAL</code>
<code>string</code>	<code>"Hello"</code>	строка (цепь литер) и её длина
<code>char</code>	<code>61X</code>	литера

На этапе лексического анализа входной поток литер разбивается (или соединяется) на символы, то есть лексические

единицы языка, часто состоящие из многих литер. Лексический анализ ничего не знает о грамматике языка. Всё, что он умеет, — разбивать линейный поток литер на линейный поток символов, попутно отбрасывая пробелы, знаки табуляции, переносы строк и комментарии.

В языке Оберон комментарии могут быть вложенными, поэтому лексический анализ учитывает и этот аспект. При этом в Обероне нет «однострочных» комментариев, которые бы работали до конца строки. Единственный вид комментария в Обероне — текст, заключённый в скобки со звёздочками: (* ... *). Такой формат комментариев был ещё в Паскале, наряду с фигурными скобками { ... }. В языке Оберон фигурные скобки используются для указания литералов типа множество¹.

Если пробелы встречаются внутри строкового литерала (то есть заключены в кавычки), они не отбрасываются анализатором — содержимое строки воспринимается буквально и передаётся на следующий этап разбора без изменений. Более того, Оберон не определяет никаких специальных последовательностей внутри строковых литералов. Например, последовательность входных литер «"\t\n"» будет распознана компилятором как строковой литерал, состоящий из 4 литер: «\», «t», «\» и «n».

Лексический анализатор выполнен в виде модуля Scanner. Модуль определяет 66 целочисленных констант, каждая из которых означает некоторый символ, выдаваемый анализатором (см. листинг 28). Символ null не экспортирован, потому что он используется только внутри самого модуля анализатора для пропуска комментариев и определения ошибочных входных литер.

¹ Речь о литералах типа SET, см. с. 87 настоящей работы.

Листинг 28. Полный список лексических символов языка Оберон

```
1 CONST
2  (** Лексические символы **)
3  null      = 0;   times*   = 1;   rdiv*    = 2;
4  div*      = 3;   mod*     = 4;   and*     = 5;
5  plus*     = 6;   minus*   = 7;   or*      = 8;
6  eql*      = 9;   neq*     = 10;  lss*     = 11;
7  leq*      = 12;  gtr*     = 13;  geq*     = 14;
8  in*       = 15;  is*      = 16;  arrow*   = 17;
9  period*   = 18;  char*    = 20;  int*     = 21;
10 real*     = 22;  false*   = 23;  true*    = 24;
11 nil*      = 25;  string*  = 26;  not*     = 27;
12 lparen*   = 28;  lbrak*   = 29;  lbrace*  = 30;
13 ident*    = 31;  if*      = 32;  while*   = 34;
14 repeat*   = 35;  case*    = 36;  for*     = 37;
15 comma*    = 40;  colon*   = 41;  becomes* = 42;
16 upto*     = 43;  rparen*  = 44;  rbrak*   = 45;
17 rbrace*   = 46;  then*    = 47;  of*      = 48;
18 do*       = 49;  to*      = 50;  by*      = 51;
19 semicolon* = 52; end*     = 53;  bar*     = 54;
20 else*     = 55;  elsif*   = 56;  until*   = 57;
21 return*   = 58;  array*   = 60;  record*  = 61;
22 pointer*  = 62;  const*   = 63;  type*    = 64;
23 var*      = 65;  procedure* = 66; begin*   = 67;
24 import*   = 68;  module*  = 69;  eot*     = 70;
```

Кроме того, определены константы `identSize = 256` и `stringSize = 1024`. Это максимальная длина идентификатора и максимальная длина строкового литерала. При превышении длин в исходном коде лексический анализатор выдаёт ошибку. После этого разбор продолжается. Это сделано для того, чтобы компилятор выдавал программисту целый список ошибок, а не останавливался на первой. Такой же подход использован и в синтаксическом анализаторе. В [54] Вирт указывает, что продолжение разбора входного текста после ошибок — важная черта компилятора, необходимое условие удобства его эксплуатации. Но, вместе с тем, это непростая задача, требующая творческого подхода, потому что по сути компилятор в некотором смысле должен корректно проанализировать *некорректно*

написанную программу — про которую точно известно, что в ней есть ошибка. Недетерминированный и неформализованный характер данной задачи обуславливает сложность её решения.

Определены и типы данных `Ident` и `StrVal` — массивы литер, длиной `identSize` и `stringSize`.

Основная процедура, экспортированная данным модулем, — процедура `Get`. Многократный её вызов из другого модуля — модуля `Parser` — и осуществляет собственно лексический разбор текста.

Модуль `Scanner` также определяет несколько экспортированных переменных, доступных только для чтения, и одну неэкспортированную переменную `ch`.

Листинг 29. Глобальные переменные модуля Scanner

```
1 VAR
2   (** Results of Get **)
3   name*: Ident;
4   intVal*: INTEGER;
5   realVal*: REAL;
6   strVal*: StrVal;
7   strLen*: INTEGER;
8
9   (** - **)
10  ch: CHAR; (** Read-ahead character *)
```

Глобальная переменная `ch` литерного типа используется для реализации метода лексического анализа с заглядыванием вперёд на одну литеру (*look ahead*). Смысл этого метода в том, что он решает следующую проблему. При считывании из файла одной литеры за другой в общем случае невозможно определить, будет ли следующая литера относиться к текущему лексическому символу или нет. Например, начав работу и считав литеру «4» (ASCII-код 52), лексический анализатор может прийти к выводу, что перед ним начало числа. Считав следующую литеру, например «7», начало числа выглядит уже как «47».

Но стоит ли читать дальше? Если будет считана очередная цифра (скажем, «3»), она будет приписана к числу:

$$x' = 10x + (ch - 48),$$

где x — старое значение числа (47), ch — ASCII-код считанной литеры, т. е. цифры «3» (ASCII-код 51), а 48 — ASCII-код цифры нуль.

Если вместо литеры «3» будет считан пробел, то его можно просто отбросить. При следующем вызове разбор начнётся с новой литеры, а пробел при разборе всё равно пропускается.

Но если следующая литера после «47» будет «<», то перед нами проблема. Просто так отбросить эту литеру нельзя, ведь это часть лексического символа, который должен быть выдан *при следующем* запуске процедуры `Get`. Ситуация осложняется ещё и тем, что литера «<» может оказаться как самостоятельным символом — знаком меньше (`lss = 11`), так и первой литерой символа «меньше или равно» («<=», `leq = 12`).

Метод чтения с заглядыванием на одну литеру вперёд решает эту проблему — при любом запуске процедуры `Get` разбор начинается с литеры, находящейся в глобальной переменной ch — к этому моменту в ней уже находится заранее считанная из файла литера. По окончании работы процедуры, она обязана обеспечить, чтобы в той же глобальной переменной находилась литера, непосредственно следующая за последней из тех, которые были отнесены к текущему лексическому символу. Разумеется, это требование в силе только в том случае, если такая литера вообще есть. Если достигнут конец файла, в ch должна быть помещена литра с кодом 0 — т. е. значение `0X`¹.

Глава 7

НИСХОДЯЩИЙ РЕКУРСИВНЫЙ СИНТАКСИЧЕСКИЙ АНАЛИЗАТОР

За этапом лексического разбора следует разбор синтаксический. Входными данными для данного этапа разбора выступают лексические символы. Задача синтаксического разбора — распознать в них (многократно вложенные друг в друга) конструкции языка. Задача синтаксического разбора сильно упрощена за счёт того, что лексический анализатор выделен

¹ Буква «X» в конце шестнадцатеричного числа на языке Оберон означает, что данное число — литерал типа `CHAR`. Например, `0AX` — литера с ASCII-кодом 10, «перевод строки».

в отдельный модуль и на данном этапе совершенно не надо заботиться о пробелах, комментариях, правильном составе идентификаторов и любых других нюансов лексики.

Синтаксический анализатор выполнен в виде модуля `Parser`. Входные символы на этом этапе — неделимые единицы, возможно, несущие дополнительную информацию. Перед началом разбора синтаксический анализатор обращается к лексическому анализатору за первым символом и помещает его в глобальную переменную `sym` (в модуле `Parser`). Происходит это посредством вызова `Scanner.Get(sym)`. Процедуры анализатора будут в первую очередь обращаться за «следующим» входным символом к этой переменной. Если символ действительно относится к данной процедуре, то после его обработки снова будет вызвана процедура `Scanner.Get`. Таким образом, обеспечивается работа анализатора с заглядыванием вперёд на один *символ*. Это похоже на соответствующий приём в модуле `Scanner`, где было реализовано заглядывание на одну *литеру* вперёд.

Кроме (неэкспортированной) глобальной переменной `sym` модуль `Parser` определяет переменную `modName` типа `Ident`¹, которая содержит имя разбираемого в данный момент модуля. `modName` принимает своё значение при разборе начального нетерминала `Module`, а в конце разбора этого нетерминала значение сверяется с именем после `END`².

Наиболее подходит для Оберона синтаксический разбор методом рекурсивного спуска. Идея этого метода в том, что для каждого нетерминала, выраженного в РБНФ, (иными словами, для каждой конструкции языка) пишется отдельная распознающая процедура. В ходе разбора соответствующей конструкции процедура считывает из входного потока все символы, производимые данным нетерминалом. Если правила для данного нетерминала содержат в правых частях другие нетерминалы, процедура вызывает соответствующие распознающие процедуры. Данный метод подробно разобран в [54, 69].

Важно, чтобы после разбора соответствующей конструкции распознающая процедура оставляла в глобальной переменной `sym` символ, следующий сразу за концом данной конструкции.

Рассмотрим часть синтаксиса Оберона, связанную с начальным нетерминалом `modulis` (см. листинг 30). Полный синтаксис Оберона в формате РБНФ есть в приложении А.

¹ См. с. 60 настоящей работы.

² В Обероне название модуля и процедуры пишется повторно после закрывающего соответствующую конструкцию слова `END`.

```
1 модуль = MODULE ident ";" [СписокИмпорта]
2     ПоследовательностьОбъявлений
3     [BEGIN ПоследовательностьОператоров]
4     END идентификатор ".".
5
6 идентификатор = буква {буква | цифра}.
7 СписокИмпорта = IMPORT импорт {"," импорт} ";".
8 импорт = [идентификатор ":@"] идентификатор.
9
10 ПоследовательностьОбъявлений =
11     [CONST {ОбъявлениеКонстанты ";"}]
12     [TYPE {ОбъявлениеТипа ";"}]
13     [VAR {ОбъявлениеПеременных ";"}]
14     {ОбъявлениеПроцедуры ";"}.
15
16 ПоследовательностьОператоров = оператор {";" оператор}.
```

В соответствии с этим фрагментом определения на РБНФ в модуле `Parser` определены процедуры `Module`, `Import`, `DeclarationSequence`, `ConstDeclaration`, `TypeDeclaration`, `VariableDeclaration`, `ProcedureDeclaration`, `Statement` и `StatementSequence`.

Аналогично оформлены и другие нетерминалы. Но это соответствие между нетерминалами РБНФ и процедурами не абсолютное: например, нетерминал `ImportaSaraks` не находит выражения в отдельной процедуре — его код встроен прямо в процедуру `Module`. Это не приводит к дублированию кода или другим проблемам, поскольку нетерминал `ImportaSaraks` используется только в правой стороне определения нетерминала `Module`, т. е. он вообще выделен в отдельный нетерминал для удобства или наглядности определения. По соображениям же программирования бывает лучше снова соединить эти процедуры в одну.

Распознающая процедура, соответствующая нетерминалу `modulis`, показана в листинге 31. Это функциональная процедура, тип возвращаемого ею значения — `Tree.Node` — будет подробно рассмотрен в гл. 9 (с. 76). При разбиении компилятора на препроцессор и постпроцессор, процедуры синтаксического анализа не запускают процедуры генератора машинного кода напрямую, а создают и возвращают узлы дерева. Эту узлы со-

бираются в единое абстрактное синтаксическое дерево, которое и является результатом работы препроцессора и — процедуры Module.

Листинг 31. Процедура распознавания начального нетерминала

```
1 (** module = MODULE ident ";" [ImportList]
2   DeclarationSequence [BEGIN StatementSequence]
3   END ident ".".
4   ImportList = IMPORT import {"," import} ";"*. *)
5 PROCEDURE Module(): Tree.Node;
6 VAR module: Tree.Node;
7   name: Ident;
8   procedures, statementSequence: Tree.Node;
9 BEGIN
10  Skip(S.module);
11  IF Check(S.ident) THEN
12    Strings.Copy(S.name, modName);
13    S.Get(sym); (* ident *)
14
15    (* IMPORT *)
16    Skip(S.semicolon);
17    IF Skipped(S.import) THEN
18      Import;
19      WHILE Skipped(S.comma) DO
20        Import
21      END;
22      Skip(S.semicolon)
23    END;
24
25    (* CONST, TYPE, VAR and BEGIN *)
26    procedures := DeclarationSequence();
27    IF Skipped(S.begin) THEN
28      statementSequence := StatementSequence()
29    END;
30
31    module := Builder.Enter(procedures, statementSequence, NIL);
32    Skip(S.end);
33    IF Check(S.ident) THEN
34      IF S.name # modName THEN
35        Machine.Error(Errors.modNameMismatch)
36      ELSE
37        S.Get(sym); (* ident *)
38        Skip(S.period)
39      END
40    END
```

```
41 END
42 RETURN module END Module;
```

Узел дерева, соответствующий конструкции `module`, имеет класс `NEnter`. Для его создания используется процедура `Enter` модуля-построитель дерева `Builder`. Она и создаёт узел соответствующего класса. В нижнем поддереве этого узла будут находиться процедуры модуля, а в правом поддереве — последовательность операторов, выполняемая при инициализации модуля (операторы, находящиеся после главного `BEGIN`).

Вспомогательные процедуры `Skip`, `Skipped` и `Check` используются вместо операторов `IF` для более удобной записи алгоритма (см. листинг 32).

Листинг 32. Вспомогательные процедуры синтаксического анализатора

```
1 (** Reports whether sym is the expected symbol.
2   If not, reports an error. *)
3 PROCEDURE Check(expectedSym: INTEGER): BOOLEAN;
4 BEGIN
5   IF sym # expectedSym THEN
6     Machine.ErrorExpected(expectedSym, sym)
7   END
8 RETURN sym = expectedSym END Check;
9
10 (** Checks that sym is the expected symbol
11    and skips it, or reports an error. *)
12 PROCEDURE Skip(expectedSym: INTEGER);
13 BEGIN
14   IF sym = expectedSym THEN
15     S.Get(sym)
16   ELSE
17     Machine.ErrorExpected(expectedSym, sym)
18   END
19 END Skip;
20
21 (** If sym is the expected symbol then skips it and
22    returns true, otherwise returns false. *)
23 PROCEDURE Skipped(expectedSym: INTEGER): BOOLEAN;
24 VAR ok: BOOLEAN;
25 BEGIN
26   ok := sym = expectedSym;
27   IF ok THEN S.Get(sym) END
28 RETURN ok END Skipped;
```

Процедура `Check` проверяет, является ли текущий символ переданным ей в качестве параметра. В случае несовпадения выдаётся ошибка компиляции вида «*This expected, but that found*», где «*this*» и «*that*» заменяются конкретными названиями символов. Это происходит в модуле `Machine`. Если же символ совпал, он остаётся на месте для дальнейшего анализа — это полезно, если это, например, символ `int`, `real` или `string`.

Процедура `Skip` работает аналогично процедуре `Check`, но пропускает символ в случае совпадения.

Процедура `Skipped` используется несколько иначе. Она не выдаёт ошибки, но пропускает символ в случае совпадения. Это удобно в тех случаях в ходе разбора конструкции, когда присутствие символа необязательно, но от этого зависит дальнейший разбор. Например, после прочтения обязательных символов `module`, `ident` и `semicolon`, соответствующих ключевому слову `MODULE`, имени модуля и точки с запятой, может следовать ключевое слово `IMPORT`, но его присутствие в коде программы необязательно. В этом месте при разборе используется конструкция `IF Skipped(S.import) THEN`.

Процедура `ErrorExpected` в модуле `Machine`, выдающая сообщение об ошибке несоответствия символа, приведена в листинге 33.

Листинг 33. Процедура вывода ошибки несоответствия символа

```
1 PROCEDURE ErrorExpected*(symExpected, symFound: INTEGER);
2 BEGIN
3   IF BeginError() THEN
4     Errors.PrintSymbol(symExpected);
5     Out.String(' expected, but ');
6     Errors.PrintSymbol(symFound);
7     Out.String(' found');
8     Out.Ln
9   END
10 END ErrorExpected;
```

Для вывода конкретных наименований символов используется модуль `Errors`, который умеет переводить числа, соответствующие символам, в их строковое выражение.

Процедура StatementSequence, распознающая последовательность операторов, выглядит следующим образом:

Листинг 34. Распознающая процедура последовательности операторов

```
1 (** StatementSequence = statement {";" statement}. *)
2 PROCEDURE StatementSequence(): Tree.Node;
3 VAR first, prev, statement: Tree.Node;
4 BEGIN
5   (* Sync *)
6   IF ¬((sym >= S.ident) & (sym <= S.for) OR
7       (sym >= S.semicolon))
8   THEN
9     Machine.Error(Errors.noStatement);
10    REPEAT S.Get(sym) UNTIL (sym >= S.ident)
11  END;
12
13  WHILE sym = S.semicolon DO S.Get(sym) END;
14  first := Statement();
15  prev := first;
16  WHILE Skipped(S.semicolon) DO
17    WHILE sym = S.semicolon DO S.Get(sym) END;
18    statement := Statement();
19    prev.link := statement;
20    prev := statement
21  END
22 RETURN first END StatementSequence;
```

Непосредственно связанная с ней процедура Statement, распознающая единичный оператор, имеет следующий вид:

Листинг 35. Распознающая процедура для оператора

```
1 (** statement = [assignment | ProcedureCall | IfStatement |
2   CaseStatement | WhileStatement | RepeatStatement |
3   ForStatement]. *)
4 PROCEDURE Statement(): Tree.Node;
5 VAR statement, designator: Tree.Node;
6 BEGIN
7   IF sym = S.ident THEN (* Assignment or procedure call *)
8     designator := Designator();
9     IF Skipped(S.becomes) THEN
10       statement := Assignment(designator)
11     ELSE
12       statement := ProcedureCall(designator)
13     END
14   ELSIF Skipped(S.if) THEN
15     statement := IfStatement()
16   ELSIF Skipped(S.while) THEN
17     statement := WhileStatement()
18   ELSIF
19     (* ... *)
20   END
21 RETURN statement END Statement;
```

Аналогично процедурам Module, StatementSequence и Statement выполнены и другие распознающие процедуры.

В Обероне четыре уровня приоритетов операций: *выражение* (expression), *простое выражение* (simple expression), *слагаемое* (term) и *множитель* (factor). Как определение этих нетерминалов в РБНФ, так и соответствующие им процедуры являются рекурсивными. Выражение состоит из одного или двух простых выражений. Простое выражение состоит из одного или более слагаемых. Слагаемое состоит из одного или более множителей.

Наконец, множитель может представлять собой массу различных конструкций, как простых, так и сложных: число, строковой литерал, NIL, TRUE, FALSE, конструкцию отрицания, литерал-множество, идентификатор (возможно, с прикреплён-

ным к нему одним или несколькими указаниями индекса массива или поля записи). Кроме всего прочего, множитель может представлять собой *выражение* в скобках. Таким образом организована косвенная рекурсия из четырёх процедур. Но рекурсия может замкнуться и по другому маршруту: например, при указании индекса в массиве также используется *выражение*.

Возьмём для примера элементарное выражение «75». Примем, для определённости, что следующий за этим числом символ — точка с запятой (класс *semicolon*). Данное выражение элементарное, поскольку оно сводится к одному единственному лексическому символу — целому числу 75. Но, чтобы это узнать, синтаксический анализатор проделывает следующий путь: сначала запускается процедура *Expression*, она вызывает *SimpleExpression* (в исходном коде видно, что такой вызов — первое действие, которое она осуществляет). В свою очередь, *SimpleExpression* вызывает процедуру *Term*¹, а та — процедуру *Factor*.

Процедура *Factor* заходит в ветвь *IF*, которая соответствует условию *sym = int* и с помощью процедуры *NewIntConst*, экспортированной модулем *Builder*, создаёт узел-константу с классом *NConst*, возвращает этот узел и завершается. Таким образом, из *Factor* мы попадаем обратно в *Term*.

Процедура *Term* переходит к вычислению условия оператора *WHILE*².

$$(times \leqslant sym \leqslant and) \quad (7.1)$$

Это условие оказывается ложным, поэтому процедура завершается, возвращая в качестве результата узел, полученный из *Factor*. И из *Term* мы возвращаемся в *SimpleExpression*.

Процедура *SimpleExpression* после вызова *Term* также переходит к циклу *WHILE*³ с похожим условием.

$$(plus \leqslant sym \leqslant or) \quad (7.2)$$

¹ Перед вызовом *Term* процедура *SimpleExpression*, в соответствии с определением соответствующего нетерминала в РБНФ, успевает проверить, нет ли перед ней знака минус или знака плюс. В нашем примере *sym = int* и *intVal = 75*.

² Условие $(times \leqslant sym \leqslant and)$ эквивалентно условию $(sym = times) \vee (sym = rdiv) \vee (sym = div) \vee (sym = mod) \vee (sym = and)$, то есть в широком смысле всем возможным операциям умножения и деления.

³ Условие $(plus \leqslant sym \leqslant or)$ эквивалентно условию $(sym = plus) \vee (sym = minus) \vee (sym = or)$, то есть в широком смысле всем возможным операциям сложения и вычитания.

И это условие оказывается ложным, — и узел, созданный ещё в `Factor`, передаётся теперь в качестве результата вызова `SimpleExpression` в процедуру `Expression`.

Процедура `Expression` теперь должна определить, нужно ли вызывать второй `SimpleExpression` справа от знака сравнения, т. е. есть ли этот знак в исходном коде. Поэтому она подряд проверяет три условия, соответствующие трём ветвям оператора `IF`¹:

$$(eq\ell \leqslant sym \leqslant geq) \quad (7.3)$$

$$(sym = in) \quad (7.4)$$

$$(sym = is) \quad (7.5)$$

Все три условия оказываются ложными, и поэтому процедура `Expression` завершается.

7.1. Модуль `Builder`

Важно отметить, что уже на этапе препроцессора должны выполняться некоторые простейшие оптимизации. Дело в том, что при объявлении констант их значение должно быть константным *выражением*, то есть оно не обязательно должно быть литералом того или иного типа — оно может быть и более сложным выражением. Но требуется, чтобы все его составные звенья также были константными (см. листинг 36), и, как следствие, чтобы значение такой константы могло быть вычислено в момент компиляции, а не только в момент исполнения программы. То же самое касается и задания длины массива.

¹ Условие $(eq\ell \leqslant sym \leqslant geq)$ эквивалентно условию $(sym = eq\ell) \vee (sym = neq) \vee (sym = lss) \vee (sym = leq) \vee (sym = gtr) \vee (sym = geq)$, т. е. всем возможным операциям сравнения.

Листинг 36. Пример использования константных выражений

```
1 CONST
2   x = 25;
3   y = (x + x) MOD 20;
4   z = x * y + x DIV y;
5
6 TYPE Array = ARRAY z * 4 DIV 5 + 3 OF INTEGER;
```

Для этого построитель *Builder*, когда ему дают команду создать новый узел — операцию «+» — не спешит сразу создавать его. Сначала он проверяет, не являются ли оба операнда константами (узлами класса *NConst*). Если это так, то вместо создания третьего узла и объединения двух поддеревьев модуль построитель возвращает один константный узел, значение которого уже на этом этапе оказывается полностью вычисленным.

Глава 8

ПРОВЕРКА ТИПОВ

Оберон — язык со строгой типизацией, поэтому компилятор в ходе своей работы должен обеспечивать согласование типов при присваивании, а также при передаче параметров в процедуры.

Если брать только синтаксис языка Оберон, а значит не учитывать контекстную информацию, в частности типы данных, то выражение «25 DIV TRUE + "R" - ORD(48)» окажется вполне корректным. Однако оно полностью лишено смысла. Отсюда можно сделать вывод, что синтаксис языка не касается согласования типов.

Проверку типов удобно осуществлять в синтаксическом анализаторе, так как, во-первых, она не сильно усложняет его, а во-вторых, синтаксическому анализатору время от времени нужны знания о типах данных, с которыми он работает. Дело в том, что охрана типа (*type guard*) в Обероне синтаксически совпадает с вызовом процедуры: и в том, и в другом случае за

идентификатором следует открывающая круглая скобка. Если в данном контексте идентификатор обозначает процедуру, то перед нами процедурный вызов. В противном случае компилятор воспримет это место как охрану типа. Поэтому учёт контекста важно производить уже на этапе синтаксического разбора.

Для проверки корректности типизации в выражениях, подобных $x = y$, синтаксическому анализатору требуется информация о типах *простых выражений* x и y , поскольку необходимо исключить случаи несогласованных сравнений, например, между числом и строкой.

Данный компилятор разделён на препроцессор и постпроцессор, между которыми лежит граница в виде промежуточного представления — абстрактного синтаксического дерева. Поэтому к моменту, когда разбор достигает операции сравнения, выражения x и y уже представлены в виде готовых поддеревьев. К счастью, корневая вершина каждого из них содержит информацию о типе соответствующего простого выражения.

Следует отметить, что каждое из таких поддеревьев — будь то x или y — может представлять собой как простой терминальный узел класса `NConst` с числовой константой, так и весьма разветвлённое дерево, включающее вызовы функциональных процедур, разыменованние указателей, переход к элементам массивов и прочее. В любом случае, информация о типе данных сохраняется, в том числе, в корне поддерева, что значительно упрощает синтаксический разбор: нет необходимости повторно углубляться в уже построенные поддеревья.

Если в компиляторе отсутствует явное разделение на препроцессор и постпроцессор и машинный код генерируется непосредственно в процессе синтаксического разбора — как это реализовано, например, в компиляторе Вирта [54] и компиляторе Интер-Оберон [71, 72], — то синтаксический анализатор вместо узлов типа `TreeNode` оперирует так называемыми предметами — записями типа `Item`, которые переносят ту же самую информацию, которую несут вершины деревьев в нашем случае. С точки зрения программной реализации распознающие процедуры тогда передают результат не через `RETURN`, а через параметры-переменные. Это позволяет вовсе не использовать динамическую память при синтаксическом разборе¹.

¹ ...если не считать размещаемой в динамической памяти таблицы символов. См. с. 73 настоящей работы.

ПРОМЕЖУТОЧНОЕ ПРЕДСТАВЛЕНИЕ ПРОГРАММЫ

Два модуля — Tree и Table — составляют границу между препроцессором и постпроцессором. Обычно в компиляторах Оберона эту роль выполняет один модуль. Мы же решили разделить его на два, поскольку эти модули решают различные обособленные задачи.

9.1. Таблица символов

Модуль Table имеет такое название, поскольку исторически сложилось, что данные об объявлениях называются в компиляторах «таблицей символов». На самом деле они мало напоминают таблицу — скорее, это граф.

В ходе синтаксического разбора препроцессор заполняет модуль Table данными об импортированных при компиляции модулях, объявленных константах, типах данных и глобальных переменных. Все эти данные хранятся непосредственно в модуле Table в виде динамической структуры данных и извлекаются из неё постпроцессором в ходе обхода абстрактного синтаксического дерева.

Всё содержимое таблицы символов модуль Table удерживает за один указатель — topScope. Тип этого указателя — Object, а базовый его тип — ObjectDesc¹. Кроме этого, модуль Table определяет типы ConstVal, Type и Module (см. листинг 37). Последний является расширением типа Object.

Object всегда обозначает *именованный* объект.

Листинг 37. Структура данных модуля Table

```
1 TYPE
2   Ident* = Scanner.Ident;
3   StrVal* = Scanner.StrVal;
4
5   ConstVal* = POINTER TO ConstValDesc;
6   Type* = POINTER TO TypeDesc;
7   Object* = POINTER TO ObjectDesc;
8   Module* = POINTER TO ModuleDesc;
9
```

¹ Вообще, сокращение «Desc» означает «descriptor» (описатель).

```

10  ConstExt* = POINTER TO ConstExtDesc;
11  ConstExtDesc*= RECORD
12      s*: StrVal
13  END;
14
15  ConstValDesc* = RECORD
16      ext*: ConstExt; (** Для строковых констант *)
17      intVal*: INTEGER;
18      realVal*: REAL;
19      setVal*: SET
20  END;
21
22  TypeDesc* = RECORD
23      form*: INTEGER; (** См. константы Type.form *)
24      paramCount*: INTEGER;
25      len*: INTEGER;
26      size*: INTEGER; (** В байтах *)
27      base*: Type;
28      dsc*: Object; (** Объект-потомок *)
29      typeObj*: Object (** Оригинальный объект,
30                          который ввёл тип *)
31  END;
32
33  ObjectDesc* = RECORD
34      class*: INTEGER; (** См. константы Object.class *)
35      name*: Ident;
36      dsc*: Object; (** Объект-потомок *)
37      type*: Type;
38      constVal*: ConstVal;
39      next*: Object
40  END;
41
42  ModuleDesc* = RECORD(ObjectDesc)
43      originalName*: Ident (** Оригинальное имя модуля,
44                             а name - это псевдоним *)
45  END;

```

Для каждого базового типа данных определена глобальная переменная (см. листинг 38). Кроме переменной `topScore`, указывающей на вершину графа объектов, определённых в программе, есть ещё две переменные того же типа: `universe` и `system`.

Листинг 38. Глобальные переменные модуля Table

```
1 VAR topScope*, universe, system*: Object;  
2   byteType*, boolType*, charType*, intType*, realType*,  
3   setType*, nilType*, noType*, strType*: Type;
```

Переменная `universe` указывает на начало области видимости, которая соответствует уровню, на котором находятся встроенные в язык объекты, такие как `INTEGER` или `BOOLEAN` (предобъявленные базовые типы данных), процедуры увеличения и уменьшения чисел `INC` и `DEC` или встроенные функции `ORD`, `CHR` и `LEN`.

Переменная `system` указывает на закрытую для обычного доступа область видимости. Это содержимое псевдомодуля `SYSTEM`: процедура `SIZE`, `ADR`, `PUT` и другие. Они становятся доступны только при обращении к модулю `SYSTEM` (разумеется, только после того, как его импортируют). В синтаксическом анализаторе псевдомодуль `SYSTEM` обрабатывается отдельной ветвью `IF`.

Все эти глобальные переменные получают своё изначальное значение при инициализации модуля `Table` (см. листинг 39).

Листинг 39. Инициализация модуля Table

```
1 BEGIN  
2   byteType := NewType(Int, 1);  
3   boolType := NewType(Bool, 1);  
4   charType := NewType(Char, 1);  
5   intType := NewType(Int, 4);  
6   realType := NewType(Real, 4);  
7   setType := NewType(Set, 4);  
8   nilType := NewType(NilType, 8);  
9   noType := NewType(NoType, 4);  
10  strType := NewType(String, 8);  
11  
12  (* Инициализация объектов универсума *)  
13  (* Стандартные функции *)  
14  (* ... *)  
15  Enter('LEN', SFunc, intType, 1, lenFunc);  
16  Enter('CHR', SFunc, charType, 1, chrFunc);  
17  Enter('ORD', SFunc, intType, 1, ordFunc);  
18  
19  (* Стандартные процедуры *)  
20  (* ... *)  
21  Enter('DEC', SProc, noType, 1, decProc);
```

```

22  Enter('INC', SProc, noType, 1, incProc);
23
24  (* Типы *)
25  (* ... *)
26  Enter('BOOLEAN', Typ, boolType, 0, 0);
27  (* ... *)
28  Enter('INTEGER', Typ, intType, 0, 0);
29
30  OpenScope;
31  topScope.next := system;
32  universe := topScope;
33  system := NIL;
34
35  (* Инициализация небезопасных объектов
36     псевдомодуля SYSTEM *)
37  (* Функции *)
38  Enter('SIZE', SFunc, intType, 1, sizeFunc);
39  Enter('ADR', SFunc, intType, 1, adrFunc);
40  (* ... *)

```

Тип `Type` используется для сохранения информации о типах данных. Каждый базовый тип связан с соответствующим экземпляром `Type`. Например, глобальная переменная `intType` в модуле `Table` указывает на такой экземпляр. Когда в программе объявляется переменная, например `x` типа `INTEGER`, в области видимости `topScope` создаётся новый объект `x`. Его поле `name` заполняется значением `"x"`, класс принимает значение `Var`, а поле `type` начинает указывать туда же, куда указывает глобальная переменная `intType`.

У записей типа `Type` есть и обратный указатель на `Object` — это нужно для тех случаев, когда у типа есть псевдонимы. Оригинальное название типа содержится в поле `name` записи `Object`, на которую указывает поле `typeObj` записи типа `Type`.

9.2. Абстрактное синтаксическое дерево

В отличие от модуля `Table`, модуль `Tree` сам по себе не хранит данных, а только предоставляет структуру данных: запись `NodeDesc` и указатель на неё `Node` (см. листинг 40). Эта запись имеет указатели `left` и `right` на другие узлы, таким образом, получается двоичное дерево. Однако для удобства в узел введён ещё один указатель `link`, который используется в случае кодирования перечней: последовательностей операторов, последовательностей определения процедур, списка формальных или фактических параметров. Несмотря на наличие в каждом

узле до трёх ссылок на другие узлы, в литературе это дерево называют, однако, двоичным [73, с. 12] [74, с. 5—6].

Листинг 40. Структура данных модуля Tree

```
1 TYPE
2   Node* = POINTER TO NodeDesc;
3   NodeDesc* = RECORD
4     left*, right*, link*: Node;
5     class*: INTEGER;    (** См. константы Node.class *)
6     subclass*: INTEGER; (** См. константы Node.subclass *)
7     type*: T.Type;
8     object*: T.Object;
9     constVal*: T.ConstVal
10  END;
```

Поле class записи Node может принимать одно из значений, определённых константами (см. листинг 41).

Листинг 41. Классы узлов абстрактного синтаксического дерева

```
1 CONST
2   (** Значения Node.class **)
3   NVar*      = 0; NVarPar*  = 1; NField*    = 2;
4   NDeref*    = 3; NIndex*  = 4; NGuard*    = 5;
5   NExpGuard* = 6; NConst*   = 7; NType*     = 8;
6   NProc*     = 9; NUpTo*    = 10; NMonadic* = 11;
7   NDyadic*   = 12; NCall*   = 13; NInitTD*  = 14;
8   NIf*       = 15; NCaseElse* = 16; NCaseDo* = 17;
9   NEnter*    = 18; NAssign* = 19; NIfElse*  = 20;
10  NCase*      = 21; NWhile*  = 22; NRepeat*  = 23;
11  NTrap*      = 28; NFixup*  = 31;
```

Конструкция IF THEN ELSIF ELSE представляется в виде абстрактного синтаксического дерева довольно любопытным образом:

1. Корень всей конструкции — узел класса NifElse.
 2. Последовательность операторов, находящаяся в ветви ELSE, выходит наверх, формируя правое поддерево корневого узла.
 3. Левое поддерево корневого узла содержит узел класса Nif.
 4. Левое поддерево каждого узла Nif представляет собой условие соответствующей ветви.
 5. Правое поддерево каждого узла Nif есть последовательность операторов, содержащаяся в соответствующей ветви.
 6. Если в исходном коде после ветви IF или ELSIF следует ещё одна ветвь ELSIF, то связанная с ней конструкция попадает в поле link предыдущего узла Nif.
- Этот принцип иллюстрируется на рис. 9-1.

```

IF  условие1 THEN послОператоров1
ELSIF условие2 THEN послОператоров2
ELSIF условие3 THEN послОператоров3
ELSE послОператоровИначе
END

```

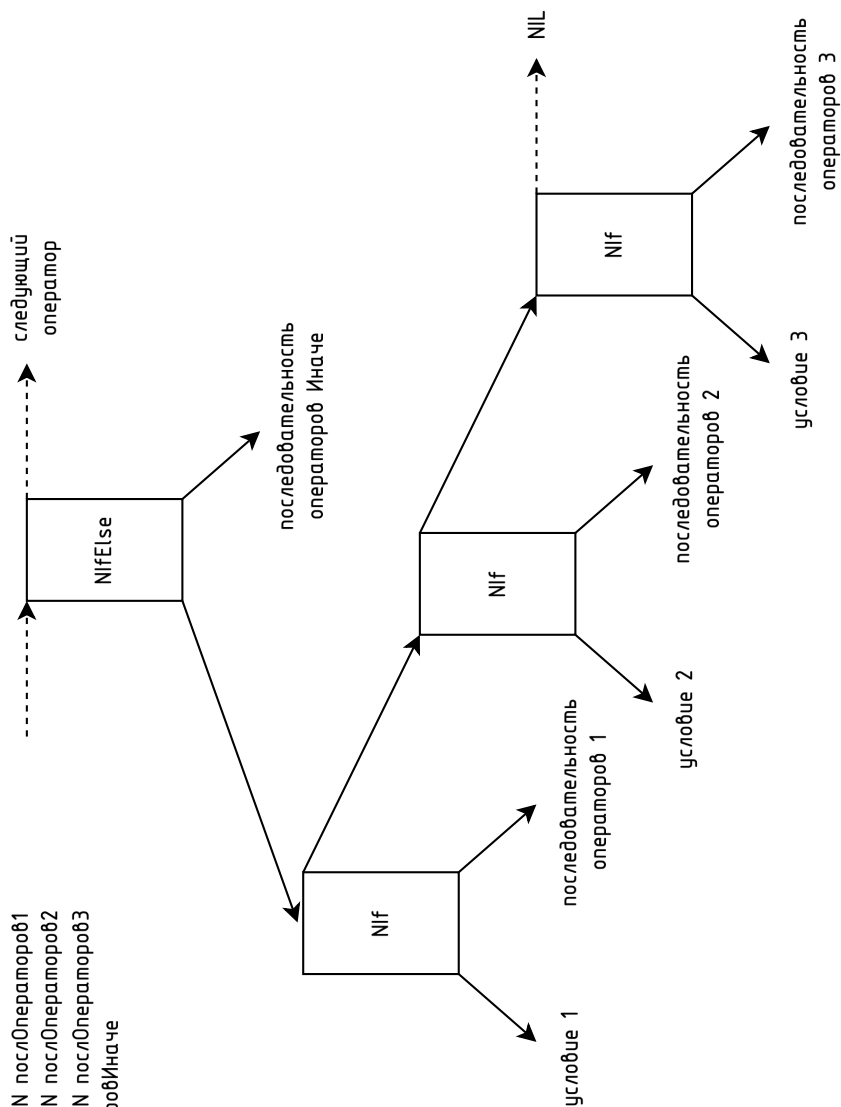


Рис. 9-1. Абстрактное синтаксическое дерево конструкции IF THEN ELSIF ELSE

ГЕНЕРАТОР КОДА

Генерация машинного кода относится к постпроцессору компилятора и технически выполнена в виде трёх модулей: обходчика дерева *Traverser*, испускателя машинного кода *Emitter* и низкоуровневый генератора *Generator*.

Обходчик получает подготовленное в препроцессоре абстрактное синтаксическое дерево. Это происходит только в том случае, если на этапе разбора исходного кода не возникло каких-либо ошибок. Общая координация организма компилятора осуществляется главным модулем *Mos*, который, собственно, и запускает препроцессор с постпроцессором.

Задача обходчика — обойти дерево в определённом порядке и на каждом этапе совершить соответствующий вызов процедуры модуля-испускателя. Испускатель, в свою очередь, вызовет необходимые процедуры генератора, в результате чего в массиве генератора будет набран соответствующий машинный код.

Модуль *Generator*, реализующий низкоуровневый генератор кода, и управляемый преимущественно модулем *Emitter*, имеет глобальную переменную `code: ARRAY maxCodeLen OF BYTE`, которая накапливает в себе производимый генератором машинный код. Для этого реализованы вспомогательные процедуры *Byte*, *Word*, *DWord*, *Set* и *String* — набор, очень похожий на соответствующий набор в компоновщике¹. Но если там вывод идёт в файл, то здесь все переданные этим процедурам значения складываются в массив `code`.

Переменная *pc* (*program counter*) в том же модуле выполняет роль указателя на индекс массива `code`, куда будет записан следующий байт, или — что то же самое — количество байтов, уже записанных в `code`.

Модуль *Emitter* содержит по одной процедуре на генерацию каждой инструкции в машинном коде. Его процедуры вызываются обходчиком *Traverser*.

¹ См. с. 84 настоящей работы.

КОМПОНОВЩИК

Если в ходе работы постпроцессора ошибок не произошло (в качестве таковых можно предположить сбой постпроцессора или вызов его нереализованных частей), главный модуль вызывает компоновщик, выполненный в виде модуля *Linker*. Процедура компоновщика *Link* получает в качестве параметра только имя выходного исполнимого файла.

Задача компоновщика заключается в том, чтобы записать различные части исполнимого файла в соответствии с форматом, принятым на целевой вычислительной системе. В нашем случае это операционная система *macOS* на базе процессора *Apple Silicon*, и формат файла называется *Mach-O*.

Составные части файла, разумеется, должны быть согласованы между собой. Основной его компонент — машинный код целевого процессора — обычно размещается в середине файла. К началу работы компоновщика итоговая длина машинного кода уже известна, а сам он находится в массиве модуля *Generator*, отвечающего за низкоуровневую генерацию.

Главная процедура компоновщика приведена в листинге 42).

Листинг 42. Главная процедура компоновщика

```

1  PROCEDURE Link*(exeFname: ARRAY OF CHAR);
2  BEGIN
3    Strings.Copy(exeFname, fname);
4    codeLen := Generator.CodeLength();
5    OpenOutputFile;
6    IF ¬Machine.hadErrors THEN
7      Output;
8    IF Machine.hadErrors THEN
9      Files.Close(F)
10     ELSE
11       Files.Register(F);
12       ChmodAndSign
13     END
14   END
15 END Link;
```

Для получения длины сгенерированного машинного кода в байтах вызывается функциональная процедура

Generator.CodeLength. Затем последовательно, с проверкой ошибок, вызываются процедуры компоновщика OpenOutputFile, Output и ChmodAndSign.

Процедура OpenOutputFile (см. листинг 43) открывает файл на запись. Имя файла было передано из главного модуля. Если файл открыть не удаётся, будет вызвана процедура сообщения об ошибке в модуле Machine. К этому моменту модуль Machine уже будет переведён в фазу постпроцессора, поэтому ошибка будет выведена без указания положения в файле с исходным кодом. Это важно, поскольку ошибки фазы постпроцессора не связаны с конкретным исходным кодом.

Листинг 43. Процедура компоновщика OpenOutputFile

```
1 PROCEDURE OpenOutputFile;
2 BEGIN
3   F := Files.New(fname);
4   IF F = NIL THEN
5     Machine.Error(Errors.outputFile)
6   ELSE
7     Files.Set(r, F, 0)
8   END
9 END OpenOutputFile;
```

Процедура Output (см. листинг 44) осуществляет необходимые вызовы процедур, выводящие части исполнимого файла: заголовок (процедура Header), команды (процедура Commands), специфичные для формата *Mach-O* так называемые цепные фиксации (процедура ChainedFixups), префиксное дерево экспортированных объектов (процедура ExportsTrie), разметку процедур (процедура FunctionStarts) и символьную таблицу (процедура SymbolTable).

Листинг 44. Процедура компоновщика Output

```
1 PROCEDURE Output;
2 BEGIN
3   Header;
4   Commands;
5
6   (* Zero-out until position 4000H - codeSpace *)
7   Zeros(4000H - codeSpace - Files.Pos(r));
8   (* Machine code *)
9   Generator.Output(r);
10  (* Unwind Info *)
```

```
11 Zeros(codeSpace - codeLen);
12
13 ChainedFixups;
14 ExportsTrie;
15 FunctionStarts;
16 SymbolTable
17 END Output;
```

Процедура `Commands` вызывает 16 других процедур: `Command0`, `Command1` и т. д. (см. листинг 45).

Листинг 45. Процедура компоновщика Commands

```
1 (** Outputs Mach-O commands to the file. *)
2 PROCEDURE Commands;
3 BEGIN
4   Command0;
5   Command1;
6   (* ... *)
7   Command15
8 END Commands;
```

Из-за требований выравнивания в исполнимом файле присутствуют значительные области, заполненные нулями. Например, секция «`unwind info`», содержащая отладочную информацию, в текущей версии компилятора заполняется нулями, что не влияет на корректность работы скомпилированной программы.

Процедура `ChmodAndSign` (см. листинг 46) выполняет два ключевых действия, необходимых для успешного запуска полученного исполнимого файла в операционной системе *macOS*.

Листинг 46. Процедура компоновщика ChmodAndSign

```
1  PROCEDURE ChmodAndSign;
2  VAR s: ARRAY 1024 OF CHAR;
3      err: INTEGER;
4  BEGIN
5      (* Chmod *)
6      s := 'chmod +x ';
7      Strings.Append(fname, s);
8      err := Machine.Exec(s);
9      IF err # 0 THEN
10         Machine.Error(Errors.chmodFailed)
11     ELSE
12         (* Sign *)
13         s := 'codesign -s - -f --timestamp=none ';
14         Strings.Append(fname, s);
15         err := Machine.Exec(s);
16         IF err # 0 THEN
17             Machine.Error(Errors.signFailed)
18         END
19     END
20 END ChmodAndSign;
```

Во-первых, она вызывает команду `chmod` с параметром «+x», помечая файл в файловой системе как исполнимый. Это действие обязательно для всех UNIX-подобных систем. В операционных системах Windows и DOS подобный шаг не требуется, поскольку исполнимость файла определяется по его расширению (.EXE, .COM, .BAT).

Во-вторых, процедура `ChmodAndSign` снабжает сгенерированный исполнимый файл цифровой подписью. Для этого она вызывает внешнюю утилиту `codesign`, которая доступна на всех современных версиях macOS. Без цифровой подписи запуск исполнимого файла будет заблокирован операционной системой. При этом, в целях безопасности, пользователю не будет отображено явное сообщение об ошибке.

Базовая подпись с помощью утилиты `codesign` может быть выполнена без участия в программе разработчика Apple, однако такая подпись не позволит обойти защиту Gatekeeper при распространении программ за пределами системы.

Компоновщик реализует вспомогательные процедуры Byte, Word, DWord, QWord, Set, String и Zeros (см. листинг 47), которые позволяют удобно осуществлять запись двоичных данных в файл.

Листинг 47. Пример вспомогательных процедур компоновщика

```
1  (** Выводит 1 байт в файл. *)
2  PROCEDURE Byte(x: BYTE);
3  BEGIN
4    Files.Write(r, x)
5  END Byte;
6
7  (** Выводит двойное слово = 4 байта в файл.
8    Порядок байтов от младшего к старшему. *)
9  PROCEDURE DWord(x: INTEGER);
10 BEGIN
11   Byte(x MOD 100H);
12   Byte(x DIV 100H MOD 100H);
13   Byte(x DIV 10000H MOD 100H);
14   Byte(x DIV 1000000H MOD 100H)
15 END DWord;
16
17 (** Выводит множество в файл как 4 байта.
18    Порядок байтов от младшего к старшему. *)
19 PROCEDURE Set(x: SET);
20 BEGIN
21   DWord(ORD(x))
22 END Set;
23
24 (** Выводит строку в файл (однобайтовые литеры)
25     с последующим добавлением нулевых байтов до
26     достижения общего количества в n байт. Если n
27     меньше длины строки, строка не усекается. *)
28 PROCEDURE String(s: ARRAY OF CHAR; n: INTEGER);
29 VAR i: INTEGER;
30 BEGIN
31   i := 0;
32   WHILE s[i] # 0X DO
33     Byte(ORD(s[i]));
34     INC(i)
35   END;
36   WHILE i < n DO
37     Byte(0);
38     INC(i)
```

```

39  END
40  END String;
41
42  (** Выводим n нулевых байтов в файл. *)
43  PROCEDURE Zeros(n: INTEGER);
44  BEGIN
45      WHILE n > 0 DO
46          Byte(0);
47          DEC(n)
48      END
49  END Zeros;

```

С процедуры `Header` начинается запись в исполнимый файл. Она представлена в листинге 48. Все записывающие процедуры компоновщика выглядят подобным образом. Они построены с использованием документации формата Mach-O и по результатам экспериментальных данных¹.

Листинг 48. Процедура вывода заголовка файла Mach-O

```

1  (** Выводим первые 32 байта - заголовок Mach-O - в файл. *)
2  PROCEDURE Header;
3  BEGIN
4      (* Магическое число FEED-FACF,
5      порядок байтов от младшего к старшему *)
6      Byte(0CFH); Byte(0FAH); Byte(0EDH); Byte(0FEH);
7      (* Тип ЦПУ = ARM64 *)
8      DWord(100000CH);
9      (* Подтип ЦПУ = ARM64_ALL *)
10     DWord(0);
11     (* Тип файла = MH_EXECUTE *)
12     DWord(2);
13     (* Количество команд загрузки *)
14     DWord(15);
15     (* Размер команд загрузки в байтах *)
16     DWord(744-16);
17     (* Флаги = {NOUNDEFS, DYLDLINK, TWOLEVEL, PIE} *)
18     Set({0, 2, 7, 21});
19     (* Резервировано = 0 *)
20     DWord(0)
21  END Header;

```

¹ См. гл. 13 на с. 91 настоящей работы.

11.1. Использование типа SET

Обыкновенно флаги обозначаются в виде целого числа в шестнадцатеричной записи и составляются с помощью поразрядной операции ИЛИ:

$$0x00200085 = 0x1 \mid 0x4 \mid 0x80 \mid 0x200000$$

Запись числа в виде `0x200000` не несёт никакой смысловой нагрузки, кроме того, что это константа, которая подлежит сцеплению с другими подобными константами посредством поразрядных операций.

В Обероне, а ранее в Модуле-2, существует тип SET. Это не тот же тип SET, который был в Паскале. В Обероне он определяется как множество небольших целых чисел [14, с. 29]. Количество этих чисел фактически определяется архитектурой целевой вычислительной системы, так как важно добиться, чтобы операции над множествами могли быть эффективно реализованы как поразрядные (битовые) операции. Характерная для современных систем мощность такого множества равна 32. На более старых системах — 16, на некоторых новых — 64. Чаще всего она равна машинному слову в битах. Идея типа данных, который бы на более высоком уровне абстракции использовал компактное представление массива булевых значений как битовых множеств, используя математически привлекательные концепции.

Никлаус Вирт указывает, что идея этого типа данных принадлежит Чарльзу Энтони Ричарду Хоару [75]. Она возникла из попытки поднять такие типы, как Bits и Bitset, на более высокий уровень математически привлекательной абстракции.

Синтаксически в языке Оберон значение (литерал) типа SET задаётся с помощью фигурных скобок:

`{0, 1, 5..8, 15}`

На машинном уровне такое множество будет представлять собой 4 байта памяти, в которых биты-единицы будут обозначать присутствие соответствующего элемента в множестве.

Таким образом, для представления флагов `0x00200085` в Обероне будет использована запись `{0, 2, 7, 21}`. Такая запись делает наглядным и тот факт, что флагов четыре. По записи `0x00200085` это не сразу видно, так как цифра 5 обозначает два флага ($4 + 1$).

Фактически, каждое число в фигурных скобках означает количество нулей, которое надо отступить справа в записи

числа в его двоичной форме, прежде чем поставить единицу. Или, что то же самое, число два, возведённое в степень номера элемента. Таким образом, множество $\{0\}$ означает число $0x00000001$, множество $\{1\}$ — число $0x00000002$, множество $\{2\}$ — $0x00000004$ и так далее. Удобство множеств по сравнению с шестнадцатеричными числовыми флагами становится особенно наглядным при использовании больших чисел. Число $0x00200000$ в виде множества представляется как короткое $\{21\}$.

Глава 12

ОСОБЕННОСТИ АРХИТЕКТУРЫ APPLE SILICON

Как уже указывалось ранее, исполнимые файлы в операционной системе macOS имеют формат Mach-O. Ключевой составляющей исполнимого файла является сегмент, содержащий машинный код, непосредственно предназначенный для исполнения на целевой архитектуре. Рассмотрим принципы построения этого машинного кода.

Архитектура ARM64, лежащая в основе процессоров Apple Silicon, использует фиксированный размер инструкции: каждая инструкция занимает ровно 4 байта, то есть 32 бита. Это свойство отличает её от архитектур с переменной длиной инструкций (например, x86), и существенно упрощает как аппаратное декодирование команд, так и анализ машинного кода человеком. Следует отметить, что 64-битная разрядность архитектуры относится не к длине инструкции, а к ширине регистров общего назначения и адресного пространства. Таким образом, инструкции остаются компактными, несмотря на расширенные возможности.

Фиксированная длина инструкций в ARM64 упрощает задачу визуального анализа: каждое двойное слово (4 байта) в шестнадцатеричном выводе машинного кода можно с уверенностью интерпретировать как инструкцию. Для сравнения, архитектура x86 и её 64-битное расширение x86_64 используют переменную длину инструкций — от 1 до 15 байт [55], — что приводит к сложности синхронизации при анализе кода с произвольного

адреса, так как делает границы между инструкциями неочевидными.

Примером может служить следующий фрагмент кода ARM64:

```
9100 0400    mov x0, #0x20
D280 0021    mov x1, #0x1
```

Здесь легко увидеть, что каждая строка представляет собой ровно одну инструкцию — 4 байта, 8 шестнадцатеричных цифр.

Кроме того, ARM64 построена по принципам RISC (Reduced Instruction Set Computer), что означает ограниченный набор инструкций, более простое и предсказуемое их поведение, отсутствие скрытых побочных эффектов и ориентацию на однотипные форматы команд. Это делает архитектуру особенно удобной для компиляторов, которые могут с высокой степенью детерминированности формировать машинный код без необходимости глубокого анализа сложных инструкций или их комбинаций, как в случае CISC-архитектур (к которым относится, например, x86).

ARM64 в контексте проектирования компилятора предоставляет значительные преимущества: как в части генерации кода, так и в части анализа его корректности, особенно на этапе отладки или обучения.

12.1. Инструкции, регистры и соглашения

Архитектура ARM64 использует набор инструкций A64, введенный в спецификации ARMv8-A. A64 является RISC-набором инструкций, в котором каждая команда выполняет простую операцию. Инструкции делятся на несколько категорий:

1. Арифметико-логические операции: ADD, SUB, AND, ORR, EOR, LSL, LSR, ASR.
2. Операции с ОЗУ: LDR, STR, LDUR, STUR, LDP, STP.
3. Управление потоком исполнения: B, BL, RET, BR, CBZ, CBNZ, TBZ, TBNZ.
4. Системные инструкции: MSR, MRS, NOP, ISB, DSB, DMB.

ARM64 предоставляет 31 универсальный 64-битный регистр общего назначения (X0—X30) и специальный регистр SP (stack pointer). Также доступны 32 векторных регистра (V0—V31) для операций с плавающей запятой и SIMD¹.

В соответствии с AAPCS64 (Procedure Call Standard for the ARM 64-bit Architecture), параметры функций передаются

¹ SIMD означает Single Instruction, Multiple Data — одна инструкция, множество данных.

через регистры X0—X7, а возвращаемое значение — через X0. Регистры X19—X28 являются сохраняемыми вызываемой процедурой (callee-saved) и должны быть восстановлены к моменту возврата из процедуры. Регистр X30 (LR) используется для хранения адреса возврата [76].

Пример. Рассмотрим простую функцию на языке Оберон:

```
1 PROCEDURE Add(a, b: INTEGER): INTEGER;  
2 RETURN a + b END Add;
```

Компилятор может транслировать её в следующий A64-код:

```
add:  
    ADD X0, X0, X1  
    RET
```

Здесь параметры *a* и *b* передаются через X0 и X1, результат возвращается в X0.

12.2. Особенности системных вызовов на macOS

Архитектура системных вызовов в macOS построена на основе ядра XNU (X is Not Unix), сочетающего элементы Mach, BSD и IOKit. Несмотря на внешнее сходство с Unix-подобными системами, системные вызовы macOS имеют ряд отличий, как в интерфейсе, так и в реализации.

На архитектуре ARM64 системные вызовы осуществляются с помощью инструкции `svc #0` (Supervisor Call). Эта инструкция инициирует переход в привилегированный режим и передаёт управление ядру. Номер системного вызова и его аргументы передаются в регистрах согласно соглашению AAPCS64, принятому в среде Darwin.

Для системных вызовов в macOS используются правила, идентичные обычным процедурным вызовам:

- 1) регистр X16 содержит номер системного вызова;
- 2) аргументы передаются в регистрах X0—X7;
- 3) возвращаемое значение помещается в X0;
- 4) ошибки указываются через значение в X0 и код `errno`.

Пример. Вызов функции `write(fd, buffer, size)` на уровне системного интерфейса выглядит следующим образом:

1	<code>mov x0, #1</code>	; дескриптор (stdout)
2	<code>adr x1, buffer</code>	; указатель на буфер
3	<code>mov x2, #13</code>	; длина
4	<code>mov x16, #0xFFFF</code>	; системный селектор (магическое
5		значение для Apple)
6	<code>mov x8, #0x2000004</code>	; номер системного вызова <code>write</code>
7	<code>svc #0</code>	; собственно вызов

Здесь `0x2000004` — это системный номер функции `write` в пространстве вызовов macOS. Префикс `0x2000000` означает, что это системный вызов BSD (в отличие от Mach или IOKit).

В отличие от Linux, где системные вызовы обычно реализованы через `svc #0` с номерами без префикса (например, `write = 64` на ARM64), macOS использует уникальную схему нумерации и дополнительные селекторы. Более того, взаимодействие с ядром в macOS часто происходит не напрямую, а через обёртки в `libSystem.dylib`, которые реализуют стандарт POSIX API.

Для генерации нативного кода под macOS компилятор должен учитывать особенности соглашения вызова системных функций, специфику нумерации и структуру передачи параметров. Неправильная реализация интерфейса системных вызовов приводит к ошибкам исполнения или некорректному взаимодействию с ядром. В связи с этим понимание механизма системных вызовов Darwin ABI имеет ключевое значение при проектировании постпроцессора компилятора.

Глава 13

ИССЛЕДОВАНИЕ ФОРМАТА ФАЙЛА MACH-O

Рассмотрим исполнимый файл `«01_exit»`, который можно получить из объектного файла при помощи следующей команды:

```
ld 01_exit.o -o 01_exit -lSystem -syslibroot  
`xcrun -sdk macosx --show-sdk-path` -e __start
```

Воспользуемся для исследования инструментом «MachO-View» — графической программой, отображающей содержимое файлов Mach-O в удобном виде. Первые 32 байта представляют собой заголовок Mach-0: (см. рис. 13-1).

Offset	Data	Description	Value
00000000	FEEDFACF	Magic Number	MH_MAGIC_64
00000004	0100000C	CPU Type	CPU_TYPE_ARM64
00000008	00000000	CPU SubType	00000000
			CPU_SUBTYPE_ARM64_ALL
0000000C	00000002	File Type	MH_EXECUTE
00000010	00000010	Number of Load Commands	16
00000014	000002E8	Size of Load Commands	744
00000018	00200085	Flags	00000001
			MH_NOUNDEFS
			00000004
			MH_DYLDLINK
			00000080
			MH_TWOLEVEL
			00200000
			MH_PIE
0000001C	00000000	Reserved	0

Рис. 13-1. Заголовок Mach-O

Заголовок Mach64 (первые 32 байта).

По смещению 1016 видим количество команд загрузки, а именно 16. В простейшем *объектном* файле их было бы 4. По смещению 1416 видим значение 744. Это суммарный размер всех команд загрузки в байтах.

Если в объектном файле все флаги были бы сняты (значение Flags было равно нулю), то в исполнимом файле оно равно 20 008 516, что соответствует четырём флагам со значениями 2^0 , 2^2 , 2^7 и 2^{21} . Вот их значения:

2^0 = NOUNDEFS — файл не содержит неопределённых символов;
 2^2 = DYLDLINK — файл собран для работы с динамическим компоновщиком;

2^7 = TWOLEVEL — используется двухуровневая система именования символов;

2^{21} = PIE — исполнимый файл является позиционно-независимым.

Двухуровневая система именования символов. В отличие от системы плоского пространства имён (Flat Namespace) двухуровневая система именования символов (Two-Level Namespace) в Mach-O позволяет избежать конфликтов между библиотеками (фреймворками), если они определяют символы с одинаковыми именами, поскольку двухуровневая система с каждым символом соотносит конкретную динамическую библиотеку.

Команды загрузки. После заголовка в файле Mach-O идёт указанное количество команд загрузки. Некоторые команды определяют целые сегменты в ОЗУ, а другие несут вспомогательную информацию. В исполнимом файле, который мы рассматриваем, объявлено 3 сегмента.

Первый из них — «__PAGEZERO» (см. рис. 13-2) — занимает в виртуальной памяти 4 Гбайт и, в общем-то, в этом и заключается его предназначение. Он явно закрывает эту память от доступа со стороны приложения, т. к. не определяет в сегменте ни одной секции.

Offset	Data	Description	Value
00000020	00000019	Command	LC_SEGMENT_64
00000024	00000048	Command Size	72
00000028	5F5F504147455A45524F000...	Segment Name	__PAGEZERO
00000038	0000000000000000	VM Address	0
00000040	0000000100000000	VM Size	4294967296
00000048	0000000000000000	File Offset	0
00000050	0000000000000000	File Size	0
00000058	00000000	Maximum VM Protection	
	00000000		VM_PROT_NONE
0000005C	00000000	Initial VM Protection	
	00000000		VM_PROT_NONE
00000060	00000000	Number of Sections	0
00000064	00000000	Flags	

Рис. 13-2. Сегмент № 0

Второй сегмент носит имя «__TEXT» (см. рис. 13-3) и в нашем случае занимает 88 байта. Содержимое этого фрагмента памяти будет заполнено 88 байтами из файла по смещению 16 296.

Offset	Data	Description	Value
00000068	00000019	Command	LC_SEGMENT_64
0000006C	000000E8	Command Size	232
00000070	5F5F544558540000000000...	Segment Name	__TEXT
00000080	0000000100000000	VM Address	4294967296
00000088	0000000000004000	VM Size	16384
00000090	0000000000000000	File Offset	0
00000098	0000000000004000	File Size	16384
000000A0	00000005	Maximum VM Protection	
		00000001	VM_PROT_READ
		00000004	VM_PROT_EXECUTE
000000A4	00000005	Initial VM Protection	
		00000001	VM_PROT_READ
		00000004	VM_PROT_EXECUTE
000000A8	00000002	Number of Sections	2
000000AC	00000000	Flags	

Рис. 13-3. Сегмент № 1

Третий сегмент носит имя «__LINKEDIT» (см. рис. 13-4) и выполняет массу служебных релей. В нашем случае данный сегмент занимает 16384 байта. Содержимое этого сегмента памяти будет заполнено первыми 16384 байтами из файла.

Offset	Data	Description	Value
00000150	00000019	Command	LC_SEGMENT_64
00000154	00000048	Command Size	72
00000158	5F5F4C494E4B4544495400...	Segment Name	__LINKEDIT
00000168	0000000100004000	VM Address	4294983680
00000170	0000000000004000	VM Size	16384
00000178	0000000000004000	File Offset	16384
00000180	00000000000001C8	File Size	456
00000188	00000001	Maximum VM Protection	
		00000001	VM_PROT_READ
0000018C	00000001	Initial VM Protection	
		00000001	VM_PROT_READ
00000190	00000000	Number of Sections	0
00000194	00000000	Flags	

Рис. 13-4. Сегмент № 2

Рассматривая команды загрузки Mach-O одну за другой, внося изменения в бинарный файл, можно изучить этот формат на практике, что и было сделано при изготовлении данного компилятора.

ЗАКЛЮЧЕНИЕ

В ходе выполненной работы была достигнута поставленная цель — создан нативный компилятор языка Оберон для процессоров Apple Silicon (ARM64) под macOS, полностью реализующий все этапы преобразования исходного кода: лексический и синтаксический анализ, проверку типов, построение промежуточного представления, генерацию машинного кода и компоновку исполняемого файла в формате Mach-O.

Для начального «раскручивания» компилятора на macOS/ARM64 была портирована среда Free Oberon и транслятор Оберона в Си Ofront+, при этом были исправлены ошибки в передаче REAL-параметров по ABI ARM64. В результате получен работоспособный прототип, пригодный не только для практического использования, но и для дальнейшего развития — в том числе готовый к перенацеливанию на другие архитектуры, внедрения оптимизаций и расширения функциональности.

Разработанный инструмент может служить наглядным учебным материалом при изучении технологий построения компиляторов и особенностей ARM64-машинного кода, а его исходные тексты легко интегрируются в открытые образовательные и исследовательские проекты.

В перспективе целесообразно расширить возможности компилятора за счёт:

- 1) внедрения фаз оптимизации кода;
- 2) поддержки различных диалектов Оберон;
- 3) автоматизации процесса сборки под различные ОС и аппаратные платформы;
- 4) проведения сравнительных тестов производительности (бенчмарков);
- 5) создание автоматической системы распространения компонентов.

Таким образом, работа не только закрыла пробел в экосистеме Оберона на Apple Silicon, но и заложила прочную основу для её дальнейшего развития и применения в образовательных и исследовательских целях.

Исходный код компилятора опубликован в GitHub-репозитории github.com/kekcleader/moc и на сайте moc.oberon.org.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Niklaus Wirth. Niklaus Wirth Official Website: Oberon. <https://people.inf.ethz.ch/wirth/Oberon/index.html>, 2016. Дата обращения: 04.05.2025.
2. Clemens Szyperski. *Insight ETHOS: On Object Orientation in Operating Systems*, volume 40 of *Informatik-Dissertationen ETH Zürich*. Verlag der Fachvereine Hochschulverlag AG an der ETH Zürich, Zürich, Switzerland, 1992. ISBN 3-7281-1948-2. PhD thesis: Swiss Federal Institute of Technology (ETH Zurich), Dissertation No 9884.
3. Информатика-21. Оберон на Ростовской АЭС, 2016. URL https://informatika-21.ru/oberon_innovation/oberonRostovAES.htm. Дата обращения: 04.05.2025.
4. Информатика-21. Оберон в энергетике, 2016. URL https://informatika-21.ru/oberon_innovation/oberonKaliningrad.htm. Дата обращения: 04.05.2025.
5. Информатика-21. Оберон и промышленная автоматизация в агропроме, 2016. URL https://informatika-21.ru/oberon_innovation/oberonAgro.htm. Дата обращения: 04.05.2025.
6. C. Eck, J. Chapuis, and H. P. Geering. Inexpensive Autopilots for Small Unmanned Helicopters. *Proceedings of the 2002 IEEE International Conference on Robotics and Automation*, pages 3005—3010, 2002. URL https://www.researchgate.net/publication/259822282_Inexpensive_Autopilots_for_Small_Unmanned_Helicopters. Дата обращения: 04.05.2025.
7. Информатика-21. Оберон в БПЛА, 2012. URL https://informatika-21.ru/oberon_innovation/oberonBPLA.htm. Дата обращения: 04.05.2025.
8. Thomas Kaegi-Trachsel and Jürg Gutknecht. Minos—the design and implementation of an embedded real-time operating system with a perspective of fault tolerance. In *Proceedings of the International Multiconference on Computer Science and Information Technology (IMCSIT)*, pages 649—656, Wisla, Poland, 2008. https://www.researchgate.net/publication/220947906_Minos-the_design_and_implementation_of_an_embedded_real-time_operating_system_with_a_perspective_of_fault_tolerance. Дата обращения: 04.05.2025.

9. Wojtek Skulski. Experiment control and data acquisition using BlackBox Component Builder. https://www.pas.rochester.edu/~skulski/Presentations/BB_Class.pdf, 2004. Презентация на CERN Oberon Day.
10. CFB Software. Astrobe—An Oberon development system for RISC5 FPGA systems. <https://www.astrobe.com/RISC5/>. Среда разработки для встраиваемых систем на языке Оберон. Дата обращения: 04.05.2025.
11. Gray. Oberon RTS—Embedded Oberon for Real-time Systems. <https://oberon-rts.org/about/forums/>, 2025. Форум по встраиваемым системам реального времени на базе Оберона. Дата обращения: 04.05.2025.
12. OberonCore. Применения языка Оберон, 2025. URL <https://oberoncore.ru/infobase/applications>. Дата обращения: 04.05.2025.
13. J. M. Spivey. Oxford Oberon-2 Compiler. https://spivey.oriel.ox.ac.uk/corner/Oxford_Oberon-2_compiler, 2024. Дата обращения: 04.05.2025.
14. Niklaus Wirth and Martin Reiser. *Programming in Oberon: Steps Beyond Pascal and Modula*. Addison-Wesley, Reading, Massachusetts, 1992. ISBN 0-201-56543-9.
15. OberonCore forum users. Курсы по программированию на языке Оберон в Риге. <https://forum.oberoncore.ru/viewtopic.php?f=35&t=6809>, 2021. Дата обращения: 04.05.2025.
16. BlackBox Component Builder Team. BlackBox Component Builder / Component Pascal. <https://blackboxframework.org/index.php?cID=home,en-us>, 2024. «The compiler is very fast». Дата обращения: 04.05.2025.
17. BlackBox Component Builder Team. BlackBox Component Builder. <https://wiki.oberon.org/blackbox>, 2024. Расширяемая среда разработки приложений на языке Компонентный Паскаль. Дата обращения: 04.05.2025.
18. Niklaus Wirth and Jürg Gutknecht. *Project Oberon: The Design of an Operating System, a Compiler, and a Computer*. ETH Zürich, revised edition edition, 2013. ISBN 0-201-54428-8.
19. Niklaus Wirth. Project Oberon (New Edition 2013). <https://people.inf.ethz.ch/wirth/ProjectOberon/index.html>. Дата обращения: 04.05.2025.

20. Niklaus Wirth. The Design of a RISC Architecture and its Implementation with an FPGA. Technical report, ETH Zürich, 2015. URL <https://people.inf.ethz.ch/wirth/FPGA-relatedWork/RISC.pdf>. Revision 1.9, November 2015. Дата обращения: 04.05.2025.
21. re2c Team. re2c—Regular Expressions to Code. <https://re2c.org/>, 2025. Дата обращения: 12.05.2025.
22. Anton Sukhachev. Parsing with PHP, Bison and re2c. <https://mrsuh.com/articles/2022/parsing-with-php-bison-and-re2c/>, 2022. Дата обращения: 12.05.2025.
23. Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*, pages 75—86, Palo Alto, CA, USA, 2004. IEEE Computer Society. <https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf> Дата обращения: 03.05.2025.
24. Qi Zhou and Zhendong Su. Toward Understanding Compiler Bugs in GCC and LLVM. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 907—921, Santa Barbara, CA, USA, 2016. ACM. <https://people.inf.ethz.ch/suz/publications/issta16-compiler-bug-study.pdf> Дата обращения: 03.05.2025.
25. Edsger Wybe Dijkstra. *A Discipline of Programming*. Englewood Cliffs, N. J.: Prentice-Hall, 1976. ISBN 978-0132158718. URL <https://archive.org/details/disciplineofprog0000dijk>.
26. Josef Templ. Ofront (Oberon2-to-C Translator). <https://wiki.oberon.org/ob/ofront>, 2020. Дата обращения: 12.05.2025.
27. Josef Templ. Ofront—Oberon-2 to C translator written in Oberon-2. <https://github.com/jtempl/ofront>, 2017. Дата обращения: 12.05.2025.
28. Чередниченко Олег Николаевич. Транслятор Оберон-языков в Си — Ofront+. <https://zx.oberon.org/ofrontplus>, 2012. Дата обращения: 12.05.2025.
29. Stewart Greenhill. [Oberon] Ofront+. <https://lists.inf.ethz.ch/pipermail/oberon/2020/015172.html>, 2020. Дата обращения: 12.05.2025.

30. Josef Templ and Oberon microsystems AG. CPfront—Component Pascal to C Translator. <https://blackbox.oberon.org/extension/CPfront>, 2019. Дата обращения: 12.05.2025.
31. Hacker News Community. Discussion on Vishap Oberon Compiler. <https://news.ycombinator.com/item?id=43626617>, 2025. Дата обращения: 12.05.2025.
32. Antranig Vartanian. Vishap Oberon Compiler. <https://github.com/vishapoberon/compiler>, 2014. Дата обращения: 12.05.2025.
33. Фольц Владислав Сергеевич. OberonJS—Oberon 07 compiler to Java Script. <https://github.com/vladfolts/oberonjs>, 2020. Дата обращения: 13.05.2025.
34. Фольц Владислав Сергеевич. Try OberonJS compiler online. <https://github.com/vladfolts/oberonjs>, 2020. Дата обращения: 13.05.2025.
35. Денисов Иван Андреевич. Online Oberon Compiler. <https://online.oberon.org/>, 2019. Дата обращения: 13.05.2025.
36. Michael Schierl. Project Oberon emulator in JavaScript and Java. <https://schierlm.github.io/OberonEmulator/>, 2020. Дата обращения: 13.05.2025.
37. Luis Boasso. oberonc: An Oberon-07 compiler for the JVM. <https://github.com/lboasso/oberonc>, 2023. Дата обращения: 13.05.2025.
38. K. John Gough. Gardens Point Component Pascal (GPCP). <https://github.com/k-john-gough/gpcp>, 2022. Дата обращения: 13.05.2025.
39. Дагаев Дмитрий Викторович. Компилятор МультиОберон. <https://oberoncore.ru/projects/multioberon>, 2019. Дата обращения: 12.05.2025.
40. Дагаев Дмитрий Викторович. Руководство пользователя компилятора MultiOberon. https://github.com/dvdagaev/Mob/blob/master/doc/MultiOberonCompilerUserGuide_ru.pdf, 2019. Дата обращения: 12.05.2025.
41. Дагаев Дмитрий Викторович. Проект MultiOberon. <https://forum.oberoncore.ru/viewtopic.php?f=157&t=6423#p108626>, 2019. Дата обращения: 12.05.2025.
42. Ефимов Артур Игоревич. Free Oberon official website. <https://free.oberon.org/en/>, 2016. Дата обращения: 15.05.2025.

43. Ефимов Артур Игоревич. Issue #131: RECORD passed by value with unexported REAL fields not working on ARM64. <https://github.com/Oleg-N-Cher/OfrontPlus/issues/131>, 2025. Дата обращения: 12.05.2025.
44. ARM Limited. ARM Architecture Reference Manual ARMv8, for ARMv8-A architecture profile. <https://developer.arm.com/documentation/ddi0487/latest>, 2021. Дата обращения: 12.05.2025.
45. Apple Inc. Mach-O File Format Reference. <https://developer.apple.com/library/archive/documentation/DeveloperTools/Conceptual/MachOTopics/>, 2009. Дата обращения: 12.05.2025.
46. Matt Pietrek. An In-Depth Look into the Win32 Portable Executable File Format. *MSDN Magazine*, Февраль 2002. URL https://coffi.readthedocs.io/en/latest/pe_format_in_depth_look_part1.pdf.
47. Apple Inc. Apple Platform Security Overview. <https://support.apple.com/guide/security/welcome/web>, 2024. Дата обращения: 12.05.2025.
48. Niklaus Wirth. The Programming Language Oberon. Technical Report Revision 1.10.2013 / 3.5.2016, ETH Zurich, 2016. URL <https://people.inf.ethz.ch/wirth/Oberon/Oberon07.Report.pdf>. Дата обращения: 15.05.2025.
49. Component Pascal Language Report. Technical report, Oberon Microsystems, Inc., Октябрь 2007. URL <https://blackbox.oberon.org/cp-lang.pdf>. Дата обращения: 15.05.2025.
50. Andreas Pirklbauer. The Programming Language Oberon-2 (2020 Edition). <https://github.com/andreaspirklbauer/Oberon-extended/blob/master/Documentation/The-Revised-Oberon2-Programming-Language.pdf>, 2020. Дата обращения: 17.05.2025.
51. Greg Comeau. A Guide to Understanding Even the Most Complex C Declarations. *Microsoft Systems Journal*, 3(5), Сентябрь 1988. URL <https://jacobfilipp.com/MSJ/1988-volume-3.html#no5>. Дата обращения: 15.05.2025.
52. Ramesh Yerraballi. C Declarations: A Short Primer. <https://users.ece.utexas.edu/~ryerraballi/CPrimer/CDeclPrimer.html>, 1998. Дата обращения: 15.05.2025.
53. Stack Overflow community. Complicated declarations of C, 2013. URL <https://stackoverflow.com/questions/14426505/complicated-declarations-of-c>. Дата обращения: 15.05.2025.

54. Niklaus Wirth. *Compiler Construction*. Self-published, Zürich, 2017. ISBN 0-201-40353-6. A slightly revised version of the book originally published by Addison-Wesley in 1996. ISBN 0-201-40353-6.
55. Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 2024. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Дата обращения: 12.05.2025.
56. Legato Application Framework Team. C Language Coding Standards. https://docs.legato.io/14_04/ccoding_stds_name_funcs.html, 2014. Дата обращения: 03.05.2025.
57. GTK Development Team. Text Widget Overview. <https://docs.gtk.org/gtk4/section-text-widget.html>, 2025. Дата обращения: 03.05.2025.
58. The Go Authors. The Go Programming Language Specification—Import declarations. https://go.dev/ref/spec#Import_declarations, 2024. Дата обращения: 03.05.2025.
59. Brian Kirk. The Oakwood Guidelines for Oberon-2 Compiler Developers. In *Proceedings of the Oakwood Conference on Oberon-2 Compiler Development*, Croydon, United Kingdom, Июнь 1993. Robinson Associates. Revision 1A, October 1995. https://oberoncore.ru/library/the_oakwood_guidelines_for_oberon-2_compiler_developers. Дата обращения: 19.05.2025.
60. Yanqing Wang. Identifier Naming Conventions and Software Coding Standards: A Case Study in One School of Software. <https://www.researchgate.net/publication/251981402>, 2010. Дата обращения: 03.05.2025.
61. Reem S. Alsuhaibani and Christian D. Newman and Michael J. Decker and Michael L. Collard and Jonathan I. Maletic. On the Naming of Methods: A Survey of Professional Developers. <https://www.cs.kent.edu/~jmaletic/papers/ICSE2021.pdf>, 2021. Дата обращения: 03.05.2025.
62. Edsger W. Dijkstra. Go To Statement Considered Harmful. *Communications of the ACM*, 11(3):147—148, 1968. doi: 10.1145/362929.362947. URL <https://dl.acm.org/doi/10.1145/362929.362947>.
63. К. Рустан М. Лейно. *Доказательство корректности программ*. ДМК-Пресс, Москва, 2024. ISBN 978-5-93700-199-3. Перевод с английского: А. Киселев; научный редактор: Д. Кондратьев.

64. Dmitrijs Ciruks. Programmas koda matemātiskā verifikācija, 2024. Darba vadītājs: Prof. Dr. math. Uldis Strautiņš. Recenzents: Dr. math. Laura Leja.
65. Elliotte Rusty Harold. Operator Overloading Considered Harmful, 2007. URL <https://cafe.elharo.com/programming/operator-overloading-considered-harmful/>. Дата обращения: 15.05.2025.
66. Niklaus Wirth. *Programming: A Tutorial*. ETH Zürich, Zürich, Switzerland, 2015. URL <https://people.inf.ethz.ch/wirth/Oberon/PIO.pdf>. Дата обращения: 05.05.2025.
67. Mark Cashman. The benefits of enumerated types in Modula-2. *ACM SIGPLAN Notices*, 26(2):35—39, Январь 1991. doi: 10.1145/122179.122183. URL <https://doi.org/10.1145/122179.122183>. Дата обращения: 16.05.2025.
68. Ермаков Илья Евгениевич. «От динамических средств расширения семантики Компонентного Паскаля — к расширяемому синтаксису». День Оберона в рамках II отраслевой конференции «Оберон-технологии, образование и проблема качества в цифровой индустрии» 2019 в г. Орле. <https://www.youtube.com/watch?v=v71zsKrF60Y>, Февраль 2020. Дата обращения: 16.05.2025.
69. Свездлов Сергей Залманович. *Языки программирования и методы трансляции: Учебное пособие*. Лань, Санкт-Петербург, 2-е изд., испр. edition, 2019. ISBN 978-5-8114-3457-2. URL https://free.oberon.org/files/Methody_transljacii.pdf.
70. Молдованова Ольга Владимировна. *Языки программирования и методы трансляции: Учебное пособие*. СибГУТИ, Новосибирск, 2012. URL <https://csc.sibsutis.ru/sites/csc.sibsutis.ru/files/courses/trans/LanguagesAndTranslationMethods.pdf>. Утверждено редакционно-издательским советом ФГОБУ ВПО «СибГУТИ» в качестве учебного пособия.
71. Бовсуновский Андрей Андреевич. InterOberon. <https://github.com/oberoncompiler/InterOberon>, 2023. InterOberon: Internationalized Oberon Compiler, GitHub.
72. Компилятор Интер-Оберон: программирование на русском и других языках. <https://www.youtube.com/watch?v=6D90j9f8AdY>, 2023. Доклад на международной онлайн-конференции «Неделя Оберона в России — 2023», YouTube.

73. Régis Crelier. *OP2: A Portable Oberon-2 Compiler*. Institut für Computersysteme, ETH Zürich, Zürich, 1990. <http://oberon2005.oberoncore.ru/paper/eth125.pdf>. Дата обращения: 19.05.2025.
74. Régis Crelier. *OP2: A Portable Oberon-2 Compiler*. In *Proceedings of the 2nd International Modula-2 Conference*, Loughborough, UK, 1991. https://oberoncore.ru/_media/library/crelier_r.op2_a_portable_oberon_2_compiler.en.pdf. Дата обращения: 19.05.2025.
75. Niklaus Wirth. SET: A neglected data type, and its compilation for the ARM. *ETH Zürich, Institut für Computer Systeme*, 2007. URL <https://people.inf.ethz.ch/wirth/Oberon/SETs.pdf>. Дата обращения: 11.05.2025.
76. ARM Limited. *Procedure Call Standard for the ARM 64-bit Architecture (AArch64)*, 2024. URL <https://github.com/ARM-software/abi-aarch64/releases/download/2024Q3/aapcs64.pdf>. Version 2024Q3. Дата обращения: 15.05.2025.

ПРИЛОЖЕНИЕ А. СИНТАКСИС ОБЕРОНА

буква = "A" | "B" | ... | "Z" | "a" | "b" | ... | "z".
цифра = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" |
"8" | "9".
шестнЦифра = цифра | "A" | "B" | "C" | "D" | "E" | "F".
идент = буква {буква | цифра}.
уточнИдент = [идент "."] идент.
идентОпр = идент ["*"].
целое = цифра {цифра} | цифра {шестнЦифра} "H".
вещественное = цифра {цифра} "." {цифра} [порядок].
порядок = ("E") ["+" | "-"] цифра {цифра}.
число = целое | вещественное.
строка = "" {литера} "" | цифра {шестнЦифра} "X".
ОбъявлениеКонстанты = идентОпр "=" КонстантноеВыражение.
КонстантноеВыражение = выражение.
ОбъявлениеТипа = идентОпр "=" тип.
тип = уточнИдент | ТипМассив | ТипЗапись | ТипУказатель |
ПроцедурныйТип.
ТипМассив = ARRAY длина {"," длина} OF тип.
длина = КонстантноеВыражение.
ТипЗапись = RECORD ["(" БазовыйТип ")"]
[ПоследСпискаПолей] END.
БазовыйТип = уточнИдент.
ПоследСпискаПолей = СписокПолей {";" СписокПолей}.
СписокПолей = СписокИдент ":" тип.
СписокИдент = идентОпр {"," идентОпр}.
ТипУказатель = POINTER TO тип.
ПроцедурныйТип = PROCEDURE [ФормальныеПараметры].
ОбъявлениеПеременных = СписокИдент ":" тип.
выражение = ПростоеВыражение [отношение ПростоеВыражение].
отношение = "=" | "#" | "<" | "<=" | ">" | ">=" | IN | IS.
ПростоеВыражение = ["+" | "-"] выражение
{ОперацияСложения выражение}.
ОперацияСложения = "+" | "-" | OR.
выражение = множитель {ОперацияУмножения множитель}.
ОперацияУмножения = "*" | "/" | DIV | MOD | "&".
множитель = число | строка | NIL | TRUE | FALSE |
множество | обозначение [ФактическиеПараметры] |
"(" выражение ")" | "¬" множитель.
обозначение = уточнИдент {селектор}.
селектор = "." идент | "[" СписокВыражений "]" |
"↑" | "(" уточнИдент ")".
множество = "{" [элемент {"," элемент}] "}".
элемент = выражение [".." выражение].
СписокВыражений = выражение {"," выражение}.
ФактическиеПараметры = "(" [СписокВыражений ")".
оператор = [присваивание | ВызовПроцедуры |
ОператорЕсли | ОператорВыбора |
ОператорПока | ОператорПовтора | ОператорFor].

присваивание = обозначение ":" выражение
 ВызовПроцедуры = обозначение [ФактическиеПараметры].
 ПослОператоров = оператор {";" оператор}.
 ОператорЕсли = IF выражение THEN ПослОператоров
 {ELSIF выражение THEN ПослОператоров}
 [ELSE ПослОператоров] END.
 ОператорВыбора = CASE выражение OF вариант
 {" | " вариант} END.
 вариант = [СписокМетокВарианта ":" ПослОператоров].
 СписокМетокВарианта = МеткиВарианта {";" МеткиВарианта }.
 МеткиВарианта = метка [".." метка].
 метка = целое | строка | уточниИдент.
 ОператорПока = WHILE выражение DO ПослОператоров
 {ELSIF выражение DO ПослОператоров} END.
 ОператорПовтора = REPEAT ПослОператоров UNTIL выражение.
 ОператорFor = FOR идент ":" выражение TO выражение
 [BY КонстантноеВыражение] DO ПослОператоров END.
 ОбъявлениеПроцедуры = ЗаголовокПроцедуры ";"
 ТелоПроцедуры идент.
 ЗаголовокПроцедуры = PROCEDURE идентОпр
 [ФормальныеПараметры].
 ТелоПроцедуры = ПослОбъявлений [BEGIN ПослОператоров]
 [RETURN выражение] END.
 ПослОбъявлений = [CONST {ОбъявлениеКонстант ";"}]
 [TYPE {ОбъявлениеТипов ";"}]
 [VAR {ОбъявлениеПеременных ";"}]
 {ОбъявлениеПроцедуры ";"}].
 ФормальныеПараметры = "(" [СекцияФП {";" СекцияФП }] ")"
 [";" уточниИдент].
 СекцияФП = [VAR] идент {";" идент} ":" ФормальныйТип.
 ФормальныйТип = {ARRAY OF} уточниИдент.
 модуль = MODULE идент ";" [СписокИмпорта] ПослОбъявлений
 [BEGIN ПослОператоров] END идент ".".
 СписокИмпорта = IMPORT импорт {";" импорт} ";".
 импорт = {идент ":"} идент.

ПРИЛОЖЕНИЕ Б. ИСХОДНЫЙ КОД КОМПИЛЯТОРА

Листинг 49. Полный исходный текст модуля Мос (главный модуль)

```
1 MODULE Мос;
2 IMPORT Out, Args, Strings, Machine, Parser,
3         Scanner, Traverser, Table, Tree, Linker;
4 CONST
5     version = '1.0.0-alpha.1';
6
7 PROCEDURE Welcome;
8 BEGIN
9     Out.String('Mac Oberon Compiler version ');
10    Out.String(version); Out.Ln;
11    Out.String('Copyright (c) 2025');
12    Out.String(' by Arthur Yefimov.');
```

```
13 END Welcome;
14
15 PROCEDURE Usage;
16 VAR s: ARRAY 256 OF CHAR;
17 BEGIN
18     Out.String('Usage:'); Out.Ln; Args.Get(0, s);
19     Out.String(' '); Out.String(s);
20     Out.String(' {parameter} sourceFile'); Out.Ln; Out.Ln;
21     Out.String('Parameters:'); Out.Ln;
22     Out.String(' -o outputFile    Specify an executable file name'); Out.Ln;
23     Out.String(' --debug        Enable debug output'); Out.Ln; Out.Ln
24 END Usage;
25
26 (** Finds period and truncates the string there. Puts the result in exe.
27     'Module.Mod' -> 'Module'.
28     If period is not found, the result is an empty string. *)
29 PROCEDURE SourceToExeFname(fname: ARRAY OF CHAR;
30                             VAR exe: ARRAY OF CHAR);
31 VAR i: INTEGER;
32 BEGIN
33     i := 0;
34     WHILE (fname[i] # 0X) & (fname[i] # '.') DO
35         exe[i] := fname[i];
36         INC(i)
37     END;
38     IF fname[i] = 0X THEN i := 0 END;
39     exe[i] := 0X
40 END SourceToExeFname;
41
42 PROCEDURE ParseArgs(VAR mainFname, exeFname: ARRAY OF CHAR);
43 VAR i, count: INTEGER;
```

```

44  s: ARRAY 256 OF CHAR;
45  ok: BOOLEAN;
46  BEGIN
47  ok := TRUE;
48  mainFname := "";
49  exeFname := "";
50  i := 1;
51  count := Args.Count();
52  WHILE i <= count DO
53  Args.Get(i, s);
54  IF s = '-o' THEN
55  Args.Get(i + 1, exeFname);
56  INC(i)
57  ELSIF s = '--debug' THEN
58  Machine.SetDebug(TRUE)
59  ELSIF mainFname = "" THEN
60  Strings.Copy(s, mainFname)
61  ELSE
62  ok := FALSE
63  END;
64  INC(i)
65  END;
66  IF ¬ok THEN
67  Out.String('Error while parsing command line arguments. ');
68  Usage
69  ELSIF exeFname = "" THEN
70  SourceToExeFname(mainFname, exeFname)
71  END
72  END ParseArgs;
73
74  PROCEDURE Run(mainFname, exeFname: ARRAY OF CHAR);
75  VAR module: Tree.Node;
76  BEGIN
77  IF ¬Machine.OpenSourceFile(mainFname) THEN
78  Out.String('Could not open source file ');
79  Out.String(mainFname); Out.String("'.'); Out.Ln
80  ELSE
81  module := Parser.Parse();
82  IF ¬Machine.hadErrors THEN
83  Machine.BackendPhase;
84  IF Machine.debugFrontend THEN
85  Out.String('--- Abstract Syntax Tree ---'); Out.Ln;
86  Tree.Print(module, 0);
87  Out.String('--- Symbol Table ---'); Out.Ln;
88  Table.Print(Table.topScope, 0)
89  END;
90
91  Traverser.SetAddressesAndSizes(Table.topScope);
92  IF ¬Machine.hadErrors THEN
93  Traverser.Traverse(module);
94  IF ¬Machine.hadErrors THEN
95  Linker.Link(exeFname)

```

```

96         END
97     END
98 END
99 END
100 END Run;
101
102 PROCEDURE PraseAndRun;
103 VAR
104     mainFname, exeFname: ARRAY 256 OF CHAR;
105 BEGIN
106     ParseArgs(mainFname, exeFname);
107     IF mainFname # '' THEN
108         Run(mainFname, exeFname)
109     ELSE
110         Out.String('No file specified. '); Out.Ln
111     END
112 END PraseAndRun;
113
114 PROCEDURE Do*;
115 BEGIN
116     IF Args.Count() = 0 THEN
117         Welcome;
118         Usage
119     ELSE
120         PraseAndRun
121     END
122 END Do;
123
124 BEGIN
125     Do
126 END Moc.

```

Листинг 50. Полный исходный текст модуля Machine (загрузка текста)

```

1  MODULE Machine;
2  IMPORT Out, Errors := MocErrors, Files, Utf8, Platform, SYSTEM;
3
4  CONST
5      maxInt* = 2147483647; (** Maximum integer value, 32-bit *)
6      maxExp* = 38; (** Maximum exponent of a real value *)
7
8      pointerSize* = 8; (** In bytes *)
9
10     debugFrontend* = TRUE;
11
12     compilerName* = 'moc';
13
14     (** compilationPhase values **)
15     frontend* = 1;

```

```

16  backend* = 2;
17
18  TYPE
19    SHORTCHAR = SYSTEM.CHAR8;
20
21    Pos* = RECORD
22      line*, col*: INTEGER
23    END;
24
25  VAR
26    fname: ARRAY 256 OF CHAR;
27    r: Files.Rider;
28    prevChar: CHAR;
29    pos*: Pos; (** Current position in the source file being read *)
30    errorPos*: Pos; (** Position of the last error. If no errors, line = -1 *)
31    hadErrors*: BOOLEAN; (** TRUE after a call to Error *)
32
33    debug: BOOLEAN;
34    compilationPhase: INTEGER; (** For proper error output, see constants *)
35
36  (** Begins output of an error if its position is different from the last one.
37    Returns true on success and false if the position is the same. *)
38  PROCEDURE BeginError(): BOOLEAN;
39  VAR ok: BOOLEAN;
40  BEGIN
41    hadErrors := TRUE;
42    ok := TRUE;
43    IF compilationPhase = backend THEN
44      Out.String(compilerName);
45      Out.String(': error: ');
46    ELSIF (pos.line # errorPos.line) OR (pos.col # errorPos.col) THEN
47      Out.String(fname); Out.Char(':');
48      Out.Int(pos.line, 0); Out.Char(':');
49      Out.Int(pos.col, 0); Out.String(': error: ');
50      errorPos := pos
51    ELSE
52      ok := FALSE
53    END
54  RETURN ok END BeginError;
55
56  PROCEDURE Error*(errno: INTEGER);
57  VAR s: ARRAY 256 OF CHAR;
58  BEGIN
59    IF BeginError() THEN
60      Errors.Message(errno, s);
61      Out.String(s); Out.Ln
62    END
63  END Error;
64
65  PROCEDURE ErrorExpected*(symExpected, symFound: INTEGER);
66  BEGIN
67    IF BeginError() THEN

```

```

68     Errors.PrintSymbol(symExpected);
69     Out.String(' expected, but ');
70     Errors.PrintSymbol(symFound);
71     Out.String(' found');
72     Out.Ln
73 END
74 END ErrorExpected;
75
76 PROCEDURE ErrorExpectedMsg*(errno, symFound: INTEGER);
77 VAR s: ARRAY 256 OF CHAR;
78 BEGIN
79     IF BeginError() THEN
80         Errors.Message(errno, s);
81         Out.String(s);
82         Out.String(', but ');
83         Errors.PrintSymbol(symFound);
84         Out.String(' found');
85         Out.Ln
86     END
87 END ErrorExpectedMsg;
88
89 PROCEDURE NotImplemented*(feature: ARRAY OF CHAR);
90 VAR s: ARRAY 256 OF CHAR;
91 BEGIN
92     IF BeginError() THEN
93         Errors.Message(Errors.notImplemented, s);
94         Out.String(s); Out.String(': ');
95         Out.String(feature); Out.Ln
96     END
97 END NotImplemented;
98
99 (** Returns next character from the file opened. On end of file return 0X.
100    Keeps track of the current position in the file in terms of line:column. *)
101 PROCEDURE Read*(VAR ch: CHAR);
102 BEGIN
103     IF r.eof THEN
104         ch := 0X
105     ELSE
106         IF prevChar = 0AX THEN
107             INC(pos.line);
108             pos.col := 1
109         ELSE
110             INC(pos.col)
111         END;
112         Files.ReadChar(r, prevChar);
113         ch := prevChar
114     END
115 END Read;
116
117 PROCEDURE OpenSourceFile*(filename: ARRAY OF CHAR): BOOLEAN;
118 VAR F: Files.File;
119 BEGIN

```

```

120   F := Files.Old(filename);
121   IF F = NIL THEN
122     fname := ""
123   ELSE
124     fname := filename;
125     Files.Set(r, F, 0);
126     prevChar := 0X
127   END;
128   pos.line := 1;
129   pos.col := 1;
130   errorPos.line := -1;
131   hadErrors := F = NIL;
132   compilationPhase := frontend
133 RETURN ~hadErrors END OpenSourceFile;
134
135 PROCEDURE Exec*(command: ARRAY OF CHAR): INTEGER;
136 VAR z: ARRAY 4096 OF SHORTCHAR;
137 BEGIN
138   Utf8.Encode(command, z);
139 RETURN Platform.System(z) END Exec;
140
141 PROCEDURE BackendPhase*;
142 BEGIN
143   compilationPhase := backend
144 END BackendPhase;
145
146 PROCEDURE SetDebug*(value: BOOLEAN);
147 BEGIN
148   debug := value
149 END SetDebug;
150
151 BEGIN
152   debug := FALSE
153 END Machine.

```

Листинг 51. Полный исходный текст модуля Scanner (лексический анализатор)

```

1  MODULE Scanner;
2  IMPORT Out, Machine, Errors := MocErrors;
3
4  CONST
5    identSize = 256;
6    stringSize = 1024;
7
8    uptoChar = 7FX; (** Used as a fix for double dot right after a digit *)
9
10   (** Lexical symbols **)
11   null      = 0;    times*    = 1;    rdiv*     = 2;    div*      = 3;
12   mod*      = 4;    and*      = 5;    plus*     = 6;    minus*    = 7;
13   or*       = 8;    eql*      = 9;    neq*      = 10;   lss*      = 11;
14   leq*      = 12;   gtr*      = 13;   geq*      = 14;   in*       = 15;
15   is*       = 16;   arrow*    = 17;   period*   = 18;   char*     = 20;
16   int*      = 21;   real*     = 22;   false*    = 23;   true*     = 24;
17   nil*      = 25;   string*   = 26;   not*      = 27;   lparen*   = 28;
18   lbrak*    = 29;   lbrace*   = 30;   ident*    = 31;   if*       = 32;
19   while*    = 34;   repeat*   = 35;   case*     = 36;   for*      = 37;
20   comma*    = 40;   colon*    = 41;   becomes*  = 42;   upto*     = 43;
21   rparen*   = 44;   rbrak*    = 45;   rbrace*   = 46;   then*     = 47;
22   of*       = 48;   do*       = 49;   to*       = 50;   by*       = 51;
23   semicolon* = 52;  end*      = 53;   bar*      = 54;   else*     = 55;
24   elsif*    = 56;   until*    = 57;   return*   = 58;   array*    = 60;
25   record*   = 61;   pointer*  = 62;   const*    = 63;   type*     = 64;
26   var*      = 65;   procedure* = 66;  begin*    = 67;   import*   = 68;
27   module*   = 69;   eot*      = 70;
28
29  TYPE
30    Ident* = ARRAY identSize OF CHAR;
31    StrVal* = ARRAY stringSize OF CHAR;
32
33  VAR
34    (** Results of Get **)
35    name*: Ident;
36    intVal*: INTEGER;
37    realVal*: REAL;
38    strVal*: StrVal;
39    strLen*: INTEGER;
40
41    (** _ **)
42    ch: CHAR; (** Read-ahead character *)
43
44    (** Converts an uppercase hexadecimal digit to an integer. *)
45  PROCEDURE HexToInt(c: CHAR): INTEGER;
46  VAR n: INTEGER;
47  BEGIN

```

```

48 IF ('0' <= c) & (c <= '9') THEN
49   n := ORD(c) - ORD('0')
50 ELSEIF ('A' <= c) & (c <= 'F') THEN
51   n := ORD(c) - (ORD('A') + 10)
52 ELSE
53   ASSERT(FALSE)
54 END
55 RETURN n END HexToInt;
56
57 (** Reports whether ch is a decimal digit. *)
58 PROCEDURE IsDigit(): BOOLEAN;
59 RETURN ('0' <= ch) & (ch <= '9')
60 END IsDigit;
61
62 (** Reports whether ch is an uppercase hexadecimal digit (0..F). *)
63 PROCEDURE IsHex(): BOOLEAN;
64 RETURN IsDigit() OR ('A' <= ch) & (ch <= 'F')
65 END IsHex;
66
67 (** Reports whether ch is a letter. *)
68 PROCEDURE IsAlpha(): BOOLEAN;
69 RETURN ('A' <= ch) & (ch <= 'Z') OR
70       ('a' <= ch) & (ch <= 'z') OR
71       (ch = ' ')
72 END IsAlpha;
73
74 (** Reports whether ch is a letter or a number. *)
75 PROCEDURE IsAlphaNum(): BOOLEAN;
76 RETURN IsAlpha() OR IsDigit()
77 END IsAlphaNum;
78
79 (** Examines name and returns one of keyword lexical symbol constants,
80     or ident if name is not a keyword. Uses the passed in actual length
81     of name for speed. *)
82 PROCEDURE IdentifyKeyword(len: INTEGER): INTEGER;
83 VAR sym: INTEGER;
84 BEGIN
85   sym := ident;
86   IF len = 2 THEN
87     IF name = "OR" THEN sym := or
88     ELSEIF name = "IN" THEN sym := in
89     ELSEIF name = "IS" THEN sym := is
90     ELSEIF name = "IF" THEN sym := if
91     ELSEIF name = "OF" THEN sym := of
92     ELSEIF name = "DO" THEN sym := do
93     ELSEIF name = "TO" THEN sym := to
94     ELSEIF name = "BY" THEN sym := by
95     END
96   ELSEIF len = 3 THEN
97     IF name = "END" THEN sym := end
98     ELSEIF name = "VAR" THEN sym := var
99     ELSEIF name = "NIL" THEN sym := nil

```

```

100     ELSIF name = "DIV" THEN sym := div
101     ELSIF name = "MOD" THEN sym := mod
102     ELSIF name = "FOR" THEN sym := for
103     END
104     ELSIF len = 4 THEN
105         IF name = "THEN" THEN sym := then
106         ELSIF name = "ELSE" THEN sym := else
107         ELSIF name = "TRUE" THEN sym := true
108         ELSIF name = "TYPE" THEN sym := type
109         ELSIF name = "REAL" THEN sym := real
110         ELSIF name = "CASE" THEN sym := case
111         END
112     ELSIF len = 5 THEN
113         IF name = "ELSIF" THEN sym := elsif
114         ELSIF name = "FALSE" THEN sym := false
115         ELSIF name = "CONST" THEN sym := const
116         ELSIF name = "WHILE" THEN sym := while
117         ELSIF name = "ARRAY" THEN sym := array
118         ELSIF name = "BEGIN" THEN sym := begin
119         ELSIF name = "UNTIL" THEN sym := until
120         END
121     ELSIF len = 6 THEN
122         IF name = "RETURN" THEN sym := return
123         ELSIF name = "RECORD" THEN sym := record
124         ELSIF name = "REPEAT" THEN sym := repeat
125         ELSIF name = "IMPORT" THEN sym := import
126         ELSIF name = "MODULE" THEN sym := module
127         END
128     ELSIF (len = 7) & (name = "POINTER") THEN sym := pointer
129     ELSIF (len = 9) & (name = "PROCEDURE") THEN sym := procedure
130     END
131     RETURN sym END IdentifyKeyword;
132
133 (** Skips a possibly nested comment. Precondition: ch = '*', *)
134 PROCEDURE SkipComment;
135 VAR prevCh: CHAR;
136 BEGIN
137     prevCh := 0X;
138     Machine.Read(ch);
139     WHILE (ch # 0X) & ¬((prevCh = '*') & (ch = ')) DO
140         prevCh := ch;
141         Machine.Read(ch);
142         IF (prevCh = '(') & (ch = '*') THEN (* Nested comment *)
143             SkipComment
144         END
145     END;
146     IF ch # 0X THEN Machine.Read(ch) END
147 END SkipComment;
148
149 PROCEDURE ReadNumber(VAR sym: INTEGER);
150 VAR i, len: INTEGER;
151 ok: BOOLEAN;

```

```

152  err: INTEGER;
153  digits: ARRAY 16 OF INTEGER;
154
155  PROCEDURE DecimalToInt(digits: ARRAY OF INTEGER; len: INTEGER);
156  VAR i: INTEGER;
157      err: INTEGER;
158  BEGIN
159      err := Errors.none;
160      intVal := 0;
161      i := 0;
162      REPEAT
163          IF digits[i] >= 10 THEN
164              err := Errors.badInt
165          ELSE
166              (* Overflow check *)
167              IF intVal > (Machine.maxInt - digits[i]) DIV 10 THEN
168                  err := Errors.intOverflow
169              ELSE
170                  intVal := intVal * 10 + digits[i]
171              END
172          END;
173          INC(i)
174      UNTIL i = len;
175      IF err # Errors.none THEN
176          intVal := 0;
177          Machine.Error(err)
178      END
179  END DecimalToInt;
180
181  (** Returns 10 in the power of e. *)
182  PROCEDURE Ten(e: INTEGER): REAL;
183  VAR x, t: REAL;
184  BEGIN
185      x := 1.0;
186      t := 10.0;
187      WHILE e > 0 DO
188          IF ODD(e) THEN x := t * x END;
189          t := t * t;
190          e := e DIV 2
191      END
192  RETURN x END Ten;
193
194  PROCEDURE ReadReal(digits: ARRAY OF INTEGER; len: INTEGER);
195  VAR i, exponent, scale: INTEGER;
196      negE: BOOLEAN;
197      ok: BOOLEAN;
198  BEGIN
199      ok := TRUE;
200      realVal := 0.0;
201      exponent := 0;
202      (* Integer part *)
203      i := 0;

```

```

204 REPEAT
205     IF digits[i] >= 10 THEN
206         ok := FALSE
207     ELSE
208         realVal := realVal * 10.0 + FLT(digits[i])
209     END;
210     INC(i)
211 UNTIL i = len;
212 IF ¬ok THEN
213     realVal := 0.0;
214     Machine.Error(Errors.badReal)
215 END;
216 (* Fraction part *)
217 WHILE IsDigit() DO
218     realVal := realVal * 10.0 + FLT(ORD(ch) - ORD('0'));
219     DEC(exponent);
220     Machine.Read(ch)
221 END;
222 (* Scale factor *)
223 IF (ch = 'E') OR (ch = 'D') THEN
224     Machine.Read(ch);
225     scale := 0;
226     IF ch = '-' THEN
227         Machine.Read(ch);
228         negE := TRUE
229     ELSE
230         negE := FALSE;
231         IF ch = '+' THEN Machine.Read(ch) END
232     END;
233     IF ¬IsDigit() THEN
234         Machine.Error(Errors.noDigit)
235     ELSE
236         REPEAT
237             scale := scale * 10 + ORD(ch) - ORD('0');
238             Machine.Read(ch)
239         UNTIL ¬IsDigit();
240         IF negE THEN DEC(exponent, scale) ELSE INC(exponent, scale) END
241     END
242 END;
243 IF exponent < 0 THEN
244     IF exponent >= -Machine.maxExp THEN
245         realVal := realVal / Ten(-exponent)
246     ELSE
247         realVal := 0.0
248     END
249 ELSIF exponent > 0 THEN
250     IF exponent <= Machine.maxExp THEN
251         realVal := Ten(exponent) * realVal
252     ELSE
253         Machine.Error(Errors.largeExponent);
254         realVal := 0.0
255     END

```

```

256     END
257 END ReadReal;
258
259 BEGIN
260     ok := TRUE;
261     len := 0;
262     REPEAT
263         IF len # LEN(digits) THEN
264             digits[len] := HexToInt(ch);
265             INC(len)
266         ELSE
267             ok := FALSE
268         END;
269         Machine.Read(ch)
270     UNTIL ¬IsHex();
271     IF ¬ok THEN
272         Machine.Error(Errors.manyDigits)
273     END;
274     IF (ch = 'H') OR (ch = 'X') THEN (* Hexadecimal integer or char *)
275         ok := TRUE;
276         intVal := 0;
277         i := 0;
278         REPEAT (* TODO Allow negative two's complement HEX constants *)
279             (* Overflow check *)
280             IF intVal > (Machine.maxInt - digits[i]) DIV 16 THEN
281                 ok := FALSE
282             ELSE
283                 intVal := intVal * 16 + digits[i]
284             END;
285             INC(i)
286         UNTIL i = len;
287         IF ¬ok THEN
288             intVal := 0;
289             Machine.Error(Errors.intOverflow)
290         END;
291         IF ch = 'X' THEN sym := char ELSE sym := int END;
292         Machine.Read(ch)
293     ELSIF ch = '.' THEN (* Real number or integer + upto *)
294         Machine.Read(ch);
295         IF ch = '.' THEN (* Upto (..) right after a digit *)
296             ch := uptoChar; (* Fix for Get to read upto symbol instead of period *)
297             DecimalToInt(digits, len);
298             sym := int
299         ELSE
300             ReadReal(digits, len);
301             sym := real
302         END
303     ELSE (* Decimal integer *)
304         DecimalToInt(digits, len);
305         sym := int
306     END
307 END ReadNumber;

```

```

308
309 PROCEDURE Get*(VAR sym: INTEGER);
310 VAR i: INTEGER;
311     closeCh: CHAR;
312     ok, comment: BOOLEAN;
313 BEGIN
314     REPEAT
315         sym := null;
316         comment := FALSE;
317         WHILE (ch # 0X) & (ch <= ' ') DO Machine.Read(ch) END;
318         IF ch = 0X THEN
319             sym := eof
320         ELSIF IsAlpha() THEN (* Also ' _ ' *)
321             name[0] := ch;
322             Machine.Read(ch);
323             ok := TRUE;
324             i := 1;
325             WHILE IsAlphaNum() DO
326                 IF i = LEN(name) - 1 THEN
327                     ok := FALSE
328                 ELSE
329                     name[i] := ch;
330                     INC(i)
331                 END;
332                 Machine.Read(ch)
333             END;
334             name[i] := 0X;
335             IF ¬ok THEN
336                 Machine.Error(Errors.longIdent);
337                 name := ""
338             END;
339             sym := IdentifyKeyword(i)
340         ELSIF IsDigit() THEN
341             ReadNumber(sym)
342         ELSIF (ch = '"') OR (ch = "'") THEN
343             closeCh := ch;
344             Machine.Read(ch);
345             ok := TRUE;
346             strLen := 0;
347             WHILE (ch >= ' ') & (ch # closeCh) DO
348                 IF strLen = LEN(strVal) - 1 THEN
349                     ok := FALSE
350                 ELSE
351                     strVal[strLen] := ch;
352                     INC(strLen)
353                 END;
354                 Machine.Read(ch)
355             END;
356             strVal[strLen] := 0X;
357             IF ch < ' ' THEN
358                 Machine.Error(Errors.unclosedStr);
359                 strVal := "";

```

```

360     strLen := 0
361 ELSE
362     Machine.Read(ch);
363     IF ¬ok THEN
364         Machine.Error(Errors.longStr);
365         strVal := "";
366         strLen := 0
367     END
368 END;
369 sym := string
370 ELSIF ch < '0' THEN (* '!'..'/' *)
371     IF ch = '#' THEN Machine.Read(ch); sym := neg
372 ELSIF ch = '&' THEN Machine.Read(ch); sym := and
373 ELSIF ch = '(' THEN
374     Machine.Read(ch);
375     IF ch = '*' THEN
376         SkipComment;
377         comment := TRUE
378     ELSE
379         sym := lparen
380     END
381 ELSIF ch = ')' THEN Machine.Read(ch); sym := rparen
382 ELSIF ch = '*' THEN Machine.Read(ch); sym := times
383 ELSIF ch = '+' THEN Machine.Read(ch); sym := plus
384 ELSIF ch = ',' THEN Machine.Read(ch); sym := comma
385 ELSIF ch = '-' THEN Machine.Read(ch); sym := minus
386 ELSIF ch = '.' THEN
387     Machine.Read(ch);
388     IF ch = '.' THEN Machine.Read(ch); sym := upto ELSE sym := period END
389 ELSIF ch = '/' THEN Machine.Read(ch); sym := rdiv
390 END (* ELSE ! $ % *)
391 ELSIF ch < 'A' THEN (* ':'..'@' *)
392     IF ch = ':' THEN
393         Machine.Read(ch);
394         IF ch = '=' THEN Machine.Read(ch); sym := becomes ELSE sym := colon END
395     ELSIF ch = ';' THEN Machine.Read(ch); sym := semicolon
396     ELSIF ch = '<' THEN
397         Machine.Read(ch);
398         IF ch = '=' THEN Machine.Read(ch); sym := leq ELSE sym := lss END
399     ELSIF ch = '=' THEN Machine.Read(ch); sym := eql
400     ELSIF ch = '>' THEN
401         Machine.Read(ch);
402         IF ch = '=' THEN Machine.Read(ch); sym := geq ELSE sym := gtr END
403     END (* ELSE ? @ *)
404 ELSIF ch < 'a' THEN (* '['..'~' *)
405     IF ch = '[' THEN Machine.Read(ch); sym := lbrak
406     ELSIF ch = ']' THEN Machine.Read(ch); sym := rbrak
407     ELSIF ch = '↑' THEN Machine.Read(ch); sym := arrow
408     END (* ELSE \ ` *)
409 ELSIF ch = '{' THEN Machine.Read(ch); sym := lbrace
410 ELSIF ch = '|' THEN Machine.Read(ch); sym := bar
411 ELSIF ch = '}' THEN Machine.Read(ch); sym := rbrace

```



```

412     ELSIF ch = '↵' THEN Machine.Read(ch); sym := not
413     ELSIF ch = uptoChar THEN Machine.Read(ch); sym := upto
414     END;
415     IF ¬comment & (sym = null) THEN
416         Machine.Read(ch);
417         Machine.Error(Errors.badChar)
418     END
419     UNTIL sym # null (* Comment *)
420 END Get;
421
422 PROCEDURE Init*;
423 BEGIN
424     Machine.Read(ch)
425 END Init;
426
427 END Scanner.

```

Листинг 52. Полный исходный текст модуля Parser (синтаксический анализатор)

```

1  MODULE Parser;
2  IMPORT Out, Strings, Table, Tree, Machine, S := Scanner, Builder,
3  Errors := MocErrors, Kernel;
4
5  TYPE
6  Ident = S.Ident;
7
8  VAR
9  sym: INTEGER; (** Read-ahead symbol *)
10
11  modName: Ident;
12  dummy: Table.Object;
13
14  PROCEDURE TestScanner;
15  BEGIN
16  WHILE sym # S.eot DO
17  IF sym = S.string THEN
18  Out.String('string ');
19  Out.Int(S.strLen, 0); Out.String(" ");
20  Out.String(S.strVal); Out.Char(" ")
21  ELSIF sym = S.int THEN
22  Out.String('int ');
23  Out.Int(S.intVal, 0)
24  ELSIF sym = S.real THEN
25  Out.String('real ');
26  Out.Real(S.realVal, 0)
27  ELSIF sym = S.ident THEN
28  Out.String('ident ');
29  Out.String(S.name)
30  ELSE

```

```

31     Errors.PrintSymbol(sym)
32     END;
33     S.Get(sym);
34     Out.Ln
35     END;
36     Out.String('eot'); Out.Ln
37 END TestScanner;
38
39 (** Reports whether sym is the expected symbol. If not, reports an error. *)
40 PROCEDURE Check(expectedSym: INTEGER): BOOLEAN;
41 BEGIN
42     IF sym # expectedSym THEN
43         Machine.ErrorExpected(expectedSym, sym)
44     END
45 RETURN sym = expectedSym END Check;
46
47 (** Checks that sym is the expected symbol and skips it, or reports an error. *)
48 PROCEDURE Skip(expectedSym: INTEGER);
49 BEGIN
50     IF sym = expectedSym THEN
51         S.Get(sym)
52     ELSE
53         Machine.ErrorExpected(expectedSym, sym)
54     END
55 END Skip;
56
57 (** If sym is the expected symbol then skips it and returns true,
58     otherwise returns false. *)
59 PROCEDURE Skipped(expectedSym: INTEGER): BOOLEAN;
60 VAR ok: BOOLEAN;
61 BEGIN
62     ok := sym = expectedSym;
63     IF ok THEN S.Get(sym) END
64 RETURN ok END Skipped;
65
66 PROCEDURE StandardProcedureCall(x: Tree.Node): Tree.Node;
67 BEGIN
68     Machine.NotImplemented('standard procedure call')
69 RETURN x END StandardProcedureCall;
70
71 (** qualident = [ident "."] ident. *)
72 PROCEDURE Qualident(): Table.Object;
73 VAR object: Table.Object;
74 BEGIN
75     object := Table.ThisObject(S.name);
76     S.Get(sym); (* ident *)
77     IF object = NIL THEN
78         Machine.Error(Errors.undefinedIdent);
79         object := dummy
80     END;
81
82     IF (object.class = Table.Mod) & Skipped(S.period) THEN

```

```

83   IF  $\neg$ Check(S.ident) THEN
84       object := dummy
85   ELSE
86       object := Table.ThisImport(object, S.name);
87       S.Get(sym); (* ident *)
88       IF object = NIL THEN
89           Machine.Error(Errors.undefIdent);
90           object := dummy
91       END
92   END
93 END
94 RETURN object END Qualident;
95
96 PROCEDURE  $\uparrow$  Expression(): Tree.Node;
97
98 (** designator = qualident {selector}.
99     selector = "." ident | "[" ExpList "]" | " $\uparrow$ " | "(" qualident ")" *. *)
100 PROCEDURE Designator(): Tree.Node;
101 VAR designator: Tree.Node;
102     qualident: Table.Object;
103 BEGIN
104     qualident := Qualident();
105     designator := Builder.NewLeaf(qualident);
106     IF designator.class # Table.SProc THEN
107         (* Selectors *)
108         WHILE sym = S.lbrak DO (* Array index *)
109             Machine.NotImplemented('array index');
110             REPEAT S.Get(sym) UNTIL (sym = S.eot) OR (sym = S.rbrak);
111             S.Get(sym) (* rbrak, TODO *)
112         ELSIF sym = S.period DO (* Record field *)
113             S.Get(sym); (*TODO*)
114             Machine.NotImplemented('record field')
115         ELSIF sym = S.arrow DO (* Pointer dereference *)
116             S.Get(sym); (*TODO*)
117             Machine.NotImplemented('pointer dereference')
118         ELSIF (sym = S.lparen) & (* Type guard *)
119             (designator.type.form IN {Table.Record, Table.Pointer})
120         DO
121             Machine.NotImplemented('type guard');
122             REPEAT S.Get(sym) UNTIL (sym = S.eot) OR (sym = S.rparen);
123             Skip(S.rparen) (*TODO*)
124         END
125     END
126 RETURN designator END Designator;
127
128 (** ActualParameters = "(" [ExpList] ")".
129     ExpList = expression {"", " expression}.
130     Precondition: sym = S.lparen *)
131 PROCEDURE ActualParameters(procedure: Tree.Node;
132     formal: Table.Object): Tree.Node;
133 VAR actual, list, last: Tree.Node;
134     n, paramCount: INTEGER;

```

```

135 BEGIN
136   Skip(S.lparen);
137   n := 0;
138   paramCount := procedure.object.type.paramCount;
139   IF sym # S.rparen THEN
140     actual := Expression();
141     IF formal = NIL THEN
142       Machine.Error(Errors.manyArguments);
143     ELSE
144       Builder.Param(actual, formal);
145       list := actual;
146       last := list;
147       formal := formal.next;
148       INC(n);
149       WHILE Skipped(S.comma) DO
150         actual := Expression();
151         IF formal # NIL THEN
152           Builder.Param(actual, formal);
153           last.link := actual;
154           formal := formal.next;
155         END;
156         INC(n)
157       END
158     END
159   END;
160   IF n < paramCount THEN
161     Machine.Error(Errors.fewArguments)
162   ELSIF n > paramCount THEN
163     Machine.Error(Errors.manyArguments)
164   END;
165   Skip(S.rparen)
166 RETURN list END ActualParameters;
167
168 (** ProcedureCall = designator [ActualParameters].
169     Precondition: designator already read. *)
170 PROCEDURE ProcedureCall(designator: Tree.Node): Tree.Node;
171 VAR formalParams: Table.Object;
172     actualParams, x: Tree.Node;
173 BEGIN
174   IF (designator.class # Tree.NProc) OR
175      (designator.object.type.form # Table.Proc) THEN
176     Machine.Error(Errors.notProcedure)
177   ELSIF designator.object.constVal.intVal > 0 THEN (* Standard procedure *)
178     IF designator.object.class = Table.SFunc THEN
179       Machine.Error(Errors.procCallFunc)
180     ELSE (* class = SProc *)
181       x := StandardProcedureCall(designator)
182     END
183   ELSE (* Non-standard procedure *)
184     formalParams := Builder.PrepareCall(designator);
185     IF sym = S.lparen THEN
186       actualParams := ActualParameters(designator, formalParams)

```

```

187     END;
188     IF designator.type.base.form # Table.NoType THEN
189         Machine.Error(Errors.procCallFunc)
190     END;
191     x := Builder.Call(designator, actualParams, formalParams)
192 END
193 RETURN x END ProcedureCall;
194
195 (** factor = number | string | NIL | TRUE | FALSE |
196    set | designator [ActualParameters] | "(" expression ")" | "¬" factor. *)
197 PROCEDURE Factor(): Tree.Node;
198 VAR x, y: Tree.Node;
199     op: INTEGER;
200     qualident, formalParams: Table.Object;
201     actualParams: Tree.Node;
202 BEGIN
203     (* Sync *)
204     IF (sym < S.char) OR (sym > S.ident) THEN
205         Machine.ErrorExpectedMsg(Errors.noExpression, sym);
206         REPEAT S.Get(sym) UNTIL (sym >= S.char) & (sym <= S.for) OR (sym >= S.then)
207     END;
208
209     IF sym = S.ident THEN
210         x := Designator();
211         IF (x.class = Tree.NProc) & (x.object.class = Table.SFunc) THEN
212             x := StandardProcedureCall(x)
213         ELSIF Skipped(S.lparen) & (x.type.form = Table.Proc) &
214             (x.type.base.form # Table.NoType)
215         THEN
216             formalParams := Builder.PrepareCall(x);
217             actualParams := ActualParameters(x, formalParams);
218             Skip(S.rparen);
219             x := Builder.Call(x, actualParams, formalParams)
220         END
221     ELSIF sym = S.int THEN
222         x := Builder.NewIntConst(S.intVal);
223         S.Get(sym)
224     ELSIF sym = S.char THEN
225         x := Builder.NewIntConst(S.intVal);
226         x.type := Table.charType;
227         S.Get(sym)
228     ELSIF sym = S.real THEN
229         x := Builder.NewRealConst(S.realVal);
230         S.Get(sym)
231     ELSIF Skipped(S.not) THEN
232         x := Factor();
233         x := Builder.MonadicOperator(S.not, x)
234     (*TODO true, false, string, nil, etc.*)
235     ELSIF Skipped(S.lparen) THEN (* Subexpression *)
236         x := Expression();
237         Skip(S.rparen)
238     ELSE

```

```

239     Machine.Error(Errors.noFactor);
240     S.Get(sym);
241     x := Builder.NewIntConst(0)
242 END
243 RETURN x END Factor;
244
245 (** term = factor {MulOperator factor}.
246     MulOperator = "*" | "/" | DIV | MOD | "&". *)
247 PROCEDURE Term(): Tree.Node;
248 VAR x, y: Tree.Node;
249     op: INTEGER;
250 BEGIN
251     x := Factor();
252     (* {MulOperator factor} *)
253     WHILE (S.times <= sym) & (sym <= S.and) DO
254         op := sym;
255         S.Get(sym); (* op *)
256         y := Factor();
257         x := Builder.DyadicOperator(op, x, y)
258     END
259 RETURN x END Term;
260
261 (** SimpleExpression = ["+" | "-"] term {AddOperator term}.
262     AddOperator = "+" | "-" | OR. *)
263 PROCEDURE SimpleExpression(): Tree.Node;
264 VAR x, y: Tree.Node;
265     op: INTEGER;
266 BEGIN
267     (* ["+" | "-"] term *)
268     IF sym = S.minus THEN
269         S.Get(sym);
270         x := Term();
271         x := Builder.MonadicOperator(S.minus, x)
272     ELSE
273         IF sym = S.plus THEN S.Get(sym) END;
274         x := Term()
275     END;
276     (* {AddOperator term} *)
277     WHILE (S.plus <= sym) & (sym <= S.or) DO
278         op := sym;
279         S.Get(sym); (* op *)
280         y := Term();
281         x := Builder.DyadicOperator(op, x, y)
282     END
283 RETURN x END SimpleExpression;
284
285 (** expression = SimpleExpression [relation SimpleExpression].
286     relation = "=" | "#" | "<" | "<=" | ">" | ">=" | IN | IS. *)
287 PROCEDURE Expression(): Tree.Node;
288 VAR x, y: Tree.Node;
289     relation: INTEGER;
290 BEGIN

```

```

291  x := SimpleExpression();
292  IF (S.eql <= sym) & (sym <= S.geq) THEN
293      relation := sym;
294      S.Get(sym); (* relation *)
295      y := SimpleExpression();
296      x := Builder.DyadicOperator(relation, x, y)
297  ELSIF sym = S.in THEN
298      Machine.NotImplemented('operator IN')
299  ELSIF sym = S.is THEN
300      Machine.NotImplemented('operator IS')
301  END
302  RETURN x END Expression;
303
304  (** assignment = designator "[:=" expression.
305      Precondition: designator and "[:=" already read. *)
306  PROCEDURE Assignment(designator: Tree.Node): Tree.Node;
307  VAR assign, expr: Tree.Node;
308  BEGIN
309      expr := Expression();
310      assign := Tree.NewNode(Tree.NAssign);
311      assign.subclass := Tree.assign;
312      assign.left := designator;
313      assign.right := expr
314  RETURN assign END Assignment;
315
316  (** IfStatement = IF expression THEN StatementSequence
317      {ELSIF expression THEN StatementSequence}
318      [ELSE StatementSequence] END. *)
319  PROCEDURE IfStatement(): Tree.Node;
320  VAR if: Tree.Node;
321  BEGIN
322
323  RETURN if END IfStatement;
324
325  (** WhileStatement = WHILE expression DO StatementSequence
326      {ELSIF expression DO StatementSequence} END. *)
327  PROCEDURE WhileStatement(): Tree.Node;
328  VAR while: Tree.Node;
329  BEGIN
330
331  RETURN while END WhileStatement;
332
333  (** RepeatStatement = REPEAT StatementSequence UNTIL expression. *)
334  PROCEDURE RepeatStatement(): Tree.Node;
335  VAR repeat: Tree.Node;
336  BEGIN
337
338  RETURN repeat END RepeatStatement;
339
340  (** statement = [assignment | ProcedureCall | IfStatement | CaseStatement |
341      WhileStatement | RepeatStatement | ForStatement]. *)
342  PROCEDURE Statement(): Tree.Node;

```

```

343 VAR statement, designator: Tree.Node;
344 BEGIN
345     IF sym = S.ident THEN (* Assignment or procedure call *)
346         designator := Designator();
347         IF Skipped(S.becomes) THEN
348             statement := Assignment(designator)
349         ELSE
350             statement := ProcedureCall(designator)
351         END
352     ELSIF Skipped(S.if) THEN
353         statement := IfStatement()
354     ELSIF Skipped(S.while) THEN
355         statement := WhileStatement()
356     ELSIF sym = S.case THEN
357         Machine.NotImplemented('case statement')
358     ELSIF sym = S.repeat THEN
359         Machine.NotImplemented('repeat statement')
360     ELSIF sym = S.for THEN
361         Machine.NotImplemented('for statement')
362     END
363 RETURN statement END Statement;
364
365 (** StatementSequence = statement {"," statement}. *)
366 PROCEDURE StatementSequence(): Tree.Node;
367 VAR first, prev, statement: Tree.Node;
368 BEGIN
369     (* Sync *)
370     IF ¬((sym >= S.ident) & (sym <= S.for) OR (sym >= S.semicolon)) THEN
371         Machine.Error(Errors.noStatement);
372         REPEAT S.Get(sym) UNTIL (sym >= S.ident)
373     END;
374
375     WHILE sym = S.semicolon DO S.Get(sym) END;
376     first := Statement();
377     prev := first;
378     WHILE Skipped(S.semicolon) DO
379         WHILE sym = S.semicolon DO S.Get(sym) END;
380         statement := Statement();
381         prev.link := statement;
382         prev := statement
383     END
384 RETURN first END StatementSequence;
385
386 (** ConstExpression = expression. *)
387 PROCEDURE ConstExpression(): Tree.Node;
388 VAR const: Tree.Node;
389 BEGIN
390     const := Expression();
391     IF const.class # Tree.NConst THEN
392         Machine.Error(Errors.notConstant);
393     const := Builder.NewIntConst(0)
394 END

```



```

395 RETURN const END ConstExpression;
396
397 (** ConstDeclaration = identdef "=" ConstExpression.
398     identdef = ident. *)
399 PROCEDURE ConstDeclaration;
400   VAR expr: Tree.Node;
401       obj: Table.Object;
402 BEGIN
403   obj := Table.NewObject(S.name, Table.Const);
404   S.Get(sym); (* ident *)
405   IF Skipped(S.eq1) THEN
406     expr := ConstExpression();
407     IF expr.class # Tree.NConst THEN
408       Machine.Error(Errors.notConstant)
409     ELSE
410       obj.type := expr.type;
411       obj.constVal := expr.constVal
412     END
413   END
414 END ConstDeclaration;
415
416 (** type = qualident | ArrayType | RecordType | PointerType | ProcedureType.
417     ArrayType = ARRAY length {"," length} OF type.
418     length = ConstExpression.
419     RecordType = RECORD ["(" BaseType ")"] [FieldListSequence] END.
420     BaseType = qualident. *)
421 PROCEDURE Type(): Table.Type;
422   VAR obj: Table.Object;
423       type: Table.Type;
424 BEGIN
425   IF sym = S.ident THEN
426     obj := Qualident();
427     IF obj.class # Table.Type THEN
428       Machine.Error(Errors.noType)
429     ELSIF (obj.type = NIL) OR (obj.type.form = Table.NoType) THEN
430       Machine.Error(Errors.undefinedType)
431     ELSE
432       type := obj.type
433     END
434   ELSIF sym = S.array THEN
435     Machine.NotImplemented('type array')
436   ELSIF sym = S.record THEN
437     Machine.NotImplemented('type record')
438   ELSIF sym = S.pointer THEN
439     Machine.NotImplemented('type pointer')
440   ELSIF sym = S.procedure THEN
441     Machine.NotImplemented('procedure type')
442   ELSE
443     Machine.Error(Errors.noType)
444   END
445 RETURN type END Type;
446

```

```

447 (** TypeDeclaration = identdef "=" type. *)
448 PROCEDURE TypeDeclaration;
449 BEGIN
450   Machine.NotImplemented('type declaration');
451   (* Sync to VAR, BEGIN, RETURN or END *)
452   REPEAT
453     S.Get(sym)
454   UNTIL (sym = S.var) OR (sym = S.begin) OR (sym = S.return) OR (sym = S.end)
455 END TypeDeclaration;
456
457 (** Reads a comma-separated identifier list, adds an object of the given
458     class for each identifier and returns the first created object.
459     IdentList = identdef {"," identdef}.
460     identdef = ident. *)
461 PROCEDURE IdentList(class: INTEGER): Table.Object;
462 VAR first, obj: Table.Object;
463 BEGIN
464   first := Table.NewObject(S.name, class);
465   S.Get(sym); (* ident *)
466   WHILE sym = S.comma DO
467     S.Get(sym); (* comma *)
468     obj := Table.NewObject(S.name, class);
469     S.Get(sym) (* ident *)
470   END
471 RETURN first END IdentList;
472
473 (** VariableDeclaration = IdentList ":" type. *)
474 PROCEDURE VariableDeclaration;
475 VAR first: Table.Object;
476     type: Table.Type;
477 BEGIN
478   first := IdentList(Table.Var);
479   Skip(S.colon);
480   type := Type();
481   WHILE first # NIL DO
482     first.type := type;
483     first := first.next
484   END
485 END VariableDeclaration;
486
487 (** DeclarationSequence = [CONST {ConstDeclaration ";"}]
488     [TYPE {TypeDeclaration ";"}] [VAR {VariableDeclaration ";"}]
489     {ProcedureDeclaration ";"}. *)
490 PROCEDURE DeclarationSequence(): Tree.Node;
491 VAR procedures: Tree.Node;
492 BEGIN
493   (* Sync *)
494   IF (sym < S.const) & (sym # S.end) & (sym # S.return) THEN
495     Machine.Error(Errors.noDeclaration);
496   REPEAT
497     S.Get(sym)
498   UNTIL (sym >= S.const) OR (sym = S.end) OR (sym = S.return)

```

```

499     END;
500
501     IF Skipped(S.const) THEN
502         WHILE sym = S.ident DO
503             ConstDeclaration();
504             Skip(S.semicolon)
505         END
506     END;
507
508     IF Skipped(S.type) THEN
509         WHILE sym = S.ident DO
510             TypeDeclaration();
511             Skip(S.semicolon)
512         END
513     END;
514
515     IF Skipped(S.var) THEN
516         WHILE sym = S.ident DO
517             VariableDeclaration();
518             Skip(S.semicolon)
519         END
520     END
521 RETURN procedures END DeclarationSequence;
522
523 (** FormalParameters = "(" [FPSection {";" FPSection}] ")" [" ":" qualident"].
524     FPSection = [VAR] ident {";" ident} ":" FormalType.
525     FormalType = {ARRAY OF} qualident. *)
526
527 (** ProcedureDeclaration = ProcedureHeading ";" ProcedureBody ident.
528     ProcedureHeading = PROCEDURE identdef [FormalParameters].
529     ProcedureBody = DeclarationSequence [BEGIN StatementSequence]
530     [RETURN expression] END.
531     identdef = ident. *)
532
533 (** import = ident [":"=" ident]. *)
534 PROCEDURE Import;
535 VAR name: Ident;
536 BEGIN
537     IF Check(S.ident) THEN
538         Strings.Copy(S.name, name);
539         S.Get(sym);
540         IF Skipped(S.becomes) THEN
541             Table.Import(name, S.name, modName);
542             S.Get(sym)
543         ELSE
544             Table.Import(name, name, modName)
545         END
546     END
547 END Import;
548
549 (** module = MODULE ident ";" [ImportList]
550     DeclarationSequence [BEGIN StatementSequence] END ident ".".

```

```

551     ImportList = IMPORT import {"", " import} ";". *)
552 PROCEDURE Module(): Tree.Node;
553 VAR module: Tree.Node;
554     name: Ident;
555     procedures, statementSequence: Tree.Node;
556 BEGIN
557     Skip(S.module);
558     IF Check(S.ident) THEN
559         Strings.Copy(S.name, modName);
560         S.Get(sym); (* ident *)
561
562         (* IMPORT *)
563         Skip(S.semicolon);
564         IF Skipped(S.import) THEN
565             Import;
566             WHILE Skipped(S.comma) DO
567                 Import
568             END;
569             Skip(S.semicolon)
570         END;
571
572         (* CONST, TYPE, VAR and BEGIN *)
573         procedures := DeclarationSequence();
574         IF Skipped(S.begin) THEN
575             statementSequence := StatementSequence()
576         END;
577
578         module := Builder.Enter(procedures, statementSequence, NIL);
579         Skip(S.end);
580         IF Check(S.ident) THEN
581             IF S.name # modName THEN
582                 Machine.Error(Errors.modNameMismatch)
583             ELSE
584                 S.Get(sym); (* ident *)
585                 Skip(S.period)
586             END
587         END
588     END
589 RETURN module END Module;
590
591 PROCEDURE Parse*(): Tree.Node;
592 VAR module: Tree.Node;
593 BEGIN
594     S.Init;
595     S.Get(sym);
596     Table.Init;
597     module := Module()
598 RETURN module END Parse;
599
600 BEGIN
601     NEW(dummy);
602     dummy.class := Table.Var;

```

```
603 dummy.type := Table.intType
604 END Parser.
```

Листинг 53. Полный исходный текст модуля Table (таблица символов)

```
1  MODULE Table;
2  IMPORT Out, Strings, Machine, Errors := MocErrors, Scanner;
3
4  CONST
5    (** Object.class values **)
6    Head* = 0; Const* = 1; Var*      = 2; VarPar* = 3; Field* = 4;
7    Typ*   = 5; SProc* = 6; SFunc* = 7; Mod*      = 8;
8
9    (** Type.form values **)
10   Byte* = 1; Bool*   = 2; Char*   = 3; Int*      = 4; Real* = 5;
11   Set*   = 6; Pointer* = 7; NilType* = 8; NoType* = 9; Proc* = 10;
12   String* = 11; Array* = 12; Record* = 13;
13
14   (** Standard procedure numbers **)
15   umlFunc* = 1; rorFunc* = 2; asrFunc* = 3; lslFunc* = 4;
16   lenFunc* = 5; chrFunc* = 6; ordFunc* = 7; fltFunc* = 8;
17   floorFunc* = 9; oddFunc* = 10; absFunc* = 11; unpkProc* = 12;
18   packProc* = 13; newProc* = 14; assertProc* = 15; exclProc* = 16;
19   inclProc* = 17; decProc* = 18; incProc* = 19;
20
21   (** System procedure numbers **)
22   sizeFunc* = 40; adrFunc* = 41; valFunc* = 42; regFunc* = 43;
23   bitFunc* = 44; copyProc* = 65; putProc* = 66; getProc* = 65;
24   putregProc* = 66;
25
26  TYPE
27   Ident* = Scanner.Ident;
28   StrVal* = Scanner.StrVal;
29
30   ConstVal* = POINTER TO ConstValDesc;
31   Type* = POINTER TO TypeDesc;
32   Object* = POINTER TO ObjectDesc;
33   Module* = POINTER TO ModuleDesc;
34
35   ConstExt* = POINTER TO ConstExtDesc;
36   ConstExtDesc* = RECORD
37     s*: StrVal
38   END;
39
40   ConstValDesc* = RECORD
41     ext*: ConstExt; (** For string constants *)
42     intVal*: INTEGER;
43     realVal*: REAL;
44     setVal*: SET
```

```

45  END;
46
47  TypeDesc* = RECORD
48    form*: INTEGER; (** See Type.form constants *)
49    paramCount*: INTEGER;
50    len*: INTEGER;
51    size*: INTEGER; (** In bytes *)
52    base*: Type;
53    dsc*: Object; (** Descendant object *)
54    typeObj*: Object (** Original object that introduced the type *)
55  END;
56
57  ObjectDesc* = RECORD
58    class*: INTEGER; (** See Object.class constants *)
59    name*: Ident;
60    dsc*: Object; (** Descendant object *)
61    type*: Type;
62    constVal*: ConstVal;
63    address*: INTEGER; (** For backend *)
64    next*: Object
65  END;
66
67  ModuleDesc* = RECORD(ObjectDesc)
68    originalName*: Ident (** Original name of the module; name is alias *)
69  END;
70
71  VAR topScope*, universe, system*: Object;
72    byteType*, boolType*, charType*, intType*, realType*: Type;
73    setType*, nilType*, noType*, strType*: Type;
74
75  PROCEDURE NewConst*(): ConstVal;
76  VAR c: ConstVal;
77  BEGIN
78    NEW(c)
79  RETURN c END NewConst;
80
81  PROCEDURE NewType(form, size: INTEGER): Type;
82  VAR t: Type;
83  BEGIN
84    NEW(t);
85    t.form := form;
86    t.size := size
87  RETURN t END NewType;
88
89  PROCEDURE NewObject*(name: Ident; class: INTEGER): Object;
90  VAR obj, x: Object;
91  BEGIN
92    x := topScope;
93    WHILE (x.next # NIL) & (x.next.name # name) DO x := x.next END;
94    IF x.next = NIL THEN
95      NEW(obj);
96      Strings.Copy(name, obj.name);

```

```

97     obj.class := class;
98     x.next := obj
99     ELSE
100     obj := x.next;
101     Machine.Error(Errors.multipleDefs)
102     END
103 RETURN obj END NewObject;
104
105 PROCEDURE OpenScope*;
106 VAR scope: Object;
107 BEGIN
108     NEW(scope);
109     scope.class := Head;
110     scope.dsc := topScope;
111     scope.next := NIL;
112     topScope := scope
113 END OpenScope;
114
115 PROCEDURE CloseScope*;
116 BEGIN
117     topScope := topScope.dsc
118 END CloseScope;
119
120 PROCEDURE Init*;
121 BEGIN
122     topScope := universe;
123     OpenScope
124 END Init;
125
126 (** Finds the object and returns it. *)
127 PROCEDURE ThisObject*(name: Ident): Object;
128 VAR scope, obj: Object;
129 BEGIN
130     scope := topScope;
131     REPEAT
132         obj := scope.next;
133         WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END;
134         scope := scope.dsc
135     UNTIL (obj # NIL) OR (scope = NIL)
136 RETURN obj END ThisObject;
137
138 (** Finds the object and returns it. *)
139 PROCEDURE ThisImport*(module: Object; name: Ident): Object;
140 VAR obj: Object;
141 BEGIN
142     IF module.name # "" THEN
143         obj := module.dsc;
144         WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END
145     END
146 RETURN obj END ThisImport;
147
148 (** Finds or adds module and returns it. *)

```

```

149 PROCEDURE ThisModule*(alias, name: Ident; declareNew: BOOLEAN): Object;
150 VAR module: Module;
151     obj, prevObj: Object;
152 BEGIN
153     (* Find module by name *)
154     prevObj := topScope;
155     obj := topScope.next;
156     WHILE (obj # NIL) & (obj(Module).originalName # name) DO
157         prevObj := obj;
158         obj := obj.next
159     END;
160     IF obj = NIL THEN (* New module *)
161         (* Make sure alias is not used *)
162         obj := topScope.next;
163         WHILE (obj # NIL) & (obj.name (*Alias*) # name) DO obj := obj.next END;
164         IF obj = NIL THEN (* All good, adding module to symbol table *)
165             NEW(module);
166             module.class := Mod;
167             Strings.Copy(name, module.originalName);
168             Strings.Copy(alias, module.name);
169             (* Add to symbol table *)
170             module.next := topScope.next;
171             topScope.next := module;
172             obj := module
173         ELSIF declareNew THEN (* Module already imported, could not declare *)
174             Machine.Error(Errors.multipleImport)
175         ELSE (* Module alias already used *)
176             Machine.Error(Errors.aliasTaken)
177         END
178     ELSIF declareNew THEN (* Module already imported, could not declare *)
179         Machine.Error(Errors.multipleImport)
180     END
181     RETURN obj END ThisModule;
182
183 (** Returns a list of objects inside module with the given name.
184     The supported module is Out. *)
185 PROCEDURE PseudolImport*(name: Scanner.Ident): Object;
186 VAR list, o, p: Object;
187     t: Type;
188 BEGIN
189     IF name = 'Out' THEN
190         (* Out.Int *)
191         NEW(o);
192         list := o;
193         o.class := Const;
194         o.name := 'Int';
195         NEW(o.constVal);
196         o.constVal.intVal := -21; (*FIXME*)
197         t := NewType(Proc, Machine.pointerSize);
198         o.type := t;
199         t.paramCount := 2;
200         t.base := noType;

```



```

201      (* Parameters *)
202      NEW(p);
203      t.dsc := p;
204      p.class := Var;
205      p.name := 'i';
206      p.type := intType;
207      NEW(p.next);
208      p := p.next;
209      p.class := Var;
210      p.name := 'n';
211      p.type := intType;
212
213      (* Out.Ln *)
214      NEW(o.next);
215      o := o.next;
216      o.class := Const;
217      o.name := 'Ln';
218      NEW(o.constVal);
219      o.constVal.intVal := -20; (*FIXME*)
220      t := NewType(Proc, Machine.pointerSize);
221      o.type := t;
222      t.paramCount := 0;
223      t.base := noType
224  END
225  RETURN list END Pseudolmport;
226
227  PROCEDURE Import*(alias, name, selfName: Scanner.Ident);
228  VAR obj, objects, module: Object;
229  BEGIN
230      Out.String('Importing '); Out.String(name);
231      Out.String(' as '); Out.String(alias);
232      Out.String(' to '); Out.String(selfName);
233      Out.Ln;
234      IF name = 'SYSTEM' THEN
235          module := ThisModule(alias, name, TRUE);
236          module.dsc := system
237      ELSE
238          objects := Pseudolmport(name);
239          IF objects = NIL THEN
240              Machine.NotImplemented('imports other than Out')
241          ELSE
242              module := ThisModule(alias, name, TRUE);
243              module.dsc := objects
244          END
245      END;
246      obj := topScope.next
247  END Import;
248
249  (** Outputs 2n spaces. *)
250  PROCEDURE Indent(n: INTEGER);
251  BEGIN
252      WHILE n # 0 DO

```

```

253     Out.String(' ');
254     DEC(n)
255 END
256 END Indent;
257
258 PROCEDURE PrintConstVal*(const: ConstVal; type: Type);
259 VAR f: INTEGER;
260 BEGIN
261     IF const # NIL THEN
262         IF type = NIL THEN
263             Out.String('NO TYPE')
264         ELSE
265             f := type.form;
266             IF (f = Byte) OR (f = Bool) OR (f = Char) OR (f = Int) OR (f = Proc) THEN
267                 Out.Int(const.intVal, 0)
268             ELSIF f = Real THEN
269                 Out.Real(const.realVal, 0)
270             END
271         END
272     END
273 END PrintConstVal;
274
275 PROCEDURE PrintType*(type: Type);
276 BEGIN
277     IF type = NIL THEN Out.String('NIL')
278     ELSIF type.form = Byte THEN Out.String('Byte')
279     ELSIF type.form = Bool THEN Out.String('Bool')
280     ELSIF type.form = Char THEN Out.String('Char')
281     ELSIF type.form = Int THEN Out.String('Int')
282     ELSIF type.form = Real THEN Out.String('Real')
283     ELSIF type.form = Set THEN Out.String('Set')
284     ELSIF type.form = Pointer THEN Out.String('Pointer')
285     ELSIF type.form = NilType THEN Out.String('NilType')
286     ELSIF type.form = NoType THEN Out.String('NoType')
287     ELSIF type.form = Proc THEN Out.String('Proc')
288     ELSIF type.form = String THEN Out.String('String')
289     ELSIF type.form = Array THEN Out.String('Array')
290     ELSIF type.form = Record THEN Out.String('Record')
291     ELSE Out.String('UNKNOWN')
292     END
293 END PrintType;
294
295 PROCEDURE PrintObject*(object: Object; indent: INTEGER);
296 BEGIN
297     Indent(indent);
298     IF object = NIL THEN
299         Out.String('NIL')
300     ELSE
301         Out.Char(" "); Out.String(object.name); Out.String(" ");
302         IF object.class = Const THEN
303             Out.String('Const ');
304             PrintConstVal(object.constVal, object.type)

```

```

305     ELSIF object.class = Var THEN
306         Out.String('Var')
307     ELSIF object.class = VarPar THEN
308         Out.String('VarPar')
309     ELSIF object.class = Typ THEN
310         Out.String('Typ')
311     ELSIF object.class = SProc THEN
312         Out.String('SProc')
313     ELSIF object.class = SFunc THEN
314         Out.String('SFunc')
315     ELSIF object.class = Mod THEN
316         Out.String('Mod')
317     ELSE
318         Out.String('class #'); Out.Int(object.class, 0)
319     END;
320     IF object.type # NIL THEN
321         Out.Char(' ');
322         PrintType(object.type)
323     END
324 END
325 END PrintObject;
326
327 PROCEDURE Print*(scope: Object; indent: INTEGER);
328 VAR obj: Object;
329 BEGIN
330     WHILE scope # NIL DO
331         Out.String('~ Scope ~'); Out.Ln;
332         obj := scope.next;
333         WHILE obj # NIL DO
334             PrintObject(obj, indent + 1); Out.Ln;
335             obj := obj.next
336         END;
337         scope := scope.dsc
338     END
339 END Print;
340
341 (** Creates a new object and places it in the beginning of the system list. *)
342 PROCEDURE Enter(name: ARRAY OF CHAR; class: INTEGER;
343     type: Type; paramCount, value: INTEGER);
344 VAR obj: Object;
345 BEGIN
346     NEW(obj);
347     Strings.Copy(name, obj.name);
348     obj.class := class;
349     obj.type := type;
350     (*obj.paramCount := paramCount; FIXME*)
351     NEW(obj.constVal);
352     obj.constVal.intVal := value;
353     obj.dsc := NIL;
354     IF class = Typ THEN
355         type.typeObj := obj
356     END;

```

```

357 (* Prepend obj to the list pointed to by system *)
358 obj.next := system;
359 system := obj
360 END Enter;
361
362 BEGIN
363   byteType := NewType(Int, 1);
364   boolType := NewType(Bool, 1);
365   charType := NewType(Char, 1);
366   intType := NewType(Int, 4);
367   realType := NewType(Real, 4);
368   setType := NewType(Set, 4);
369   nilType := NewType(NilType, 8);
370   noType := NewType(NoType, 4);
371   strType := NewType(String, 8);
372
373   (* Initialize Universe objects *)
374   (* Standard functions *)
375   Enter('UML', SFunc, intType, 2, umlFunc);
376   Enter('ROR', SFunc, intType, 2, rorFunc);
377   Enter('ASR', SFunc, intType, 2, asrFunc);
378   Enter('LSL', SFunc, intType, 2, lslFunc);
379   Enter('LEN', SFunc, intType, 1, lenFunc);
380   Enter('CHR', SFunc, charType, 1, chrFunc);
381   Enter('ORD', SFunc, intType, 1, ordFunc);
382   Enter('FLT', SFunc, realType, 1, fltFunc);
383   Enter('FLOOR', SFunc, intType, 1, floorFunc);
384   Enter('ODD', SFunc, boolType, 1, oddFunc);
385   Enter('ABS', SFunc, intType, 1, absFunc);
386
387   (* Standard procedures *)
388   Enter('UNPK', SProc, noType, 2, unpackProc);
389   Enter('PACK', SProc, noType, 2, packProc);
390   Enter('NEW', SProc, noType, 1, newProc);
391   Enter('ASSERT', SProc, noType, 1, assertProc);
392   Enter('EXCL', SProc, noType, 2, exclProc);
393   Enter('INCL', SProc, noType, 2, inclProc);
394   Enter('DEC', SProc, noType, 1, decProc);
395   Enter('INC', SProc, noType, 1, incProc);
396
397   (* Types *)
398   Enter('SET', Typ, setType, 0, 0);
399   Enter('BOOLEAN', Typ, boolType, 0, 0);
400   Enter('BYTE', Typ, byteType, 0, 0);
401   Enter('CHAR', Typ, charType, 0, 0);
402   Enter('LONGREAL', Typ, realType, 0, 0); (* Alias to REAL *)
403   Enter('REAL', Typ, realType, 0, 0);
404   Enter('LONGINT', Typ, intType, 0, 0); (* Alias to INTEGER *)
405   Enter('INTEGER', Typ, intType, 0, 0);
406
407   OpenScope;
408   topScope.next := system;

```

```

409 universe := topScope;
410 system := NIL;
411
412 (* Initialize unsafe pseudo-module SYSTEM objects *)
413 (* Functions *)
414 Enter('SIZE', SFunc, intType, 1, sizeFunc);
415 Enter('ADR', SFunc, intType, 1, adrFunc);
416 Enter('VAL', SFunc, intType, 2, valFunc);
417 Enter('REG', SFunc, intType, 1, regFunc);
418 Enter('BIT', SFunc, boolType, 1, bitFunc);
419 (* Procedures *)
420 Enter('COPY', SProc, noType, 2, copyProc);
421 Enter('PUT', SProc, noType, 2, putProc);
422 Enter('GET', SProc, noType, 2, getProc);
423 Enter('PUTREG', SProc, noType, 2, putregProc)
424 END Table.

```

Листинг 54. Полный исходный текст модуля Tree (абстрактное синтаксическое дерево)

```

1 MODULE Tree;
2 IMPORT Out, Scanner, T := Table, Errors := MoeErrors;
3
4 CONST
5   (** Node.class values **)
6   NVar* = 0; NVarPar* = 1; NField* = 2; NDeref* = 3; NIndex* = 4;
7   NGuard* = 5; NExpGuard* = 6; NConst* = 7; NType* = 8; NProc* = 9;
8   NUpTo* = 10; NMonadic* = 11; NDyadic* = 12; NCall* = 13; NInitTD* = 14;
9   NIf* = 15; NCaseElse* = 16; NCaseDo* = 17; NEnter* = 18; NAssign* = 19;
10  NIfElse* = 20; NCase* = 21; NWhile* = 22; NRepeat* = 23; NTrap* = 28;
11  NFixup* = 31;
12
13   (** Node.subclass values **)
14   assign* = 1;
15
16 TYPE
17   Ident = Scanner.Ident;
18
19   Node* = POINTER TO NodeDesc;
20   NodeDesc* = RECORD
21     left*, right*, link*: Node;
22     class*: INTEGER; (** See Node.class constants *)
23     subclass*: INTEGER; (** See Node.subclass constants *)
24     type*: T.Type;
25     object*: T.Object;
26     constVal*: T.ConstVal
27   END;
28
29   (** Outputs 2n spaces. *)
30   PROCEDURE Indent(n: INTEGER);

```

```

31 BEGIN
32   WHILE n # 0 DO
33     Out.String(' ');
34     DEC(n)
35   END
36 END Indent;
37
38 PROCEDURE PrintNodeName(node: Node);
39 BEGIN
40   IF node.class = NVar THEN Out.String('NVar')
41   ELSIF node.class = NVarPar THEN Out.String('NVarPar')
42   ELSIF node.class = NField THEN Out.String('NField')
43   ELSIF node.class = NDeref THEN Out.String('NDeref')
44   ELSIF node.class = NIndex THEN Out.String('NIndex')
45   ELSIF node.class = NGuard THEN Out.String('NGuard')
46   ELSIF node.class = NExpGuard THEN Out.String('NExpGuard')
47   ELSIF node.class = NConst THEN Out.String('NConst')
48   ELSIF node.class = NType THEN Out.String('NType')
49   ELSIF node.class = NProc THEN Out.String('NProc')
50   ELSIF node.class = NUpTo THEN Out.String('NUpTo')
51   ELSIF node.class = NMonadic THEN Out.String('NMonadic')
52   ELSIF node.class = NDyadic THEN Out.String('NDyadic')
53   ELSIF node.class = NCall THEN Out.String('NCall')
54   ELSIF node.class = NInitTD THEN Out.String('NInitTD')
55   ELSIF node.class = NIf THEN Out.String('NIf')
56   ELSIF node.class = NCaseElse THEN Out.String('NCaseElse')
57   ELSIF node.class = NCaseDo THEN Out.String('NCaseDo')
58   ELSIF node.class = NEnter THEN Out.String('NEnter')
59   ELSIF node.class = NAssign THEN Out.String('NAssign')
60   ELSIF node.class = NIfElse THEN Out.String('NIfElse')
61   ELSIF node.class = NCase THEN Out.String('NCase')
62   ELSIF node.class = NWhile THEN Out.String('NWhile')
63   ELSIF node.class = NRepeat THEN Out.String('NRepeat')
64   ELSIF node.class = NTrap THEN Out.String('NTrap')
65   ELSIF node.class = NFixup THEN Out.String('NFixup')
66   ELSE Out.String('UNKNOWN')
67   END
68 END PrintNodeName;
69
70 PROCEDURE Print*(node: Node; indent: INTEGER);
71 BEGIN
72   IF node # NIL THEN
73     Indent(indent); PrintNodeName(node);
74     IF (node.class = NDyadic) OR (node.class = NMonadic) THEN
75       Out.Char(' ');
76       Errors.PrintSymbol(node.subclass)
77     END;
78     IF (node.object # NIL) OR (node.class = NEnter) THEN
79       Out.String(' (Object: ');
80       T.PrintObject(node.object, 0);
81       Out.Char(')')
82     END;

```

```

83   IF node.constVal # NIL THEN
84       Out.String(' (Const: ');
85       Out.Int(node.constVal.intVal, 0);
86       Out.Char(')')
87   END;
88   Out.Ln;
89   IF node.left # NIL THEN
90       Indent(indent); Out.String('Left:'); Out.Ln;
91       Print(node.left, indent + 1);
92   END;
93   IF node.right # NIL THEN
94       Indent(indent); Out.String('Right:'); Out.Ln;
95       Print(node.right, indent + 1);
96   END;
97   IF node.link # NIL THEN
98       Indent(indent); Out.String('~ Link ~'); Out.Ln;
99       Print(node.link, indent)
100  END
101  END
102  END Print;
103
104  PROCEDURE NewNode*(class: INTEGER): Node;
105  VAR node: Node;
106  BEGIN
107      NEW(node);
108      node.class := class;
109      node.subclass := 0
110  RETURN node END NewNode;
111
112  END Tree.

```

Листинг 55. Полный исходный текст модуля Builder (построитель абстрактного синтаксического дерева)

```

1  MODULE Builder;
2  IMPORT Out, Machine, Errors := MocErrors, Table, Tree, S := Scanner;
3
4  PROCEDURE Enter*(procedures, statementSequence: Tree.Node;
5      containingProc: Table.Object): Tree.Node;
6  VAR enter: Tree.Node;
7  BEGIN
8      enter := Tree.NewNode(Tree.NEnter);
9      enter.type := Table.noType;
10     enter.object := containingProc;
11     enter.left := procedures;
12     enter.right := statementSequence
13  RETURN enter END Enter;
14
15  PROCEDURE NewIntConst*(value: INTEGER): Tree.Node;
16  VAR x: Tree.Node;

```

```

17 BEGIN
18   x := Tree.NewNode(Tree.NConst);
19   x.type := Table.intType;
20   x.constVal := Table.NewConst();
21   x.constVal.intVal := value
22 RETURN x END NewIntConst;
23
24 PROCEDURE NewRealConst*(value: REAL): Tree.Node;
25 VAR x: Tree.Node;
26 BEGIN
27   x := Tree.NewNode(Tree.NConst);
28   x.type := Table.realType;
29   x.constVal := Table.NewConst();
30   x.constVal.realVal := value
31 RETURN x END NewRealConst;
32
33 PROCEDURE NewLeaf*(object: Table.Object): Tree.Node;
34 VAR node: Tree.Node;
35 BEGIN
36   IF object.class = Table.Var THEN
37     node := Tree.NewNode(Tree.NVar)
38   ELSIF object.class = Table.VarPar THEN
39     node := Tree.NewNode(Tree.NVarPar)
40   ELSIF object.class = Table.Const THEN
41     IF (object.type # NIL) & (object.type.form = Table.Proc) THEN
42       node := Tree.NewNode(Tree.NProc)
43     ELSE (* constant *)
44       node := Tree.NewNode(Tree.NConst);
45       IF object.constVal # NIL THEN
46         node.constVal := Table.NewConst();
47         node.constVal↑ := object.constVal↑
48       END
49     END
50   ELSIF object.class = Table.Typ THEN
51     node := Tree.NewNode(Tree.NType)
52   ELSIF object.class IN {Table.SProc, Table.SFunc} THEN
53     node := Tree.NewNode(Tree.NProc)
54   ELSE
55     ASSERT(FALSE)
56   END;
57   node.object := object;
58   node.type := object.type
59 RETURN node END NewLeaf;
60
61 (** Performs an optimized monadic operation op on x or creates a new
62     monadic operator node. Returns the modified or created node.
63     The supported operations are: ¬, −, IS, convesion, ABS, CAP,
64     ODD, ADR, CC. *)
65 PROCEDURE MonadicOperator*(op: INTEGER; x: Tree.Node): Tree.Node;
66 VAR mondaic: Tree.Node;
67 BEGIN
68   IF x.class = Tree.NConst THEN

```



```

69   IF op = S.not THEN
70       IF x.type.form = Table.Bool THEN
71           x.constVal.intVal := 1 - x.constVal.intVal
72       (* TODO Table.Set *)
73       ELSE
74           Machine.Error(Errors.badMonadicOperand)
75       END
76   ELSIF op = S.minus THEN
77       IF x.type.form = Table.Int THEN
78           x.constVal.intVal := -x.constVal.intVal
79       (* TODO form = Table.Set *)
80       ELSE
81           Machine.Error(Errors.badMonadicOperand)
82       END
83   END
84   ELSE
85       mondaic := Tree.NewNode(Tree.NMonadic);
86       mondaic.subclass := op;
87       mondaic.type := x.type; (*FIXME*)
88       mondaic.left := x
89   END
90   RETURN mondaic END MonadicOperator;
91
92   (** Performs an optimized dyadic operation op on NConst nodes x and y.
93       Returns the modified node x.
94       The supported operations are: *, DIV, MOD, &, +, -, OR, =, #,
95       <, <=, >, >=, IN, ASH, LEN, BIT, LSH, ROT. *)
96   PROCEDURE DyadicConstOperator*(op: INTEGER; x, y: Tree.Node): Tree.Node;
97   VAR f: INTEGER;
98   BEGIN
99       x.object := NIL;
100      f := x.type.form;
101      IF y.type.form # f THEN
102          Machine.Error(Errors.incompatibleTypes)
103      ELSIF op = S.times THEN
104          IF f = Table.Int THEN
105              x.constVal.intVal := x.constVal.intVal * y.constVal.intVal
106          ELSIF f = Table.Real THEN
107              x.constVal.realVal := x.constVal.realVal * y.constVal.realVal
108          ELSE
109              Machine.Error(Errors.badDyadicOperands)
110          END
111      ELSIF op = S.div THEN (*TODO*)
112      ELSIF op = S.mod THEN (*TODO*)
113      ELSIF op = S.and THEN (*TODO*)
114      ELSIF op = S.plus THEN
115          IF f = Table.Int THEN
116              x.constVal.intVal := x.constVal.intVal + y.constVal.intVal
117          ELSIF f = Table.Real THEN
118              x.constVal.realVal := x.constVal.realVal + y.constVal.realVal
119          ELSE
120              Machine.Error(Errors.badDyadicOperands)

```

```

121     END
122   ELSIF op = S.minus THEN
123     IF f = Table.Int THEN
124       x.constVal.intVal := x.constVal.intVal - y.constVal.intVal
125     ELSIF f = Table.Real THEN
126       x.constVal.realVal := x.constVal.realVal - y.constVal.realVal
127     ELSE
128       Machine.Error(Errors.badDyadicOperands)
129     END
130   ELSIF op = S.or THEN (*TODO*)
131   ELSIF op = S.eql THEN (*TODO*)
132   ELSIF op = S.neq THEN (*TODO*)
133   ELSIF op = S.lss THEN (*TODO*)
134   ELSIF op = S.leq THEN (*TODO*)
135   ELSIF op = S.gtr THEN (*TODO*)
136   ELSIF op = S.geq THEN (*TODO, IN etc. *)
137   END
138 RETURN x END DyadicConstOperator;
139
140 (** Performs an optimized dyadic operation op on x and y or creates a new
141     dyadic operator node. Returns the modified node x or a new node.
142     The supported operations are: *, DIV, MOD, &, +, -, OR, =, #,
143     <, <=, >, >=, IN, ASH, LEN, BIT, LSH, ROT. *)
144 PROCEDURE DyadicOperator*(op: INTEGER; x, y: Tree.Node): Tree.Node;
145 VAR dyadic: Tree.Node;
146 BEGIN
147   IF (x.class = Tree.NConst) & (y.class = Tree.NConst) THEN
148     dyadic := DyadicConstOperator(op, x, y)
149   ELSE
150     dyadic := Tree.NewNode(Tree.NDyadic);
151     dyadic.subclass := op;
152     dyadic.type := x.type; (*FIXME change type depending on the op?*)
153     dyadic.left := x;
154     dyadic.right := y
155   END
156 RETURN dyadic END DyadicOperator;
157
158 (** Appends list y to the end of the list x..last. *)
159 PROCEDURE Link*(VAR x, last: Tree.Node; y: Tree.Node);
160 BEGIN
161   IF x = NIL THEN x := y ELSE last.link := y END;
162   WHILE y.link # NIL DO y := y.link END;
163   last := y
164 END Link;
165
166 PROCEDURE Param*(VAR actual: Tree.Node; formal: Table.Object);
167 VAR t: Table.Type;
168 BEGIN
169   IF formal.type.form # Table.NoType THEN
170     IF actual.type # formal.type THEN
171       Machine.Error(Errors.incompatibleTypes)
172     END

```

```

173 END
174 END Param;
175
176 (** Returns a list of formal parameters of procedure x. *)
177 PROCEDURE PrepareCall*(x: Tree.Node): Table.Object;
178 VAR list: Table.Object;
179 BEGIN
180   list := x.object.type.dsc
181 RETURN list END PrepareCall;
182
183 (** Creates and returns a new NCall node. *)
184 PROCEDURE Call*(procedure, actualParams: Tree.Node;
185   formalParams: Table.Object): Tree.Node;
186 VAR call: Tree.Node;
187 BEGIN
188   call := Tree.NewNode(Tree.NCall);
189   call.left := procedure;
190   call.right := actualParams
191   (* TODO check parameters *)
192 RETURN call END Call;
193
194 END Builder.

```

Листинг 56. Полный исходный текст модуля Traverser (обходчик абстрактного синтаксического дерева)

```

1 MODULE Traverser;
2 IMPORT Out, Machine, Errors := MocErrors, E := Emitter, Generator, Table, Tree;
3
4 PROCEDURE Designator(n: Tree.Node; VAR z: E.Item);
5 BEGIN
6   IF (n.class = Tree.NVar) OR (n.class = Tree.NVarPar) THEN
7     z.node := n;
8     z.type := n.object.type;
9     z.mode := E.Abs; (* Global variable *)
10    z.address := n.object.address;
11    z.offset := 0;
12    z.index := 0
13  ELSE
14    Machine.NotImplemented('designator of this type')
15  END
16 END Designator;
17
18 PROCEDURE Expression(n: Tree.Node; VAR z: E.Item);
19 VAR x, y: E.Item;
20   f: INTEGER;
21   const: Table.ConstVal;
22   type: Table.Type;
23 BEGIN
24   IF n.class = Tree.NConst THEN

```

```

25   IF n.type.form IN {Table.Int, Table.Char} THEN
26       z.mode := Table.Const;
27       z.address := n.constVal.intVal
28   ELSE
29       Machine.NotImplemented('constant expression of this type')
30   END
31   ELSIF n.class = Tree.NVar THEN
32       Designator(n, z)
33   ELSIF n.class = Tree.NMonadic THEN
34       Machine.NotImplemented('monadic expression')
35   ELSIF n.class = Tree.NDyadic THEN
36       Machine.NotImplemented('dyadic expression')
37   ELSIF n.class = Tree.NProc THEN
38       Machine.NotImplemented('procedure expression')
39   ELSE
40       Machine.NotImplemented('this expression class')
41   END
42   END Expression;
43
44   PROCEDURE Statement(n: Tree.Node);
45   VAR x, y, z: E.Item;
46       con: Table.ConstVal;
47   BEGIN
48       WHILE ¬Machine.hadErrors & (n # NIL) DO
49           IF n.class = Tree.NEnter THEN
50               IF n.object = NIL THEN (* n is module *)
51                   E.Enter;
52                   Statement(n.right);
53                   E.Exit
54               ELSE (* n is procedure *)
55                   Machine.NotImplemented('procedure declaration')
56               END
57           ELSIF n.class = Tree.NAssign THEN
58               IF n.subclass = Tree.assign THEN
59                   Expression(n.right, x);
60                   E.Relation(x); (* Load condition code if required *)
61                   Expression(n.left, z);
62                   E.Assign(z, x)
63               ELSE
64                   Machine.NotImplemented('this statement type')
65               END
66           ELSIF n.class = Tree.NCall THEN
67               IF n.left.object.class # Table.Const THEN
68                   Machine.NotImplemented('procedure variable call');
69               ELSE
70                   con := n.left.object.constVal;
71                   IF con.intVal = -20 THEN
72                       E.OutLn
73                   ELSIF con.intVal = -21 THEN
74                       E.OutInt(n.right)
75                   ELSE
76                       Machine.NotImplemented('this standard procedure call');

```

```

77         END
78     END
79 ELSE
80     Machine.NotImplemented('this node class');
81 END;
82 n := n.link
83 END
84 END Statement;
85
86 PROCEDURE SetAddressesAndSizes*(topScope: Table.Object);
87 VAR object: Table.Object;
88     size: INTEGER;
89 BEGIN
90     size := 0;
91     object := topScope.next;
92     WHILE object # NIL DO
93         IF object.class = Table.Var THEN
94             object.address := size;
95             INC(size, object.type.size)
96         END;
97         object := object.next
98     END;
99     Generator.SetDataLen(size)
100 END SetAddressesAndSizes;
101
102 PROCEDURE Traverse*(module: Tree.Node);
103 BEGIN
104     Generator.Init;
105     Statement(module)
106 END Traverse;
107
108 END Traverser.

```

Листинг 57. Полный исходный текст модуля Emitter (испускатель машинного кода)

```

1  MODULE Emitter;
2  IMPORT Out, Machine, Errors := MocErrors, Table, Tree, G := Generator;
3
4  CONST
5      (** Item.mode values for ARM64 **)
6      Abs* = 20;
7
8  TYPE
9      Item* = RECORD
10         mode*: INTEGER; (** Object.class values + Item.mode values *)
11         type*: Table.Type;
12         node*: Tree.Node;
13         address*, offset*, index*: INTEGER
14     END;

```

```

15
16 PROCEDURE Print*(x: Item);
17 BEGIN
18   Out.String('address='); Out.Int(x.address, 0);
19   Out.String(' mode='); Out.Int(x.mode, 0);
20   Out.String(' type='); Table.PrintType(x.type);
21   Out.String(' node='); Tree.Print(x.node, 0)
22 END Print;
23
24 PROCEDURE Assign*(VAR z, x: Item);
25 BEGIN
26   IF z.type.form # Table.Int THEN
27     Machine.NotImplemented('assignment to a non-INTEGER')
28   ELSEIF x.mode # Table.Const THEN
29     Machine.NotImplemented('assignment of a non-constant value')
30   ELSE
31     G.MovHImm(0, x.address);
32     G.AdRp(1, 2);
33     IF z.address # 0 THEN
34       G.AddImm(1, 1, z.address)
35     END;
36     G.StrH(0, 1)
37   END
38 END Assign;
39
40 PROCEDURE Relation*(VAR x: Item);
41 BEGIN
42   (* TODO *)
43 END Relation;
44
45 PROCEDURE OutLn*;
46 BEGIN
47   G.MovImm( 0, 1);      (* stdout *)
48   G.AdRp ( 1, 5);      (* address of 0AX *)
49   G.AddImm( 1, 1, 0FFFH);
50   G.MovImm( 2, 1);      (* length = 1 *)
51   G.MovImm(16, 4);      (* write *)
52   G.Svc(0)
53 END OutLn;
54
55 PROCEDURE OutInt*(params: Tree.Node);
56 BEGIN
57   G.MovImm( 0, 1);      (* stdout *)
58   G.AdRp ( 1, 2);      (* address *)
59   G.AddImm( 1, 1, params.object.address);
60   G.MovImm( 2, 1);      (* length = 1 *)
61   G.MovImm(16, 4);      (* write *)
62   G.Svc(0)
63 END OutInt;
64
65 PROCEDURE Enter*;
66 BEGIN

```

```

67 END Enter;
68
69 PROCEDURE Exit*;
70 BEGIN
71   G.Exit
72 END Exit;
73
74 END Emitter.

```

Листинг 58. Полный исходный текст модуля Generator (низкоуровневый генератор)

```

1  MODULE Generator;
2  IMPORT Out, Strings, Files, Machine, Errors := MocErrors;
3
4  CONST
5    maxCodeLen* = 2000H;
6    maxDataLen* = 2000H;
7
8  VAR
9    code: ARRAY maxCodeLen OF BYTE;
10   pc: INTEGER;
11
12   dataLen: INTEGER; (** For global variables *)
13
14   outOfMemory: BOOLEAN;
15
16  PROCEDURE OutOfMemory;
17  BEGIN
18    IF ¬outOfMemory THEN
19      outOfMemory := TRUE;
20      Machine.Error(Errors.outOfMemory)
21    END
22  END OutOfMemory;
23
24  (** Outputs 1 byte. *)
25  PROCEDURE Byte*(x: BYTE);
26  BEGIN
27    IF pc = LEN(code) THEN
28      OutOfMemory
29    ELSE
30      code[pc] := x;
31      INC(pc)
32    END
33  END Byte;
34
35  (** Outputs 2 bytes. Little-endian. *)
36  PROCEDURE Word*(x: INTEGER);
37  BEGIN
38    IF pc > LEN(code) - 2 THEN

```

```

39     OutOfMemory
40 ELSE
41     code[pc] := x MOD 100H;
42     INC(pc);
43     code[pc] := x DIV 100H MOD 100H;
44     INC(pc)
45 END
46 END Word;
47
48 (** Outputs a double word = 4 bytes. Little-endian.*)
49 PROCEDURE DWord*(x: INTEGER);
50 BEGIN
51     IF pc > LEN(code) - 4 THEN
52         OutOfMemory
53     ELSE
54         code[pc] := x MOD 100H;
55         INC(pc);
56         code[pc] := x DIV 100H MOD 100H;
57         INC(pc);
58         code[pc] := x DIV 10000H MOD 100H;
59         INC(pc);
60         code[pc] := x DIV 1000000H MOD 100H;
61         INC(pc)
62     END
63 END DWord;
64
65 (** Outputs a set as 4 bytes. Little-endian.*)
66 PROCEDURE Set(x: SET);
67 BEGIN
68     DWord(ORD(x))
69 END Set;
70
71 (** Outputs string to the file (one byte per character). *)
72 PROCEDURE String(s: ARRAY OF CHAR);
73 VAR i: INTEGER;
74 BEGIN
75     i := 0;
76     WHILE s[i] # 0X DO
77         Byte(ORD(s[i]));
78         INC(i)
79     END
80 END String;
81
82 (** Generates code that terminates the program. *)
83 PROCEDURE Exit*;
84 BEGIN
85     Word(00000H); Word(0D280H);
86     Word(00030H); Word(0D280H);
87     Word(00001H); Word(0D400H)
88 END Exit;
89
90 (** MOV x{r}, #{imm} *)

```



```

91  PROCEDURE MovImm*(r, imm: INTEGER);
92  VAR n: INTEGER;
93  BEGIN
94    n := (294H * 10000H + imm MOD 10000H) * 20H + r;
95    n := -7FFFFFFFH - 1 + n; (* Set highest bit for 64-bit *)
96    DWord(n)
97  END MovImm;
98
99  (** MOV w{r}, #{imm} *)
100 PROCEDURE MovHImm*(r, imm: INTEGER);
101 VAR n: INTEGER;
102 BEGIN
103   n := (294H * 10000H + imm MOD 10000H) * 20H + r;
104   (* Keep highest bit 0 for 32-bit operand *)
105   DWord(n)
106 END MovHImm;
107
108 (** ADRP w{r}, #{imm} *)
109 PROCEDURE Adrp*(r, imm: INTEGER);
110 VAR n, immlo, immhi: INTEGER;
111 BEGIN
112   immlo := imm MOD 4;
113   immhi := imm DIV 4;
114   n := ((immlo * 20H + 10H) * 80000H + immhi) * 20H + r;
115   n := -7FFFFFFFH - 1 + n; (* Set highest bit for op=ADRP *)
116   DWord(n)
117 END Adrp;
118
119 (** MOV x{r1}, x{r2}, #{imm} *)
120 PROCEDURE AddImm*(r1, r2, imm: INTEGER);
121 VAR n: INTEGER;
122 BEGIN
123   n := ((22H * 2000H + imm MOD 1000H) * 20H + r2) * 20H + r1;
124   n := -7FFFFFFFH - 1 + n; (* Set highest bit for sf=64bit *)
125   DWord(n)
126 END AddImm;
127
128 (** STR w{r1}, [x{r2}] *)
129 PROCEDURE StrH*(r1, r2: INTEGER);
130 VAR n, Rm, option, S: INTEGER;
131 BEGIN
132   Rm := 0;
133   option := 0;
134   S := 0;
135   n := (((1C8H * 20H + Rm) * 8 + option) * 2 + S) * 80H + r2) * 20H + r1;
136   n := -7FFFFFFFH - 1 + n; (* Set highest bit *)
137   DWord(n)
138 END StrH;
139
140 (** SVC {a}, a is ignored for now. *)
141 PROCEDURE Svc*(a: INTEGER);
142 VAR n: INTEGER;

```

```

143 BEGIN
144   n := 54000001H;
145   n := -7FFFFFFFH - 1 + n; (* Set highest bit *)
146   DWord(n)
147 END Svc;
148
149 PROCEDURE CodeLength*(): INTEGER;
150 RETURN pc END CodeLength;
151
152 PROCEDURE SetDataLen*(len: INTEGER);
153 BEGIN
154   dataLen := len
155 END SetDataLen;
156
157 PROCEDURE DataLength*(): INTEGER;
158 RETURN dataLen END DataLength;
159
160 PROCEDURE OutputCode*(VAR r: Files.Rider);
161 BEGIN
162   Files.WriteBytes(r, code, pc);
163 END OutputCode;
164
165 PROCEDURE OutputData*(VAR r: Files.Rider);
166 VAR i: INTEGER;
167 BEGIN
168   FOR i := 0 TO dataLen - 1 DO Files.Write(r, 0) END
169 END OutputData;
170
171 PROCEDURE Init*;
172 VAR i: INTEGER;
173 BEGIN
174   FOR i := 0 TO LEN(code) - 1 DO code[i] := 0 END;
175   pc := 0;
176   outOfMemory := FALSE
177 END Init;
178
179 END Generator.

```

Листинг 59. Полный исходный текст модуля Linker (компоновщик)

```
1  MODULE Linker;
2  IMPORT Out, Strings, Files, Machine, Errors := MocErrors, E := Emitter,
3     Generator, Table, Tree, SYSTEM;
4
5  CONST
6     codeSpace = Generator.maxCodeLen; (** Maximum code length *)
7     dataSpace = Generator.maxDataLen; (** Maximum data length *)
8
9  TYPE
10     HUGEINT = SYSTEM.INT64;
11
12  VAR
13     fname: ARRAY 256 OF CHAR;
14     F: Files.File;
15     r: Files.Rider;
16
17     codeLen: INTEGER;
18     dataLen: INTEGER;
19
20     (** Outputs 1 byte to the file. *)
21     PROCEDURE Byte(x: BYTE);
22     BEGIN
23         Files.Write(r, x)
24     END Byte;
25
26     (** Outputs 2 bytes to the file. Little-endian.**)
27     PROCEDURE Word(x: INTEGER);
28     BEGIN
29         Byte(x MOD 100H);
30         Byte(x DIV 100H MOD 100H)
31     END Word;
32
33     (** Outputs a double word = 4 bytes to the file. Little-endian.**)
34     PROCEDURE DWord(x: INTEGER);
35     BEGIN
36         Byte(x MOD 100H);
37         Byte(x DIV 100H MOD 100H);
38         Byte(x DIV 10000H MOD 100H);
39         Byte(x DIV 1000000H MOD 100H)
40     END DWord;
41
42     (** Outputs a quadruple word = 8 bytes to the file. Little-endian.**)
43     PROCEDURE QWord(x: HUGEINT);
44     BEGIN
45         Byte(x MOD 100H);
46         Byte(x DIV 100H MOD 100H);
47         Byte(x DIV 10000H MOD 100H);
```

```

48   Byte(x DIV 1000000H MOD 100H);
49   Byte(x DIV 1000000H DIV 100H MOD 100H);
50   Byte(x DIV 1000000H DIV 10000H MOD 100H);
51   Byte(x DIV 1000000H DIV 1000000H MOD 100H);
52   Byte(x DIV 1000000H DIV 1000000H DIV 100H MOD 100H)
53 END QWord;
54
55 (** Outputs a set as 4 bytes to the file. Little-endian.*)
56 PROCEDURE Set(x: SET);
57 BEGIN
58     DWord(ORD(x))
59 END Set;
60
61 (** Outputs string to the file (one byte per character) and then
62     zero-bytes until total of n bytes is reached.
63     If n is smaller than the string, the string is not truncated. *)
64 PROCEDURE String(s: ARRAY OF CHAR; n: INTEGER);
65 VAR i: INTEGER;
66 BEGIN
67     i := 0;
68     WHILE s[i] # 0X DO
69         Byte(ORD(s[i]));
70         INC(i)
71     END;
72     WHILE i < n DO
73         Byte(0);
74         INC(i)
75     END
76 END String;
77
78 (** Outputs n zero bytes to the file. *)
79 PROCEDURE Zeros(n: INTEGER);
80 BEGIN
81     WHILE n > 0 DO
82         Byte(0);
83         DEC(n)
84     END
85 END Zeros;
86
87 (** Outputs first 32 bytes – the Mach-O header to the file. *)
88 PROCEDURE Header;
89 BEGIN
90     (* FEED-FACF Magic Number, little-endian *)
91     Byte(0CFH); Byte(0FAH); Byte(0EDH); Byte(0FEH);
92     (* CPU Type = ARM64 *)
93     DWord(100000CH);
94     (* CPU Subtype = ARM64_ALL *)
95     DWord(0);
96     (* File Type = MH_EXECUTE *)
97     DWord(2);
98     (* Number of Load Commands *)
99     DWord(16);

```

```

100  (* Size of Load Commands in bytes *)
101  DWord(744-16+152); (* 152 is data segment command *)
102  (* Flags = {NOUNDEFS, DYLDLINK, TWOLEVEL, PIE} *)
103  Set({0, 2, 7, 21});
104  (* Reserved = 0 *)
105  DWord(0)
106  END Header;
107
108  PROCEDURE Command0;
109  BEGIN
110    (* Command LC_SEGMENT_64 *)
111    DWord(19H);
112    (* Command Size in bytes *)
113    DWord(72);
114    (* Segment Name *)
115    String('__PAGEZERO', 16);
116    (* VM Address *)
117    QWord(0);
118    (* VM Size *)
119    QWord(1000000H * 100H); (* 4 GB *)
120    (* File Offset *)
121    QWord(0);
122    (* File Size *)
123    QWord(0);
124    (* Maximum VM Protection *)
125    DWord(0);
126    (* Initial VM Protection *)
127    DWord(0);
128    (* Number of Sections *)
129    DWord(0);
130    (* Flags *)
131    Set({})
132  END Command0;
133
134  PROCEDURE Command1;
135
136  PROCEDURE Section0;
137  BEGIN
138    (* Section Name *)
139    String('__text', 16);
140    (* Segment Name *)
141    String('__TEXT', 16);
142    (* Address *)
143    QWord(1000000H * 100H + 4000H - codeSpace);
144    (* Size in bytes *)
145    QWord(codeLen);
146    (* Offset *)
147    DWord(4000H - codeSpace);
148    (* Alignment = 2^2 = 4 *)
149    DWord(2);
150    (* Relocations Offset *)
151    DWord(0);

```

```

152     (* Number of Relocations *)
153     DWord(0);
154     (* Flags = S_ATTR_SOME_INSTRUCTIONS + S_ATTR_PURE_INSTRUCTIONS *)
155     Set({10, 31});
156     (* Reserved *)
157     DWord(0);
158     DWord(0);
159     DWord(0)
160 END Section0;
161
162 PROCEDURE Section1;
163 BEGIN
164     (* Section Name *)
165     String('__unwind_info', 16);
166     (* Segment Name *)
167     String('__TEXT', 16);
168     (* Address *)
169     QWord(1000000H * 100H + 4000H - codeSpace + codeLen);
170     (* Size in bytes *)
171     QWord(codeSpace - codeLen);
172     (* Offset *)
173     DWord(4000H - codeSpace + codeLen);
174     (* Alignment = 2^2 = 4 *)
175     DWord(2);
176     (* Relocations Offset *)
177     DWord(0);
178     (* Number of Relocations *)
179     DWord(0);
180     (* Flags = S_REGULAR (empty set) *)
181     Set({});
182     (* Reserved *)
183     DWord(0);
184     DWord(0);
185     DWord(0)
186 END Section1;
187
188 BEGIN (* Command1 *)
189     (* Command LC_SEGMENT_64 *)
190     DWord(19H);
191     (* Command Size in bytes *)
192     DWord(232);
193     (* Segment Name *)
194     String('__TEXT', 16);
195     (* VM Address *)
196     QWord(1000000H * 100H); (* 4 GB *)
197     (* VM Size *)
198     QWord(4000H);
199     (* File Offset *)
200     QWord(0);
201     (* File Size *)
202     QWord(4000H);
203     (* Maximum VM Protection = VM_PROT_READ, VM_PROT_EXECUTE *)

```

```

204 Set({0, 2});
205 (* Initial VM Protection = VM_PROT_READ, VM_PROT_EXECUTE *)
206 Set({0, 2});
207 (* Number of Sections *)
208 DWord(2);
209 (* Flags *)
210 Set({});
211 (* Sections *)
212 Section0;
213 Section1
214 END Command1;
215
216 PROCEDURE Command2;
217
218 PROCEDURE Section0;
219 BEGIN
220   (* Section Name *)
221   String('__data', 16);
222   (* Segment Name *)
223   String('__DATA', 16);
224   (* Address *)
225   QWord(100004H * 1000H);
226   (* Size in bytes *)
227   QWord(dataLen);
228   (* Offset *)
229   DWord(4000H);
230   (* Alignment = 2^0 = 1 *)
231   DWord(0);
232   (* Relocations Offset *)
233   DWord(0);
234   (* Number of Relocations *)
235   DWord(0);
236   (* Flags = S_REGULAR *)
237   Set({});
238   (* Reserved *)
239   DWord(0);
240   DWord(0);
241   DWord(0)
242 END Section0;
243
244 BEGIN (* Command2 *)
245   (* Command LC_SEGMENT_64 *)
246   DWord(19H);
247   (* Command Size in bytes *)
248   DWord(152);
249   (* Segment Name *)
250   String('__DATA', 16);
251   (* VM Address *)
252   QWord(100004H * 1000H); (* 4 GB + D, where D = 16 KB *)
253   (* VM Size *)
254   QWord(4000H); (* D = 16 KB *)
255   (* File Offset *)

```

```

256   QWord(4000H); (* 16 KB *)
257   (* File Size *)
258   QWord(4000H); (* D = 16 KB *)
259   (* Maximum VM Protection = VM_PROT_READ, VM_PROT_WRITE *)
260   Set({0, 1});
261   (* Initial VM Protection = VM_PROT_READ, VM_PROT_WRITE *)
262   Set({0, 1});
263   (* Number of Sections *)
264   DWord(1);
265   (* Flags *)
266   Set({});
267   (* Sections *)
268   Section0
269 END Command2;
270
271 PROCEDURE Command3;
272 BEGIN
273   (* Command LC_SEGMENT_64 *)
274   DWord(19H);
275   (* Command Size in bytes *)
276   DWord(72);
277   (* Segment Name *)
278   String('__LINKEDIT', 16);
279   (* VM Address *)
280   QWord(100000H * 1000H + 8000H);
281   (* VM Size *)
282   QWord(4000H); (* 16 KB *)
283   (* File Offset *)
284   QWord(8000H); (* 32 KB *)
285   (* File Size *)
286   QWord(176);
287   (* Maximum VM Protection *)
288   DWord(1);
289   (* Initial VM Protection *)
290   DWord(1);
291   (* Number of Sections *)
292   DWord(0);
293   (* Flags *)
294   Set({})
295 END Command3;
296
297 PROCEDURE Command4;
298 BEGIN
299   (* Command LC_DYLD_CHAINED_FIXUPS *)
300   (*DWord(34H);*) (* FIXME 34 00 00 80, what is 8 for? *)
301   Byte(34H); Byte(0); Byte(0); Byte(80H);
302   (* Command Size in bytes *)
303   DWord(16);
304   (* Data Offset *)
305   DWord(8000H);
306   (* Data Size *)
307   DWord(56)

```



```

308 END Command4;
309
310 PROCEDURE Command5;
311 BEGIN
312     (* Command LC_DYLD_EXPORTS_TRIE *)
313     (*DWord(33H);*) (* FIXME what is 8 for? *)
314     Byte(33H); Byte(0); Byte(0); Byte(80H);
315     (* Command Size in bytes *)
316     DWord(16);
317     (* Data Offset *)
318     DWord(8000H + 56);
319     (* Data Size *)
320     DWord(48)
321 END Command5;
322
323 PROCEDURE Command6;
324 BEGIN
325     (* Command LC_SYMTAB *)
326     DWord(2);
327     (* Command Size in bytes *)
328     DWord(24);
329     (* Symbol Table Offset, 8 bytes contain the procedure starts table, Seg. 13 *)
330     DWord(8000H + 56 + 48 + 8); (* 8000H + 112 *)
331     (* Number of Symbols *)
332     DWord(2);
333     (* String Table Offset *)
334     DWord(8000H + 56 + 48 + 8 + 32); (* 8000H + 144 *)
335     (* String Table Size *)
336     DWord(32)
337 END Command6;
338
339 PROCEDURE Command7;
340 BEGIN
341     (* Command LC_DYSYMTAB *)
342     DWord(0BH);
343     (* Command Size in bytes *)
344     DWord(80);
345
346     (* LocSymbol Index *)
347     DWord(0);
348     (* LocSymbol Number *)
349     DWord(0);
350
351     (* Defined ExtSymbol Index *)
352     DWord(0);
353     (* Defined ExtSymbol Number *)
354     DWord(2);
355
356     (* Undefined ExtSymbol Index *)
357     DWord(2);
358     (* Undefined ExtSymbol Number *)
359     DWord(0);

```

```

360
361 (* TOC (Table of Contents) Offset *)
362 DWord(0);
363 (* TOC Entries *)
364 DWord(0);
365
366 (* Module Table Offset *)
367 DWord(0);
368 (* Module Table Entries *)
369 DWord(0);
370
371 (* ExtRef Table Offset *)
372 DWord(0);
373 (* ExtRef Table Entries *)
374 DWord(0);
375
376 (* IndSym Table Offset *)
377 DWord(0);
378 (* IndSym Table Entries *)
379 DWord(0);
380
381 (* ExtReloc Table Offset *)
382 DWord(0);
383 (* ExtReloc Table Entries *)
384 DWord(0);
385
386 (* LocReloc Table Offset *)
387 DWord(0);
388 (* LocReloc Table Entries *)
389 DWord(0);
390 END Command7;
391
392 PROCEDURE Command8;
393 BEGIN
394     (* Command LC_LOAD_DYLINKER *)
395     DWord(0EH);
396     (* Command Size in bytes *)
397     DWord(32);
398     (* String Offset *)
399     DWord(12);
400     (* Name *)
401     String('/usr/lib/dyld', 20)
402 END Command8;
403
404 PROCEDURE Command9;
405 BEGIN
406     (* Command LC_UUID *)
407     DWord(1BH);
408     (* Command Size in bytes *)
409     DWord(24);
410     (* UUID, 16 bytes, TODO generate random *)
411     Word(09A45H); Word(0ADB9H); Word(02E6CH); Word(09B32H);

```

```

412 Word(0E48AH); Word(06EECH); Word(07BCDH); Word(0EC6DH)
413 END Command9;
414
415 PROCEDURE Command10;
416 BEGIN
417     (* Command LC_BUILD_VERSION *)
418     DWord(32H);
419     (* Command Size in bytes *)
420     DWord(32);
421     (* Platform = MacOS *)
422     DWord(1);
423     (* Minimum OS Version = 15.0.0 *)
424     DWord(0F0000H);
425     (* SDK Version, 0 = undefined *)
426     DWord(0);
427     (* Tool Count - number of elements in the array below *)
428     DWord(1);
429
430     (* Array of Tools, 1 element *)
431     (* Tool Type = linker ld *)
432     DWord(3);
433     (* Tool Version = 1115.7.3 *)
434     DWord(45B0703H)
435 END Command10;
436
437 PROCEDURE Command11;
438 BEGIN
439     (* Command LC_SOURCE_VERSION *)
440     DWord(2AH);
441     (* Command Size in bytes *)
442     DWord(16);
443     (* Version = 0.0 *)
444     QWord(0)
445 END Command11;
446
447 PROCEDURE Command12;
448 BEGIN
449     (* Command LC_MAIN *)
450     Byte(28H); Byte(0); Byte(0); Byte(80H);
451     (* Command Size in bytes *)
452     DWord(24);
453     (* Entry Offset *)
454     QWord(4000H - codeSpace);
455     (* Initial Stack Size, may be 0 *)
456     QWord(0)
457 END Command12;
458
459 PROCEDURE Command13;
460 BEGIN
461     (* Command LC_LOAD_DYLIB *)
462     DWord(0CH);
463     (* Command Size in bytes *)

```

```

464     DWord(56);
465     (* String Offset *)
466     DWord(24);
467     (* Time Stamp = 1970-01-01 00:00:02 *)
468     DWord(2);
469     (* Current Version = 1351.0.0 *)
470     DWord(5470000H);
471     (* Compatibility Version *)
472     DWord(10000H);
473     (* Name *)
474     String('/usr/lib/libSystem.B.dylib', 32)
475 END Command13;
476
477 PROCEDURE Command14;
478 BEGIN
479     (* Command LC_FUNCTION_STARTS *)
480     DWord(26H);
481     (* Command Size in bytes *)
482     DWord(16);
483     (* Data Offset *)
484     DWord(8000H + 104);
485     (* Data Size *)
486     DWord(8)
487 END Command14;
488
489 PROCEDURE Command15;
490 BEGIN
491     (* Command LC_DATA_IN_CODE *)
492     DWord(29H);
493     (* Command Size in bytes *)
494     DWord(16);
495     (* Data Offset *)
496     DWord(8000H + 112);
497     (* Data Size *)
498     DWord(0)
499 END Command15;
500
501 (** Outputs Mach-O commands to the file. *)
502 PROCEDURE Commands;
503 BEGIN
504     Command0;
505     Command1;
506     Command2;
507     Command3;
508     Command4;
509     Command5;
510     Command6;
511     Command7;
512     Command8;
513     Command9;
514     Command10;
515     Command11;

```

```

516     Command12;
517     Command13;
518     Command14;
519     Command15
520 END Commands;
521
522 (** Outputs 56 bytes to the file. *)
523 PROCEDURE ChainedFixups;
524 BEGIN
525     (* Fixups Version *)
526     DWord(0);
527     (* Starts Offset *)
528     DWord(20H);
529     (* Imports Offset *)
530     DWord(30H);
531     (* Symbols Offset *)
532     DWord(30H);
533     (* Imports Count *)
534     DWord(0);
535     (* Imports Format *)
536     DWord(1);
537     (* Symbols Format *)
538     DWord(0);
539
540     (* Starts in Image *)
541     (* Segment Count *)
542     DWord(0);
543     (* Segment Info Offset - DWord array (empty) *)
544
545     (* Starts in Segment *)
546     (* Size *)
547     DWord(3);
548     (* Page Size *)
549     DWord(0);
550     (* Pointer Format *)
551     DWord(0);
552     (* Segment Offset *)
553     DWord(0);
554     (* Maximum Valid Pointer *)
555     DWord(0);
556     (* Page Count *)
557     Word(0);
558     (* Page Start - DWord array *)
559     Word(0)
560 END ChainedFixups;
561
562 (** Outputs 48 bytes to the file. *)
563 PROCEDURE ExportsTrie;
564 BEGIN
565     Byte(0); Byte(1); Byte(5FH); Byte(0);
566     Byte(12H); Byte(0); Byte(0); Byte(0);
567

```

```

568   Byte(0);   Byte(2);   Byte(0);   Byte(0);
569   Byte(0);   Byte(3);   Byte(0);   Byte(9CH);
570   Byte(7FH); Byte(0);   Byte(0);   Byte(2);
571
572   String('__mh_execute_header', 0);
573   Byte(0);   Byte(9);
574
575   String('start', 0);
576   Byte(0);   Byte(0DH); Byte(0)
577 END ExportsTrie;
578
579 PROCEDURE FunctionStarts;
580 BEGIN
581   Byte(9CH); Byte(7FH); Word(0); DWord(0)
582 END FunctionStarts;
583
584 PROCEDURE SymbolTable;
585 BEGIN
586   (* Proper Symbol Table *)
587   DWord(2);   DWord(10010FH); DWord(0);   DWord(1);
588   DWord(16H); DWord(10FH);   DWord(3F9CH); DWord(1);
589
590   (* Name Strings *)
591   Byte(20H); Byte(0);
592   String('__mh_execute_header', 0); Byte(0);
593   String('__start', 0); DWord(0)
594 END SymbolTable;
595
596 PROCEDURE Output;
597 BEGIN
598   Header;
599   Commands;
600
601   (* Zero-out until position 4000H - codeSpace *)
602   Zeros(4000H - codeSpace - Files.Pos(r));
603
604   (* Machine code *)
605   Generator.OutputCode(r);
606   (* Unwind Info *)
607   Zeros(codeSpace - codeLen);
608
609   (* Data (global variables) *)
610   Generator.OutputData(r);
611   Zeros(4000H - dataLen - 1);
612   Byte(0AH); (* For Out.Ln *)
613
614   ChainedFixups;
615   ExportsTrie;
616   FunctionStarts;
617   SymbolTable
618 END Output;
619

```

```

620 PROCEDURE OpenOutputFile;
621 BEGIN
622   F := Files.New(fname);
623   IF F = NIL THEN
624     Machine.Error(Errors.outputFile)
625   ELSE
626     Files.Set(r, F, 0)
627   END
628 END OpenOutputFile;
629
630 PROCEDURE ChmodAndSign;
631 VAR s: ARRAY 1024 OF CHAR;
632     err: INTEGER;
633 BEGIN
634   (* Chmod *)
635   s := 'chmod +x ';
636   Strings.Append(fname, s);
637   err := Machine.Exec(s);
638   IF err # 0 THEN
639     Machine.Error(Errors.chmodFailed)
640   ELSE
641     (* Sign *)
642     s := 'codesign -s - -f --timestamp=none ';
643     Strings.Append(fname, s);
644     err := Machine.Exec(s);
645     IF err # 0 THEN
646       Machine.Error(Errors.signFailed)
647     END
648   END
649 END ChmodAndSign;
650
651 PROCEDURE Link*(exeFname: ARRAY OF CHAR);
652 BEGIN
653   Strings.Copy(exeFname, fname);
654   codeLen := Generator.CodeLength();
655   dataLen := Generator.DataLength();
656   OpenOutputFile;
657   IF ¬Machine.hadErrors THEN
658     Output;
659   IF Machine.hadErrors THEN
660     Files.Close(F)
661   ELSE
662     Files.Register(F);
663     ChmodAndSign
664   END
665 END
666 END Link;
667
668 END Linker.

```

Листинг 60. Полный исходный текст модуля MocErrors (коды и текст ошибок)

```
1 MODULE MocErrors;
2 IMPORT Out, Strings, Int;
3
4 CONST
5   (** Errors **)
6   none*           = 0;
7   longIdent*      = 1;
8   longStr*        = 2;
9   unclosedStr*    = 3;
10  intOverflow*     = 4;
11  manyDigits*     = 5;
12  badInt*          = 6;
13  badReal*        = 7;
14  badChar*        = 8;
15  largeExponent*  = 9;
16  modNameMismatch* = 10;
17  multipleImport* = 11;
18  multipleDefs*   = 12;
19  aliasTaken*     = 13;
20  notConstant*    = 14;
21  notType*        = 15;
22  notProcedure*   = 16;
23  procCallFunc*   = 17;
24  undefinedType*  = 18;
25  undefinedIdent* = 19;
26  incompatibleTypes* = 20;
27  badMonadicOperand* = 21;
28  badDyadicOperands* = 22;
29  notImplemented* = 23;
30  fewArguments*   = 24;
31  manyArguments*  = 25;
32
33  noDigit*        = 70;
34  noDeclaration*  = 71;
35  noStatement*    = 72;
36  noExpression*   = 73;
37  noFactor*       = 74;
38  noType*         = 75;
39
40  (** General errors **)
41  outputFile*     = 101;
42  outOfMemory*    = 102;
43  chmodFailed*    = 103;
44  signFailed*     = 104;
45
46  (** Lexical symbols **)
47  null = 0; times = 1; rdiv = 2; div = 3; mod = 4;
```



```

48 and = 5; plus = 6; minus = 7; or = 8; eql = 9;
49 neq = 10; lss = 11; leq = 12; gtr = 13; geq = 14;
50 in = 15; is = 16; arrow = 17; period = 18;
51 char = 20; int = 21; real = 22; false = 23; true = 24;
52 nil = 25; string = 26; not = 27; lparen = 28; lbrak = 29;
53 lbrace = 30; ident = 31;
54 if = 32; while = 34; repeat = 35; case = 36; for = 37;
55 comma = 40; colon = 41; becomes = 42; upto = 43; rparen = 44;
56 rbrak = 45; rbrace = 46; then = 47; of = 48; do = 49;
57 to = 50; by = 51; semicolon = 52; end = 53; bar = 54;
58 else = 55; elsif = 56; until = 57; return = 58;
59 array = 60; record = 61; pointer = 62; const = 63; type = 64;
60 var = 65; procedure = 66; begin = 67; import = 68; module = 69; eot = 70;

```

```

61
62 PROCEDURE PrintSymbol*(sym: INTEGER);
63 BEGIN
64     IF      sym = string THEN Out.String('String')
65     ELSIF  sym = int    THEN Out.String('Integer')
66     ELSIF  sym = real   THEN Out.String('Real')
67     ELSIF  sym = ident  THEN Out.String('Identifier')
68     ELSIF  sym = times  THEN Out.String('Times')
69     ELSIF  sym = rdiv   THEN Out.String('Rdiv')
70     ELSIF  sym = div    THEN Out.String('Div')
71     ELSIF  sym = mod    THEN Out.String('Mod')
72     ELSIF  sym = and    THEN Out.String('And')
73     ELSIF  sym = plus   THEN Out.String('Plus')
74     ELSIF  sym = minus  THEN Out.String('Minus')
75     ELSIF  sym = or     THEN Out.String('OR')
76     ELSIF  sym = eql    THEN Out.String('=')
77     ELSIF  sym = neq    THEN Out.String('#')
78     ELSIF  sym = lss    THEN Out.String('<')
79     ELSIF  sym = leq    THEN Out.String('<=')
80     ELSIF  sym = gtr    THEN Out.String('>')
81     ELSIF  sym = geq    THEN Out.String('>=')
82     ELSIF  sym = in     THEN Out.String('IN')
83     ELSIF  sym = is     THEN Out.String('IS')
84     ELSIF  sym = arrow  THEN Out.String('↑')
85     ELSIF  sym = period THEN Out.String('.')
86     ELSIF  sym = char   THEN Out.String('CHAR')
87     ELSIF  sym = false  THEN Out.String('FALSE')
88     ELSIF  sym = true   THEN Out.String('TRUE')
89     ELSIF  sym = nil    THEN Out.String('NIL')
90     ELSIF  sym = not    THEN Out.String('¬')
91     ELSIF  sym = lparen THEN Out.String('(')
92     ELSIF  sym = lbrak  THEN Out.String('[')
93     ELSIF  sym = lbrace THEN Out.String('{')
94     ELSIF  sym = if     THEN Out.String('IF')
95     ELSIF  sym = while  THEN Out.String('WHILE')
96     ELSIF  sym = repeat THEN Out.String('REPEAT')
97     ELSIF  sym = case   THEN Out.String('CASE')
98     ELSIF  sym = for    THEN Out.String('FOR')
99     ELSIF  sym = comma  THEN Out.String(',')

```

```

100  ELSIF sym = colon      THEN Out.String(':')
101  ELSIF sym = becomes   THEN Out.String(':=')
102  ELSIF sym = upto      THEN Out.String('...')
103  ELSIF sym = rparen    THEN Out.String(')')
104  ELSIF sym = rbrak     THEN Out.String(']')
105  ELSIF sym = rbrace    THEN Out.String('}')
106  ELSIF sym = then      THEN Out.String('THEN')
107  ELSIF sym = of        THEN Out.String('OF')
108  ELSIF sym = do        THEN Out.String('DO')
109  ELSIF sym = to        THEN Out.String('TO')
110  ELSIF sym = by        THEN Out.String('BY')
111  ELSIF sym = semicolon THEN Out.String(';')
112  ELSIF sym = end       THEN Out.String('END')
113  ELSIF sym = bar       THEN Out.String('|')
114  ELSIF sym = else      THEN Out.String('ELSE')
115  ELSIF sym = elsif     THEN Out.String('ELSIF')
116  ELSIF sym = until     THEN Out.String('UNTIL')
117  ELSIF sym = return    THEN Out.String('RETURN')
118  ELSIF sym = array     THEN Out.String('ARRAY')
119  ELSIF sym = record    THEN Out.String('RECORD')
120  ELSIF sym = pointer   THEN Out.String('POINTER')
121  ELSIF sym = const     THEN Out.String('CONST')
122  ELSIF sym = type      THEN Out.String('TYPE')
123  ELSIF sym = var       THEN Out.String('VAR')
124  ELSIF sym = procedure THEN Out.String('PROCEDURE')
125  ELSIF sym = begin     THEN Out.String('BEGIN')
126  ELSIF sym = import    THEN Out.String('IMPORT')
127  ELSIF sym = module    THEN Out.String('MODULE')
128  ELSIF sym = eot       THEN Out.String('eot')
129  ELSE Out.String('symbol #'); Out.Int(sym, 0)
130  END
131 END PrintSymbol;
132
133 PROCEDURE Message*(errno: INTEGER; VAR s: ARRAY OF CHAR);
134 BEGIN
135     IF      errno = longIdent      THEN s := 'Identifier too long'
136     ELSIF  errno = longStr        THEN s := 'String too long'
137     ELSIF  errno = unclosedStr    THEN s := 'Unclosed string literal'
138     ELSIF  errno = intOverflow    THEN s := 'Integer overflow'
139     ELSIF  errno = manyDigits     THEN s := 'Too many digits'
140     ELSIF  errno = badInt        THEN s := 'Bad integer literal'
141     ELSIF  errno = badReal       THEN s := 'Bad real literal'
142     ELSIF  errno = badChar       THEN s := 'Unexpected input character'
143     ELSIF  errno = largeExponent THEN s := 'Exponent too large'
144     ELSIF  errno = modNameMismatch THEN s := 'Module name does not match'
145     ELSIF  errno = multipleImport THEN s := 'Module already imported'
146     ELSIF  errno = multipleDefs  THEN s := 'Object already defined'
147     ELSIF  errno = aliasTaken    THEN s := 'Module alias already taken'
148     ELSIF  errno = notConstant   THEN s := 'Expression must be constant'
149     ELSIF  errno = notType       THEN s := 'This is not a type'
150     ELSIF  errno = notProcedure  THEN s := 'This is not a procedure'
151     ELSIF  errno = procCallFunc  THEN s := 'Procedure call of a function'

```

```

152  ELSIF errno = undefinedType      THEN s := 'This type is undefined'
153  ELSIF errno = undefinedIdent     THEN s := 'Undefined identifier'
154  ELSIF errno = incompatibleTypes  THEN s := 'Incompatible types'
155  ELSIF errno = badMonadicOperand THEN
156      s := 'Operation not applicable to this type'
157  ELSIF errno = badDyadicOperands THEN
158      s := 'Operation not applicable to these types'
159  ELSIF errno = notImplemented      THEN s := 'Feature not implemented'
160  ELSIF errno = fewArguments        THEN s := 'Too few arguments to procedure'
161  ELSIF errno = manyArguments       THEN s := 'Too many arguments to procedure'
162  ELSIF errno = noDigit             THEN s := 'Digit expected'
163  ELSIF errno = noDeclaration       THEN
164      s := 'CONST, TYPE, VAR, BEGIN, RETURN or END expected'
165  ELSIF errno = noStatement         THEN s := 'Statement expected'
166  ELSIF errno = noExpression        THEN s := 'Expression expected'
167  ELSIF errno = noFactor            THEN s := 'Factor expected'
168  ELSIF errno = noType              THEN
169      s := 'ARRAY, RECORD, POINTER, PROCEDURE or identifier expected'
170  ELSIF errno = outputFile          THEN s := 'Could not open output file'
171  ELSIF errno = outOfMemory         THEN s := 'Out of code memory'
172  ELSIF errno = chmodFailed         THEN s := 'ChMod command failed'
173  ELSIF errno = signFailed          THEN s := 'CodeSign command failed'
174  ELSE
175      s := '#ERR'; Int.Append(errno, s)
176  END
177  END Message;
178
179  END MocErrors.

```

— для заметок —

Артур Игоревич ЕФИМОВ
ПОСТРОЕНИЕ КОМПИЛЯТОРА
С ЯЗЫКА ПРОГРАММИРОВАНИЯ ОБЕРОН
ДЛЯ ПРОЦЕССОРА ARM64

Магистерская диссертация

Подписано к печати 30.01.26
Бесплатно Рига. Латв. ССР